Apostila de Instrutor

Aluno, se você receber esta apostila por engano, favor comunicar a administração



INFRAESTRUTURA ÁGIL COM DOCKER E DOCKER SWARM

Curso DO-26



Caelum

caelum
ensino e inovação

Conheça também:

alura

alura.com.br

Casa do Código
Livros e Tecnologia

casadocodigo.com.br

Blog da Caelum	Newsletter
blog.caelum.com.br	caelum.com.br/newsletter

Facebook	Twitter
facebook.com.br/caelumbr	twitter.com/caelum

Sumário

1 Sobre o curso **1**

1.1 Uma breve introdução 1

1.2 Infraestrutura ágil e Docker 1

1.3 Para quem é esse curso 2

1.4 Código fonte do curso 2

1.5 Bibliografia complementar 2

2 O que são containers? **3**

2.1 O que é Docker? 3

2.2 Quais problemas o Docker resolve? 4

2.3 Máquina Virtual vs Container 5

2.4 Docker CE vs Docker EE 8

2.5 Docker dentro cultura DevOps 8

2.6 Instalando o Docker no Linux 8

2.7 Instalando no Windows 10

2.8 Instalando no Mac 11

3 Conhecendo o Docker Daemon **12**

3.1 O primeiro container 12

3.2 Um pouco de arquitetura 12

3.3 Subindo o primeiro container 13

3.4 Exercício: O primeiro container 14

3.5 Para saber mais: container ls 15

3.6 Entendendo o ciclo de vida dos containers 15

3.7 Mais informações sobre o docker container ls 16

3.8 Inspecionando conteúdo dos containers 17

3.9 Exercício: Ciclo de vida do container 18

4 Criando nossas próprias imagens **20**

4.1 Entendendo o conceito de imagem 20

4.2 Exercício: Reutilizando camadas 22
4.3 Drivers de armazenamento 22
4.4 A estratégia Copy-on-Write 23
4.5 Imagens x Containers 23
4.6 Listando imagens 25
4.7 Baixando e removendo imagens 25
4.8 O que é a camada de escrita? 26
4.9 Comitando imagem a partir da container layer 26
4.10 Criando imagem a partir do Dockerfile 27
4.11 Mais instruções do Dockerfile 29
4.12 Exercício: Criando nosso primeiro Dockerfile 30
4.13 Acessando portas do container a partir da máquina host 32
4.14 Exercício: Acessando aplicações dentro do container 33
4.15 Versionando imagem com tags 34
4.16 Exercício: Versionamento da imagem 35
4.17 Multi-stage builds 36

5 Publicando imagens no Docker Hub 38
5.1 O que é o Docker Hub? 38
5.2 Docker Hub vs DTR 38
5.3 Exercício: Criando sua conta no Docker Hub 39
5.4 Para saber mais: registry 41
5.5 Publicando nossas imagens em um registro 42
5.6 Buscando imagens de um registro 43
5.7 Exercício: Manipulando imagens em um registro 43

6 Containers e persistência de dados 46
6.1 Camadas e imagens 46
6.2 O problema da camada de escrita 47
6.3 Exercício opcional: Conhecendo a camada de escrita 49
6.4 Criando o primeiro volume 49
6.5 Exercício: Usando volumes 51
6.6 Conhecendo o bind mount 53
6.7 Exercício: Persistindo dados através de bind mounts 54
6.8 Trabalhando com TMPFS 55
6.9 Variáveis de ambiente 56
6.10 Variáveis de construção: ARG 56
6.11 Exercício: Trabalhando variáveis de ambiente. 57

7 Comunicando containers **60**

7.1 Conhecendo a rede bridge 60

7.2 Conhecendo a rede host 61

7.3 Para saber mais: redes MACVLAN 62

7.4 Criando nossa própria rede 62

7.5 Exercício: Conceitos de redes do Docker 64

7.6 Acessando serviços em outros containers 65

7.7 Exercício: Comunicando diferentes containers 66

7.8 Criando subredes com o Docker 67

7.9 Exercício Opcional: Criando uma subrede 68

8 Utilizando logs e debug **70**

8.1 Introdução e problemas 70

8.2 Exibindo e salvando logs 70

8.3 Conhecendo os diferentes drivers de log 71

8.4 Exercício: Utilizando comandos de logs 72

8.5 Para saber mais: Alterando o driver de logging 73

8.6 Utilizando o Docker em modo debug 73

8.7 Exercício em casa: ativando o modo debug 75

8.8 Para saber mais: usando o daemon em modo manual 76

9 Coordenando containers com Docker Compose **77**

9.1 Conhecendo o Docker Compose 78

9.2 Executando com docker-compose 80

9.3 Subindo vários containers 81

9.4 Build da imagem com Docker Compose 82

9.5 Exercício: Primeiro arquivo de composição 84

9.6 Ordem dos serviços 84

9.7 Exercício: Criando múltiplos serviços 85

9.8 Desafio Docker Compose 87

10 Isolando ambientes com a Docker Machine **95**

10.1 O que é Docker Machine? 95

10.2 Criando ambientes virtuais para execução 95

10.3 Gerenciando e acessando a máquina virtual 96

10.4 Exercício: Criando ambientes com a Docker Machine 97

10.5 Para saber mais: Docker Machine e AWS 98

10.6 Para saber mais: Docker Machine e Windows 99

11 Gerenciando containers com Docker Swarm **100**

11.1 Escalando a aplicação com conceito de Microservices 100
11.2 Docker e Microservices 100
11.3 O Docker Compose para gerenciar? 101
11.4 Docker Swarm - Orquestrando containers 101
11.5 Criando nosso primeiro cluster 103
11.6 Nós managers vs nós workers 105
11.7 Exercício: Criando o primeiro swarm. 106
11.8 Entendendo o driver de rede overlay 106
11.9 Subindo container como serviço 107
11.10 Balanceando cargas entre containers 108
11.11 Exercício: Avançando com conceitos do swarm 109
11.12 Serviços replicados vs Serviços globais 110
11.13 Implantando múltiplos serviços com stacks 111
11.14 Para saber mais: Kubernetes 113
11.15 Para saber mais: Volumes no Swarm 113
11.16 Desafio Docker Swarm 113

Versão: 24.8.11

CAPÍTULO 1

SOBRE O CURSO

1.1 UMA BREVE INTRODUÇÃO

O Docker é um software *Open Source*, criado pela empresa **Docker Inc**, antiga **dotCloud**, que permite a criação e isolamento de ambientes virtuais. Para tal, ele faz uso de **Linux Containers**. Um **container** contém tudo o que é necessário para executar um ou mais processos, isolados do resto do sistema. Neste curso aprenderemos como todo esse isolamento é feito, além de colocar em prática conceitos vistos sobre esta ferramenta.

Além disso, o Docker permite a criação de um pacote, chamado de **imagem**, que pode ser facilmente construído e distribuído. Essa imagem possui todas as dependências para rodar a aplicação e é independente da plataforma de desenvolvimento (Java, PHP, C# etc).

Por sua vez, o **Docker Swarm** ajuda no gerenciamento e se preocupa com a distribuição e coordenação dos containers em várias máquinas. Através dele, temos as garantias que os containers continuem rodando (**disponibilidade**) e que atendam a demanda (**escalabilidade**).

Construiremos nossas próprias imagens e veremos como distribuí-las, como configurar e executar os containers, além de gerenciá-los dentro de um *cluster* com o **Docker Swarm**.

1.2 INFRAESTRUTURA ÁGIL E DOCKER

O Docker permite com que definamos a nossa infraestrutura dentro de arquivos simples de texto, chamados de **Dockerfile**. O *Dockerfile* possui as etapas e configurações necessárias para criar sua imagem.

O arquivo deve ser *"commitado"* e versionado dentro do repositório fonte e deve evoluir em conjunto com o código fonte da aplicação. Assim, fica transparente como criar e rodar a aplicação, sem depender de uma infraestrutura complexa.

O Docker se encaixa perfeitamente no processo de **Infraestrutura como Código** (*Infrastructure as Code* - **IaC**), que faz parte do movimento **DevOps**, que visa criar uma cultura de colaboração entre os times de desenvolvimento e operações.

Você pode ver mais sobre *Infraestrutura como Código* no artigo de **Martin Fowler**: https://martinfowler.com/bliki/InfrastructureAsCode.html

1.3 PARA QUEM É ESSE CURSO

Este curso é para pessoas que trabalham com desenvolvimento e/ou infraestrutura e querem aprender sobre o **Docker** e **Docker Swarm**, para melhorar o processo de desenvolvimento e entrega de software em suas empresas. Ao sair do curso, você estará seguro para usar e aplicar o Docker dentro do seu projeto de software.

1.4 CÓDIGO FONTE DO CURSO

Os arquivos e códigos fontes que serão utilizados como exemplo nesse curso estão disponíveis no **GitLab**, no seguinte endereço:

https://gitlab.com/aovs/projetos-cursos/do26-projeto

Fique à vontade para navegar pelo código fonte do projeto e baixá-lo para realizar os exercícios em seu computador.

1.5 BIBLIOGRAFIA COMPLEMENTAR

- *Docker Deep Dive - Nigel Poulton*

Capítulo 2

O QUE SÃO CONTAINERS?

2.1 O QUE É DOCKER?

Quando falamos em desenvolvimento de software, é comum termos diversos ambientes, por exemplo:

- Desenvolvimento
- Teste
- Homologação
- Produção

Outra coisa comum no mundo de desenvolvimento, é ter divergências entre estes ambientes.

Quem nunca ouviu a frase: "Na minha máquina funciona!"? O Docker chegou com tudo para resolver esse problema!

O Docker é um sistema de virtualização não convencional. E o que isso quer dizer? Em virtualizações convencionais temos um software instalado na máquina Host que gerencia as máquinas virtuais (ex.: VirtualBox, VMWare, Parallels e etc...) e em cada máquina virtual temos uma instalação completa do S.O. que queremos virtualizar, além de ter o próprio hardware virtualizado.

Se por exemplo, precisarmos de uma biblioteca comum para todas as máquinas virtuais, será necessário que eu instale em cada uma delas, mas já com o Docker a abordagem é diferente, ele utiliza o conceito de container. O que isso quer dizer? Começaremos a entender melhor agora a diferença entre containers e a virtualização tradicional.

Compreendendo o conceito de containers

Se pensarmos em transporte de cargas, o container foi uma revolução nessa área, pois antes dele o tempo de carga e descarga de um navio era gigantesco, e o trabalho era feito manualmente. Além disso, tínhamos as chances de perdas de mercadorias por conta de quebras ou deterioração.

Com a chegada dos containers foi possível transportar mercadorias de forma segura, fácil manipulação e com pouco, ou nenhum, trabalho braçal. É exatamente esse conceito que o Docker visa aplicar em nossas aplicações.

Porém, como isso funciona na prática com o Docker?

Ganhos ao usar containers do Docker

Imagine nosso software como uma mercadoria a ser transportarda, por exemplo, do ambiente de desenvolvimento para o ambiente de produção.

Para fazer isso, precisamos garantir que nosso ambiente de **produção** tenha todos os pré-requisitos instalados, de preferência uma versão do S.O. parecida com a do ambiente de **desenvolvimento**, entre outros cuidados que devem ser tomados (relacionados a permissões, serviços dependentes etc...).

Com isso, não precisamos nos preocupar com pré-requisitos instalados em outro ambiente, versão do S.O., permissões e, se quisermos, podemos ter containers para os serviços dependentes também. Dessa forma minimizamos muito a divergência entre os ambientes.

Mas como o Docker faz isso?

A ideia de containers é muito antiga, bem mais que o Docker. No início, o Docker utilizava uma tecnologia chamada LXC (Linux Container).

O projeto LXC utiliza internamente diversas funcionalidades presentes no Kernel do Linux. Abaixo, é possível ver algumas dessas funcionalidades:

- chroot - Reponsável por mapear os diretórios do S.O. e criar o ponto de montagem (/, /etc, /dev, /proc entre outros).
- cgroup - Reponsável por controlar os recursos por processo. Com ele podemos por exemplo limitar o uso de memória e/ou processador para um processo específico.
- kernel namespace - Com ele, podemos isolar processos, pontos de montagem, usuários, interfaces de redes e até mesmo o próprio kernel. Com esse isolamento, conseguimos a sensação de estar usando uma máquina diferente da máquina host. Pois enxergamos somente o ponto de montagem especifico e processos especificos.
- kernel capabilities - Entre outras coisas, conseguimos rodar alguns comandos de forma privilegiada.

2.2 QUAIS PROBLEMAS O DOCKER RESOLVE?

O Docker resolve problemas clássicos no desenvolvimento de software, como:

- dependências de aplicativos ausentes ou incorretas, como bibliotecas, interpretadores, código/binários, usuários. Exemplo: executando um aplicativo Java na versão da VM errada.
- conflitos entre programas em execução no mesmo computador, como dependências ou portas utilizadas. Exemplo: caminhos globais de bibliotecas e conflitos nas versões, ou vários processos tentando usar a porta 80.
- falta de controle dos recursos como CPU e memória que um aplicativo pode usar. Exemplo: Conter um programa descontrolado que consome toda a memória da máquina.

- scripts ausentes, complicados ou imaturos para instalar, iniciar, parar e desinstalar um aplicativo. Exemplo: vários scripts bash que precisam ser executados em ordem ou só são entendidos por alguns desenvolvedores.

O Docker resolve esses problemas inovando em duas áreas:

- empacotamento da aplicação de maneira uniforme.
- isolamento e gerenciamento do processo.

2.3 MÁQUINA VIRTUAL VS CONTAINER

Para fugir de criar e manter vários ambientes, e também de servidores ociosos com alto tempo e custo de subir e manter aplicações em servidores físicos, surgiu como solução a *virtualização*, ou seja, surgiram as máquinas virtuais.

A criação das máquinas virtuais foi possível graças ao Hypervisor, que funciona em cima do sistema operacional, permitindo a virtualização dos recursos físicos do nosso sistema. Assim, criamos uma máquina virtual que tem acesso a uma parte do nosso HD, memória RAM e CPU, criando um computador dentro do outro.

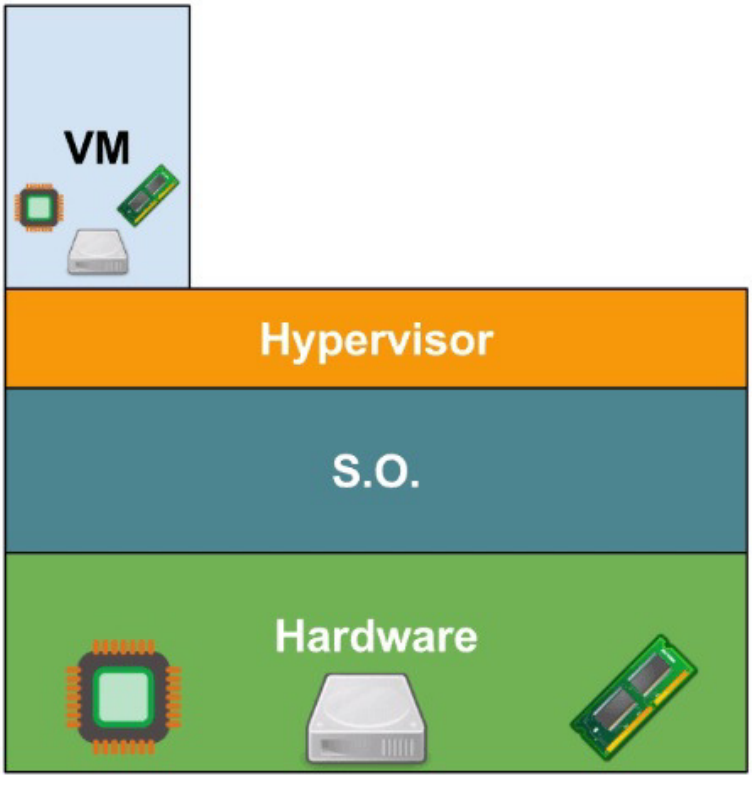


Figura 2.1: Tecnologia Hypervisor

E, se tivermos uma máquina virtual que acessa parte do nosso hardware, conseguimos colocar dentro dela nossas aplicações. Podemos replicar este pensamento para diversas máquinas virtuais.

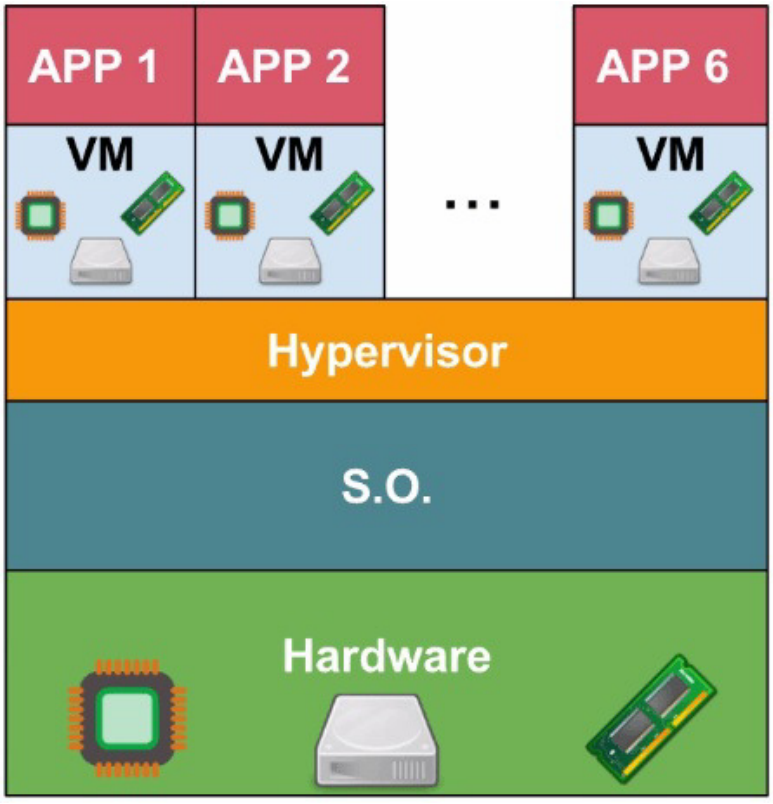


Figura 2.2: Aplicação na VM

Assim, reduzimos a quantidade de servidores e, consequentemente, os custos com luz e rede. Além disso, dividimos melhor o nosso hardware, aproveitando melhor os seus recursos e diminuindo a ociosidade, padronizando ambientes de homologação e desenvolvimento.

Apesar de resolver os problemas do uso de vários servidores físicos, as máquinas virtuais também possuem problemas. Podemos perceber, por exemplor que para podermos hospedar a nossa aplicação em uma máquina virtual, também precisamos instalar um sistema operacional nela.

Com o Docker, graças ao uso de namespaces, não precisamos instalar qualquer sistema operacional dentro do container, pois o compartilhamento de kernel é feito de maneira automática.

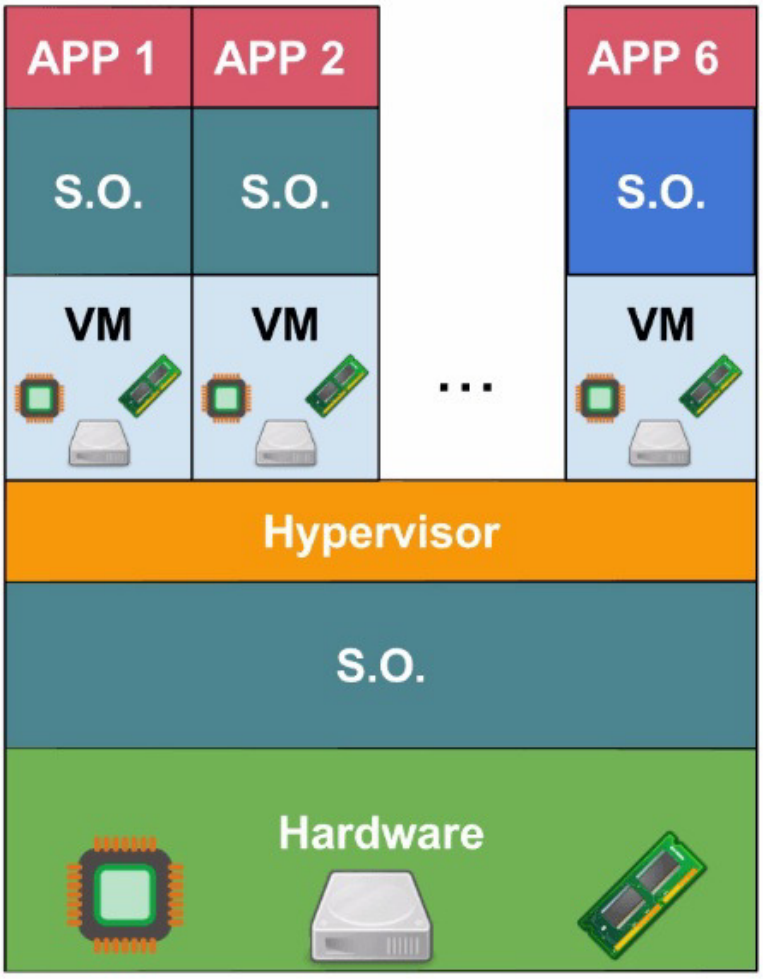


Figura 2.3: Vários Sistemas operacionais

Cada aplicação necessita de um sistema operacional para poder ser executada. Esses sistemas podem ser diferentes, e, apesar dos sistemas operacionais serem um software, eles possuem um custo de memória RAM, disco e processamento. Ou seja, precisamos de uma capacidade mínima para manter as funcionalidades de um sistema operacional, aumentando o seu custo de manutenção a cada sistema que tivermos. E mais, temos o custo de virtualização do Hypervisor, que virtualiza todo nosso hardware, o que acaba por tornar as máquinas virtuais algo de custo bem alto.

Além disso, há um custo de configuração, isto é, liberar portas, instalar alguma biblioteca específica, etc. É exigida toda a configuração que um sistema operacional pede. Também devemos sempre estar atentos à sua segurança, mantendo o sistema de cada máquina virtual sempre atualizado.

Muitas vezes, o tempo voltado para a manutenção das máquinas virtuais era o mesmo tempo voltado para a nossa aplicação em si. Ou seja, acabávamos dividindo o valor da nossa empresa, ao invés de focar somente nas aplicações, dividíamos o trabalho com a manutenção dos sistemas operacionais.

Então, como melhorar essa situação? Pensando nisso, surgiram os containers Linux e o Docker para simplificar toda essa configuração e gerenciamento.

2.4 DOCKER CE VS DOCKER EE

Atualmente o Docker está disponível em duas versões: Docker Community Edition(CE) e o Docker Enterprise Edition(EE), que foi adquirido pela empresa Mirantis, inc em novembro de 2019.

Ambas as versões têm acesso a toda a API, com a diferença sendo o perfil desejado de aplicações. No **Docker EE**, temos um ambiente homologado pela Docker com toda infraestrutura certificada, a segurança também é voltada para o mundo corporativo. Também na versão **Enterprise**, ganhamos acesso a um registro privado para a empresa, o *Docker Trusted Registry*, e contamos com a possibilidade de trablhar com *Role-Based Access Control*, ou seja, podemos criar e manipular papéis para os funcionários que trabalham com o Docker em nossa empresa. Outro ganho interessante é o *Universal Control Plane*, com ele, podemos gerenciar o Docker e seus containers a partir de uma dashboard de fácil acesso e manipulação.

Ou seja, sempre dê a preferência para utilizar o **Docker EE** para manter seu projeto em produção.

2.5 DOCKER DENTRO CULTURA DEVOPS

Na essência, o DevOps é um movimento cultural que visa a colaboração entre equipes de TI para agilizar a entrega de software sem diminuir a qualidade. Nesse momento entra o Docker, que facilita o compartilhamento de software e suas dependências entre os ambientes e diferentes equipes.

Com o Docker, as configurações de construção do software são simples arquivos de texto, que podem fazer parte do repositório de código desde o inicio, ficando transparente para todos os envolvidos. Indiretamente, os containers e o Docker estreitam a relação entre os desenvolvedores e os responsáveis pelo operacional, gerando uma colaboração eficiente.

2.6 INSTALANDO O DOCKER NO LINUX

Neste passo a passo, será visto como instalar o Docker no Ubuntu 64 bits. Todos os comandos listados devem ser executados no seu terminal.

O **Docker** é dividido em três canais de *releases*: Stable, Test e Nightly. Durante o curso, utilizaremos a versão Stable:

Antes de mais nada, remova possíveis versões antigas do Docker:

```
sudo apt-get remove docker docker-engine docker.io
```

Depois, atualize o banco de dados de pacotes e garanta que o `curl` está instalado:

```
sudo apt-get update
```

e

```
sudo apt-get install curl
```

Agora, adicione ao sistema a chave GPG oficial do repositório Docker:

```
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo apt-key add -
```

Adicione o repositório do Docker às fontes do **apt** para que possamos posteriormente fazer o download de todas as ferramentas necessárias:

```
sudo add-apt-repository \
   "deb [arch=amd64] https://download.docker.com/linux/ubuntu \
   $(lsb_release -cs) \
   stable"
```

Atualize o banco de dados de pacotes, pare ter acesso aos pacotes do Docker a partir do novo repositório adicionado:

```
sudo apt-get update
```

Por fim, instale o pacote **docker-ce**:

```
sudo apt-get install docker-ce
```

Caso você queira, você pode verificar se o Docker foi instalado corretamente verificando a sua versão:

```
sudo docker version
```

E para executar o Docker sem precisar de **`sudo`** , adicione o seu usuário ao grupo **`docker`** :

```
sudo usermod -aG docker $USER
```

Será necessário deslogar e logar o usuário para que o passo anterior surta efeito.

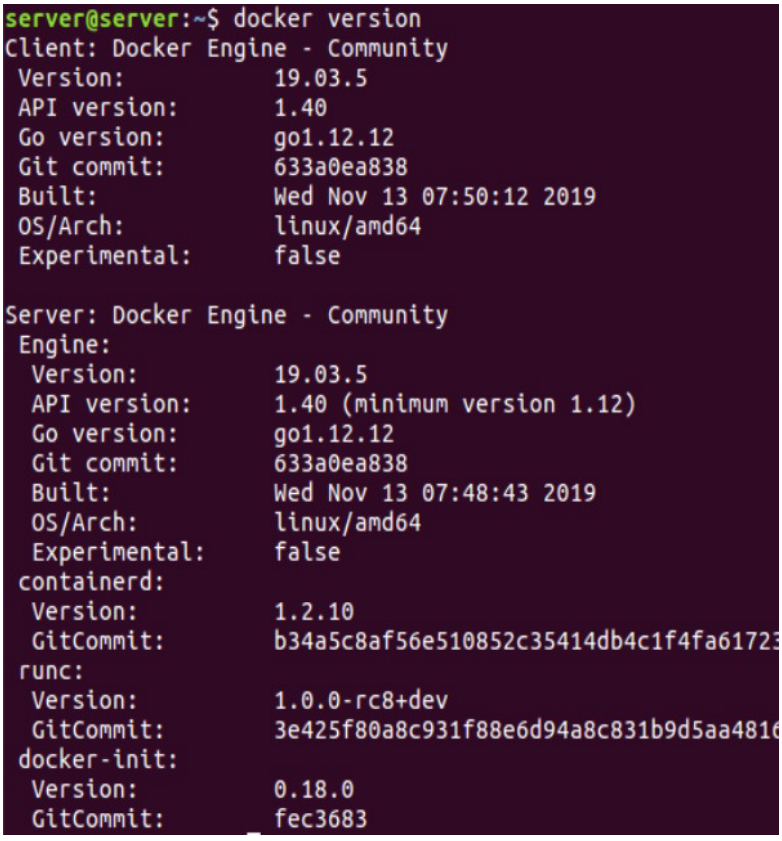


Figura 2.4: Comando: docker version

2.7 INSTALANDO NO WINDOWS

O Docker Desktop for Windows é a versão comunitária do Docker para Microsoft Windows. Você pode baixar o Docker Desktop for Windows no Docker Hub. Crie uma conta no Docker Hub e acesse https://docs.docker.com/docker-for-windows/install/. Mas, antes de instalar o Docker, primeiramente devemos nos atentar aos requisitos:

- Arquitetura 64 bits.
- Versão do Windows **Pro**, **Enterprise** ou **Education**.
- Virtualização habilitada.

O Windows por padrão deixa a virtualização habilitada, podemos conferir indo em **Gerenciador de Tarefas**, clicando na aba **Performance**.

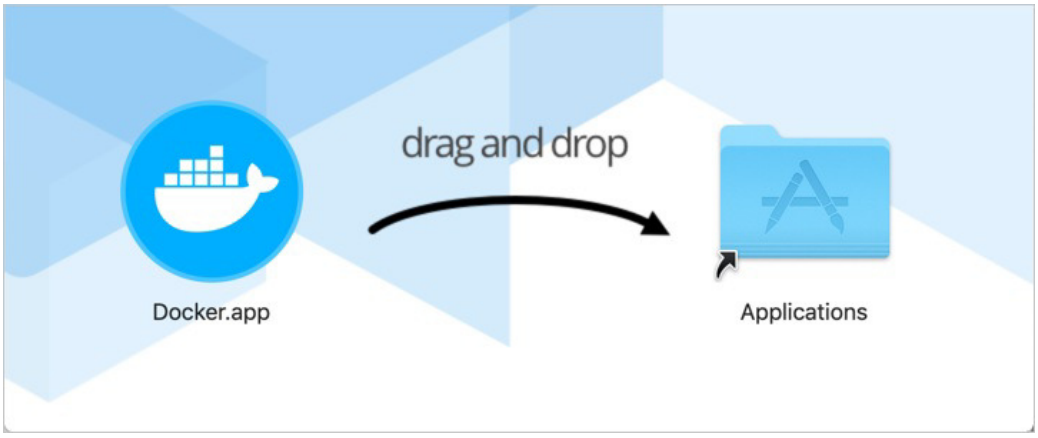
Seguindo o passo a passo do instalador até chegar em *Finish*, precisamos encerrar a sessão do Windows e iniciá-la novamente. Ao fazer o login, precisamos habilitar o Hyper-V, clicando em Ok, para reiniciar o computador.

Quando o computador terminar a reinicialização, irá aparecer um ícone do Docker na barra inferior, à direita, ao lado do relógio. O Docker pode demorar um pouco para inicializar, mas quando a mensagem **Docker is running** for exibida, significa que ele foi instalado com sucesso e já podemos utilizá-lo na linha de comando.

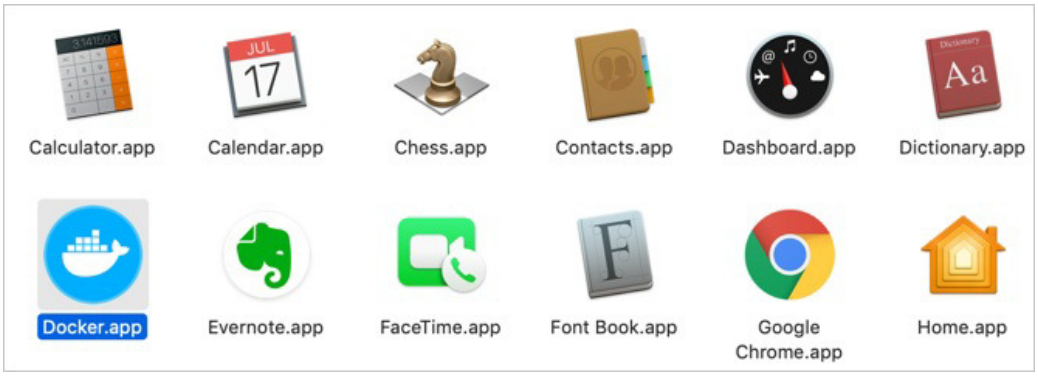
Vale ressaltar que hoje já existem os **Windows Containers**, porém, neste curso utilizaremos os **Linux Containers**. Falaremos mais para frente como o Docker Desktop for Windows consegue utilizar os Linux Containers sem problemas.

2.8 INSTALANDO NO MAC

O Docker também pode ser instalado no Mac. Basta acessar o link https://docs.docker.com/docker-for-mac/install/, fazer o download do arquivo **Docker.dmg** e arrastá-lo para a pasta de aplicações.



Após isso, você deverá clicar duas vezes em **Docker.app** em sua pasta de aplicações.



CAPÍTULO 3

CONHECENDO O DOCKER DAEMON

3.1 O PRIMEIRO CONTAINER

Uma vez instalado, podemos interagir com o **docker daemon** usando o **docker cli** (*docker command line interface*). É importante entender que temos uma diferença entre o conceito do **daemon** e do **cli**:

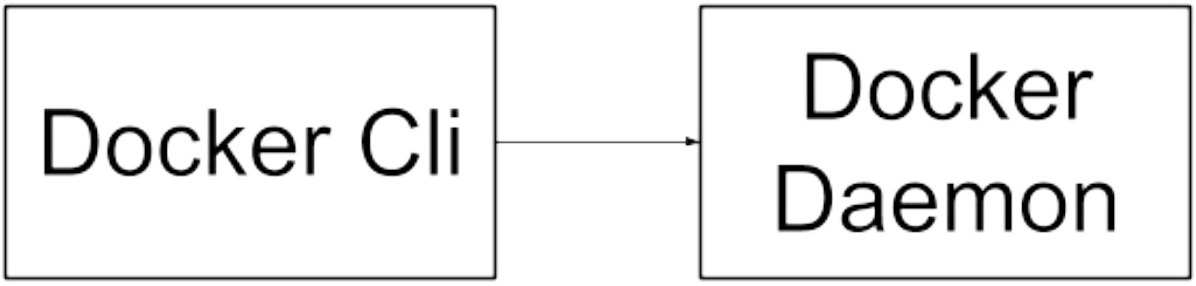


Figura 3.1: Docker Cli e Docker Daemon

Ou seja, ao executar o comando `docker`, estamos usando o docker cli, que envia um comando para o docker daemon.

A comunicação entre o `cli` e o `daemon` é feita através de uma **API REST**, ou seja, quando executamos um comando para listar informações de containers sendo utilizados em nossa máquina, por exemplo, é feita uma requisição do tipo **GET** para a rota da API `/containers/json`.

Para rodar um container, devemos usar o comando `run`, que recebe como parâmetro o nome de uma imagem. Ainda veremos o que são as imagens e como construir. O importante agora é saber que imagens possuem nomes e contém o suficiente para execução de um container.

3.2 UM POUCO DE ARQUITETURA

Arquiteturalmente falando, vimos que o Docker é dividido entre um **cli**, **API REST** e **daemon**, porém, podemos aprofundar mais um pouco nesse aspecto.

Hoje em dia, o Docker trabalha com diversos componentes. Além dos já citados, podemos também incluir o **containerd**, responsável pela execução e ciclo de vida dos containers, e o **OCI**, responsável por ser o runtime dos containers, ou seja, ele trabalha como uma interface com o kernel.

Um exemplo prático para criação de um container no Linux, seria: o comando de criação é enviado pelo **Docker CLI** e recebido pelo **Docker Daemon** através de uma **API REST**, então o **containerd** fica responsável pela execução da criação do container. Utilizando a interface **OCI** para se comunicar com o kernel, o container é criado.

Um ponto interessante a ressaltar é o seguinte, como o **OCI** é apenas uma interface, precisamos que algo implemente essa interface. No caso do Linux, a implementação do **OCI** é o **runc**.

3.3 SUBINDO O PRIMEIRO CONTAINER

O comando completo para executar o famoso *Hello World* usando o Docker será:

```
docker run hello-world
```

A imagem `hello-world` já deve existir em algum lugar para poder ser executada. O resultado é bem simples, apenas para visualização:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.

....
```

A primeira execução pode demorar um pouco, pois o docker daemon precisa baixar a imagem. Posteriormente, entenderemos de onde vêm as imagens e como esse processo funciona.

Veja que o container foi executado e encerrado automaticamente. Podemos ver isso usando o comando `docker container ls` que lista todos os containers em execução.

Para ver os containers que estão com status de **parado** podemos adicionar a flag `-a` ao comando:

```
docker container ls -a
```

Repare que a coluna *Status* indica o estado do container. O container recém-executado não está mais em execução pois dentro dele não há mais nenhum processo em execução em sua árvore de processos. Um container só permanece "vivo" enquanto há um ou mais processos em execução dentro dele.

Também podemos executar containers em modo interativo, ou seja, acessamos o "interior" do container. Para isso, basta adicionar mais algumas informações ao comando `docker run`, são elas as flags `-i` e `-t`, que podemos simplificar e tratar como `-it`. Por exemplo:

```
docker run -it ubuntu
```

A flag `-i` significa que queremos executar o terminal em modo *interativo*, enquanto a `-t` nos permite associar o *tty* padrão do container ao nosso terminal. Em outras palavras, ao rodar o comando acima, estamos acessando o terminal de um container **ubuntu**. Perceba que o terminal anexado ao nosso não será mais o de nossa máquina host, e sim o terminal de uma máquina ubuntu que está sedo executada dentro de um container. Logo, qualquer comando executado dentro desse terminal **não** terá

relação alguma com nossa máquina host.

```
docker run -it ubuntu
root@2830306d2f0b:/# ls
bin  boot  dev  etc  home  lib  lib64  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  va
r
root@2830306d2f0b:/# uname -a
Linux 2830306d2f0b 4.9.184-linuxkit #1 SMP Tue Jul 2 22:58:16 UTC 2019 x86_64 x86_64 x86_64 GNU/Linux
root@2830306d2f0b:/#
```

Mais para frente, veremos que existem outras formas de garantir que o container continue rodando sem a necessidade de `-it`.

3.4 EXERCÍCIO: O PRIMEIRO CONTAINER

Objetivo

Nos passos seguintes, criaremos o nosso primeiro container e utilizaremos comandos para listar o container criado. Usaremos os comandos:

- `docker run`
- `docker container ls`
- `docker ps`

O conteúdo recém explicado pode ser lido na documentação oficial através do link https://docs.docker.com/engine/docker-overview/. Alguns conteúdos ainda serão explicados no decorrer do curso.

Passo a passo

1. Abra o terminal, será através desta ferramenta que trabalharemos com o Docker.
2. Para executar um container, é necessário o uso do comando `docker run`. Além disso, também deve ser informado o nome do container a ser executado. Nesse exercício, será executado o container chamado `hello-world`. O comando final será:

   ```
   docker run hello-world
   ```

   O que aconteceu? O comando funcionaria sem internet?
3. Liste os containers que estão *em execução*. Para isso, execute o comando:

   ```
   docker container ls
   ```

   O container recém executado foi exibido?

   Obs: Alternativamente você pode usar o comando `docker ps`.

4. Agora, execute o comando:

   ```
   docker container ls -a
   ```

   Veja que agora o container criado foi exibido, porém com status de *Exited*.

   Obs: Alternativamente você pode usar o comando `docker ps -a`.

5. Execute o container e depois liste os containers novamente:

   ```
   docker run hello-world
   docker container ls -a
   ```

   Repare que para cada execução, ou seja, para cada comando `docker run`, é criado um novo container que possui seu próprio ID.

6. Agora, mantendo o atual aberto, abra um novo terminal. No novo terminal execute um novo container "ubuntu" no modo interativo:

   ```
   docker run -i -t ubuntu
   ```

   Com isso, começará a ser feito o download do conteúdo necessário para execução do container.

   Repare que o prompt do terminal mudou, pois agora estamos conectados ao terminal de dentro do container:

   ```
   root@cd4cb6d8b050:/#
   ```

   Qualquer comando executado nesse prompt refletirá apenas dentro do container.

7. Volte no outro terminal e digite o comando:

   ```
   docker container ls
   ```

   Repare que aparece o container em execução (*status up*).

8. Para finalizar o container `ubuntu`, feche o terminal que está executando o container. Depois verifique se o container realmente terminou com o comando `docker container ls`.

3.5 PARA SABER MAIS: CONTAINER LS

O que significa cada coluna gerada comando `docker container ls`? Em caso de dúvidas, pesquise o comando e acesse o link https://docs.docker.com/engine/reference/commandline/container_ls/ e veja mais sobre os outros parâmetros além do `-a`. Para que eles servem?

3.6 ENTENDENDO O CICLO DE VIDA DOS CONTAINERS

Já usamos o comando `docker run`, que faz vários passos de uma vez só. O primeiro é baixar a

imagem especificada caso a mesma não esteja em disco, o segundo é criar o container, já o terceiro é alocar todos os recursos necessários para finalmente executá-lo.

Para executar o container e termos total controle sobre nossa aplicação, é preciso entender o ciclo da vida de um container. Um container funciona exatamente como um processo do sistema operacional, conhecer o ciclo de vida de um processo do sistema já será de grande ajuda para compreender sobre containers.

Uma vez criado o container, podemos suspendê-lo para, por exemplo, executar algum backup de seus dados. Para pausar um container, temos o comando `pause` .

```
docker container pause <container-id/nome-container>
```

Para voltar a executar, usamos `unpause` :

```
docker container unpause <container-id/nome-container>
```

Caso deseje realmente para o container, temos o comando `stop` :

```
docker container stop <container-id/name>
```

E para iniciar novamente, usamos o comando `start` :

```
docker container start <container-id/name>
```

É importante saber que um container ocupa espaço no nosso sistema mesmo no estado parado. Podemos evitar consumo em disco desnecessário removendo containers de nossa máquina.

Para remover o container usamos o comando:

```
docker rm <container-id/nome-container>
```

O comando acima só funcionará se o container não estiver em execução. Para forçar a remoção, podemos usar a flag `--force` :

```
docker rm <container-id/nome-container> --force
```

Todos os comandos relacionados com o container se encontram na documentação. Veremos alguns de maneira mais detalhada no decorrer do curso:

https://docs.docker.com/engine/reference/commandline/container/

3.7 MAIS INFORMAÇÕES SOBRE O DOCKER CONTAINER LS

Já falamos que um container ocupa espaço no sistema, porém, quanto espaço extamente? Para tal, podemos usar os comandos `docker container ls` , adicionando a flag `-s` (ou `--size`):

```
docker container ls -as
```

A flag `-a` (`--all`) serve para mostrar todos os containers, não apenas os em execução. O

problema é que a saída pode ficar muito grande, ficando confuso de entender. Para filtrar a saída melhor, podemos a adicionar a flag `-f` (`--format`). A flag format permite especificar as colunas que queremos ver usando um template definido pela sintaxe `{{.NomeDaColuna}}` .

No comando abaixo mostramos o ID, Status e tamanho dos containers de maneira tabular:

```
docker container ls -as --format 'table {{.ID}}\t{{.Names}}\t{{.Status}}\t{{.Size}}'
```

A saída ficará assim:

```
CONTAINER ID        STATUS                      SIZE
45433996ed02        Exited (0) 13 minutes ago   0B (virtual 64.2MB)
2830306d2f0b        Exited (0) 16 minutres ago  17B (virtual 64.2MB)
28ebe6245f55        Exited (0) 2 hours ago      12B (virtual 64.2MB)
```

3.8 INSPECIONANDO CONTEÚDO DOS CONTAINERS

O comando `docker container ls` e `docker ps` já oferecem várias informações valiosas sobre os containers como status, tamanho, id, nome etc.

Uma alternativa para obtermos detalhes ainda mais internos sobre o container, é o comando `docker inspect` . As informações do comando vão muito além do que o `docker container ls` e possui informações sobre a rede, ambiente, a máquina host e muito mais. A sintaxe básica é:

```
docker inspect <container-id/nome-container>
```

Com uma possível saída:

```
[
    {
        "Id": "621326f9265aec7df8c2b9637b70c7825c985f9ff1458cfc02f3fe7b106160ae",
        "Created": "2019-12-27T21:02:27.1782469Z",
        "Path": "/bin/bash",
        "Args": [],
        "State": {
            "Status": "running",
            "Running": true,
            "Paused": false,
            "Restarting": false,
            "OOMKilled": false,
            "Dead": false,
            "Pid": 5639,
            "ExitCode": 0,
            "Error": "",
            "StartedAt": "2019-12-27T21:02:27.9589442Z",
            "FinishedAt": "0001-01-01T00:00:00Z"
        },
        ....
    }
    ....
]
```

A saída acima foi encurtada pois a saída do comando é muito grande. Por conta disso, também torna-se útil usar a formatação:

```
docker inspect --format='{{.Name}}' <container-id>
```

O comando acima mostrará apenas o nome do container baseado no id usado como entrada.

3.9 EXERCÍCIO: CICLO DE VIDA DO CONTAINER

Objetivo

Nos passos seguintes, criaremos outro container e utilizaremos novamente comandos para listar o container criado. Além disso, veremos como podemos parar, iniciar e remover containers. O objetivo é entender e controlar o ciclo de vida dos containers.

Passo a passo

1. No terminal, execute o comando usando a flag `-it` :

   ```
   docker run -it ubuntu
   ```

   Com isso, temos um container em execução e nos vinculamos ao seu terminal.

2. Abra um segundo terminal e digite o comando:

   ```
   docker container ls
   ```

   Copie ou anote a ID do container recém-criado, precisaremos dele no próximo passo.

3. Ainda no segundo terminal, vamos parar a execução do container usando o comando `docker container stop` . Esse comando precisa saber qual container queremos parar. Então adicione ao final do comando o nome ou id do seu container, algo como:

   ```
   docker container stop 2e8a9d4fdcaf
   ```

   ou usando o nome do container:

   ```
   docker container stop angry_pike
   ```

   Lembre-se, o id e nome do container podem ser obtidos através do comando `docker container ls` .

4. Volte ao primeiro terminal, o que aconteceu?

5. Agora, execute o comando `docker container ls -a` e repare o status do container criado no início do exercício.

6. Para retomar a execução de um container parado, execute o comando `docker container start <ID|NOME>` . Assim como no caso do `docker container stop` , devemos passar o id ou nome do container que queremos iniciar, execute:

   ```
   docker container start <ID|NOME_CONTAINER>
   ```

7. No segundo terminal, execute o comando `docker container ls` e repare que o container voltou a ser executado.

8. Para podermos nos conectar ao container novamente, podemos utilizar o comando `docker attach`, seguido do id ou nome do container que queremos nos acoplar. Execute este comando no primeiro terminal. Com esse comando, voltamos ao terminal de dentro do container:

   ```
   docker attach <ID|NOME_CONTAINER>
   ```

9. Além de iniciar e parar containers, também podemos removê-los com o comando `docker rm`. Tente informar o id do container que deseja remover, o que acontece ao usar o comando com um container em execução?

10. Para remover um container pelo comando `rm`, devemos pará-lo antes com o comando `docker stop`, ou podemos removê-lo de maneira forçada com o comando `docker rm <id-container> --force`.

11. Veja que como o container foi completamente removido, ele sequer é exibido pelo comando `docker container ls` ou `docker container ls -a`.

    Com isso, começará a ser feito o download do conteúdo necessário para execução do container.

    Repare que o prompt do terminal mudou, pois agora estamos conectados ao terminal de dentro do container:

    ```
    root@cd4cb6d8b050:/#
    ```

    Qualquer comando executado nesse prompt refletirá apenas dentro do container.

12. Volte no outro terminal e digite o comando:

    ```
    docker container ls
    ```

    Repare que aparece o container em execução (*status up*).

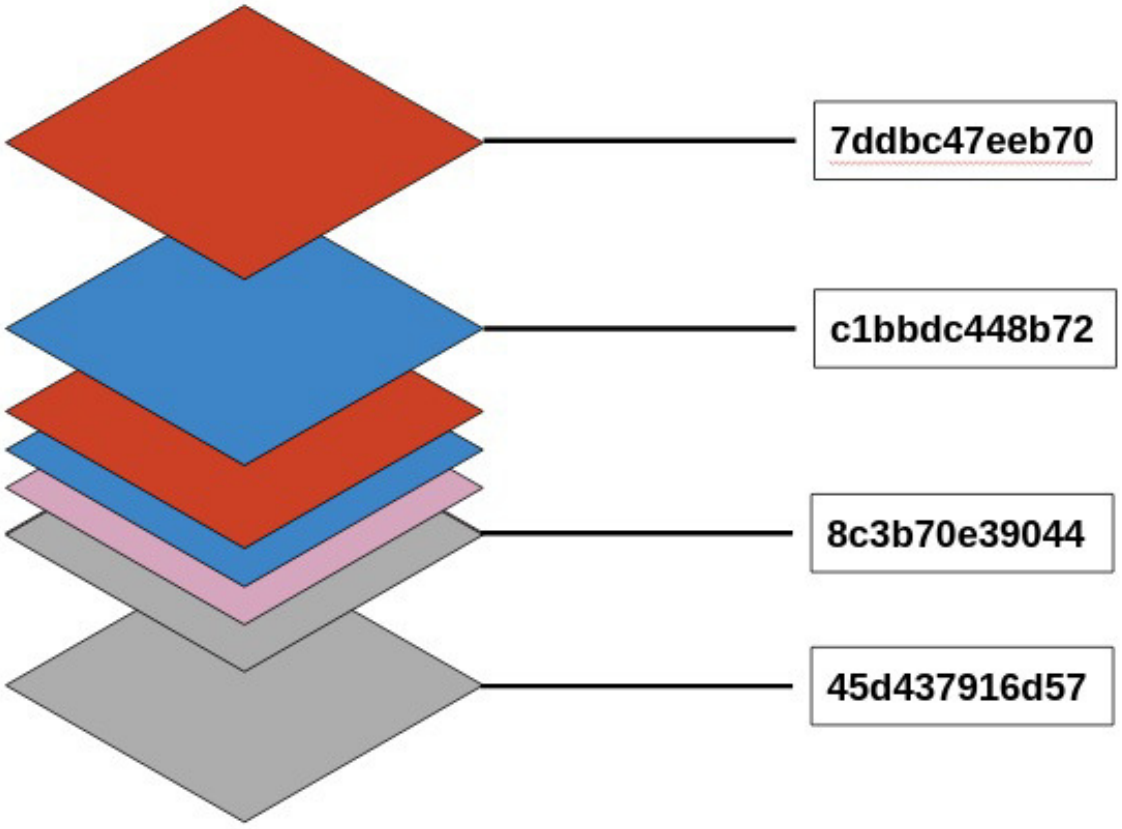
Capítulo 4

CRIANDO NOSSAS PRÓPRIAS IMAGENS

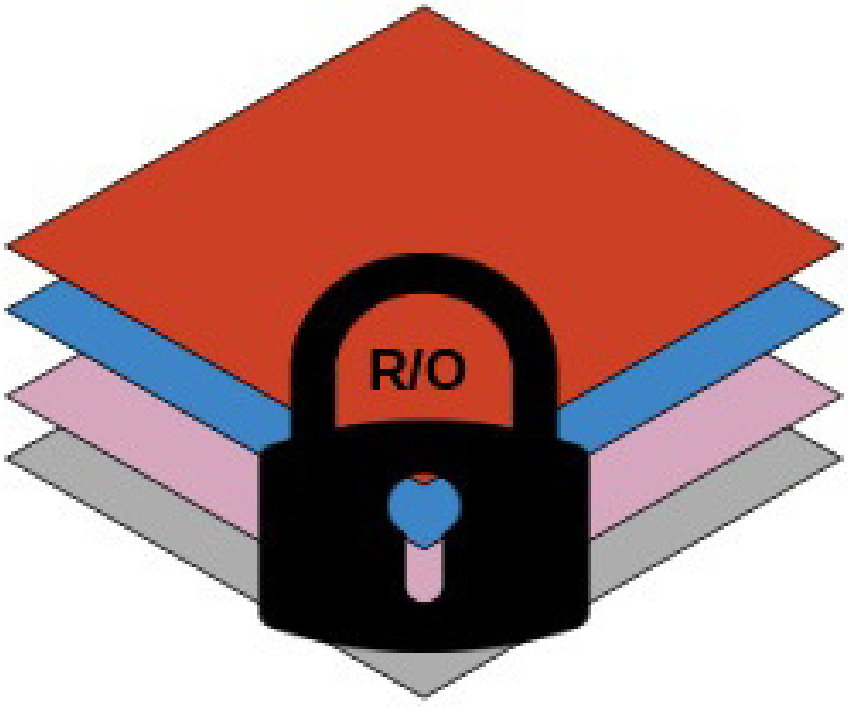
4.1 ENTENDENDO O CONCEITO DE IMAGEM

Antes de realmente criar a imagem, vamos mergulhar um pouco no conceito e entender melhor o que é uma imagem. De início, precisamos entender que uma imagem é composta de uma ou mais camadas.

As camadas compõem diferentes trechos de uma imagem, e, quando empilhadas, formam a imagem em si. Cada camada possui seu próprio ID.



Porém, qual a vantagem dessa abordagem em camadas? A princípio não há vantagem alguma em dividir uma imagem em pedaços menores. A grande ideia do Docker é fazer com que essas camadas sejam somente de leitura, não permitindo então modificações feitas diretamente em cima delas.



Como temos a garantia que as camadas jamais serão modificadas, os desenvolvedores podem reutilizá-las a partir de uma imagem existente em diferentes projetos. A reutilização economiza tempo e espaço, pois o usuário não precisará criar tudo do zero caso já tenha uma camada necessária em disco. Podemos, por exemplo, definir a imagem `ubuntu` como base, que por suas vez tem uma outra imagem como base, e adicionar uma nova camada para conter o servidor **nginx**. Todos os outros projetos que precisam de um servidor nginx podem se basear nessa imagem e assim por diante. Em outras palavras, cada camada personaliza e especifica mais o container.

O comando `docker history` mostra as camadas de uma imagem, informando quais comandos foram executados para que pudessem ser criadas:

```
docker history ubuntu
IMAGE               CREATED             CREATED BY                                      SIZE
549b9b86cb8d        9 days ago          /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B

<missing>           9 days ago          /bin/sh -c mkdir -p /run/systemd && echo 'do…'   7B

<missing>           9 days ago          /bin/sh -c set -xe   && echo '#!/bin/sh' > /…   745B

<missing>           9 days ago          /bin/sh -c [ -z "$(apt-get indextargets)" ]     987kB

<missing>           9 days ago          /bin/sh -c #(nop) ADD file:53f100793e6c0adfc…   63.2MB
```

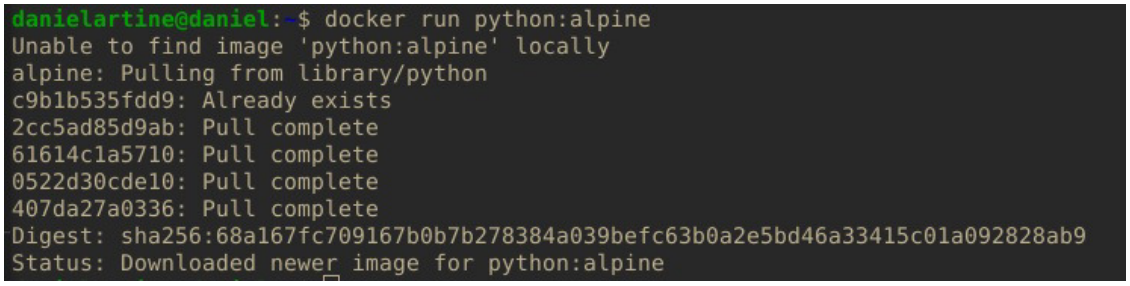
4.2 EXERCÍCIO: REUTILIZANDO CAMADAS

Objetivo

Neste exercício, veremos como o Docker é capaz de reutilizar camadas já baixadas em disco. Além disso, fixaremos comandos já executados.

Passo a passo

1. No terminal, execute um container com o seguinte comando `docker run alpine`.
2. Assim como em casos anteriores, o container foi executado e parou pois não havia nenhum processo em execução. Veja seu status atual com o comando `docker container ls -a`.
3. Agora, execute o comando `docker run python:alpine`.
4. Analise a saída do comando anterior e veja que o Docker conseguiu reutilizar uma camada já baixada em sua máquina.



5. Execute o comando `docker history python:alpine` e `docker history alpine` e tente identificar a camada em comum.

4.3 DRIVERS DE ARMAZENAMENTO

Atualmente, o Docker possui diferentes drivers para armazenamento e manipulação de imagens e camadas, sendo o mais recente o **overlay2**. Dentre os vários papéis do driver de armazenamento, um deles é empilhar as camadas para criação da imagem. O processo de empilhamento está diretamente ligado ao modo como escrevemos nosso `Dockerfile`.

Existem diferentes drivers que podemos utilizar com o Docker, são eles: **overlay2**, **aufs**, **btrfs**, **devicemapper**, **overlayfs**, **zfs** e **vfs**. Cada um com sua respectiva utilização.

- **overlay2** - é o driver recomendado para uso em qualquer distribuição linux que o suporte. Não requer quaisquer configurações extras.

- **aufs** - é o driver recomendado para uso nas versões do Docker 18.06 e anteriores ou qualquer versão do kernel linux igual ou anterior ao 3.13.
- **devicemapper** - ainda pode ser usado em alguns projetos, porém necessita configuração manual para usos em produção. Até o lançamento do overlay2, era o driver recomendado para usar em sistemas CentOS e RHEL.
- **btrfs e zfs** - estes drivers são recomendados para uso em sistemas que usam estes drivers também como sistema de arquivos do host. Com isso, podemos ter funcionalidades avançadas, como por exemplo *snapshots*.
- **vfs** - este driver é usado para situações de testes, não devendo ser usado em produção por possuir uma performance baixíssima.

O driver que será utilizado pode ser alterado através de uma configuração no arquivo `/etc/docker/daemon.json` . Por exemplo, para utilizar o driver **devicemapper**, basta adicionar ao arquivo o seguinte trecho:

```
{
    "storage-driver": "devicemapper"
}
```

Em seguida, o serviço do Docker precisa ser reiniciado. Para isso, podemos utilizar o comando `sudo systemctl restart docker` .

4.4 A ESTRATÉGIA COPY-ON-WRITE

Os drivers de armazenamento utilizam uma técnica chamada Copy-on-Write (CoW) para melhor desempenho quando copiam e/ou compartilham arquivos. Se um arquivo existe em uma camada e alguma outra camada necessita acessar esse arquivo, o próprio arquivo será usado. No primeiro momento em que esse arquivo for modificado em alguma outra camada, ele será copiado para a camada em questão e então alterado. Com esse processo, conseguimos reduzir consideravelmente o número de operações de entrada e saída, aumentando a eficiência desse processo.

4.5 IMAGENS X CONTAINERS

O Docker trabalha com o conceito de imagens, ou seja, para colocar um container em funcionamento, o Docker precisa **obrigatoriamente** de uma imagem. Já vimos isso acontecendo usando o comando `docker run` , que precisa do nome da imagem para funcionar.

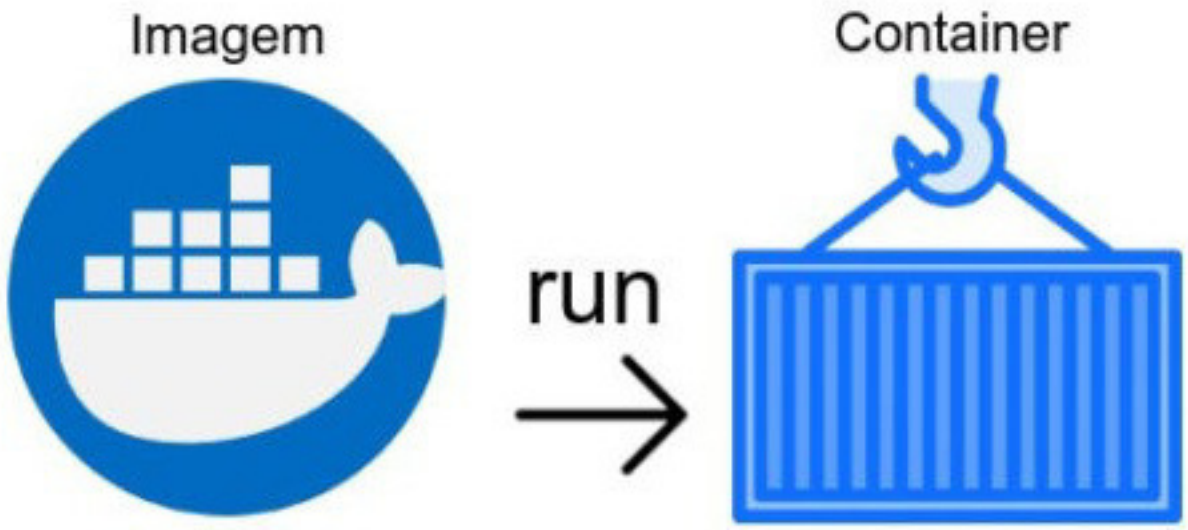


Figura 4.4: Imagem e Container

Quando executamos o comando `docker run`, se a imagem não for encontrada localmente, é baixada de um registro remoto, que por padrão é o *Docker Hub*. Por conta disso, a primeira execução de um container sempre será mais demorada que as seguintes.

Já usamos duas imagens que estão presentes no Docker Hub, uma chamada de `ubuntu` e outra de `hello-world`. Alguma pessoa ou organização criou essas imagens, definiu o conteúdo, colocou nomes e fez sua publicação.

O objetivo desse capítulo é entender como criar as nossas próprias imagens, qual a relação delas com os containers e como disponibilizá-las no Docker Hub. Isso é um passo fundamental, pois nosso objetivo será criar uma imagem própria, que possuirá uma aplicação com todas as dependências, além disso, queremos distribuir a imagem para outros desenvolvedores poderem testar e verificar.

Agora que já entendemos os conceitos que rodeiam uma imagem, como são criadas, o que são camadas e qual a diferença entre containers e imagens, podemos avançar e focar nossa atenção nos comandos de manipulação de imagens que o Docker nos oferece. Veremos inicialmente como listá-las, e, posteriormente, veremos como podemos baixar, criar, alterar, remover e distribuí-las.

4.6 LISTANDO IMAGENS

Para listar todas as imagens podemos usar o comando `docker images` ou `docker image ls` . O retorno desse comando é algo semelhante a isso:

```
docker image ls
REPOSITORY                  TAG                 IMAGE ID            CREATED             SIZE
ubuntu                      latest              549b9b86cb8d        2 days ago          64.2MB
...
```

Como podemos listar e inspecionar containers, o mesmo é possivel com imagens, por exemplo:

```
docker image inspect ubuntu
```

Que mostra informações valiosas sobre os varáveis de ambiente e tags. Aqui também pode ser usado a flag `--format` para filtrar as informações, por exemplo:

```
docker inspect ubuntu --format "{{.RepoTags}}"
```

```
[ubuntu:latest]
```

4.7 BAIXANDO E REMOVENDO IMAGENS

Caso não seja mais necessário, podemos remover a imagem do disco local pelo comando:

```
docker image rm <id-imagem|nome-imagem>
```

Ou simplesmente usar o atalho `docker rmi <id-imagem|nome-imagem>` .

Já vimos que o `docker run` automaticamente baixa, cria e roda o container. Porém, também é possível apenas baixar a imagem apenas, sem executar container algum. Para tal podemos usar o comando `docker pull` e, por exemplo, baixar a imagem do Ubuntu:

```
docker pull ubuntu
```

```
Com uma possível saída:
```

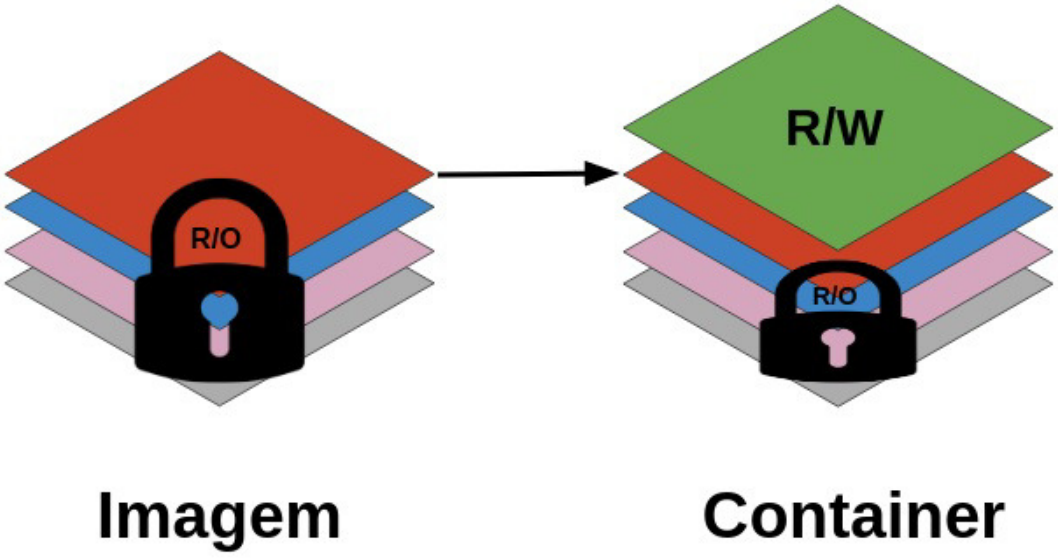
```
docker pull ubuntu
Using default tag: latest
latest: Pulling from library/ubuntu
2746a4a261c9: Pull complete
4c1d20cdee96: Pulling fs layer
0d3160e1d0de: Pulling fs layer
c8e37668deea: Waiting
....
```

Perceba que são executados vários downloads (*pulls*), pois uma imagem é composta de várias camadas (*layers*).

4.8 O QUE É A CAMADA DE ESCRITA?

Como podemos escrever dentro de um container para, por exemplo, instalar um pacote ou escrever um arquivo de log, como já viemos fazendo, já que as camadas são de leitura? A resposta é simples: quando um novo container é criado, também é criada uma nova camada extremamente leve apenas para escrita! Essa camada é chamada de *container layer* e guarda as alterações feitas pelo container em execução.

Por conta disso, quando criamos múltiplos containers a partir de uma mesma imagem em nossa máquina, o consumo será mínimo, pois todas as camadas em comum serão reutilizadas, adicionando apenas uma camada de escrita para cada container criado.



4.9 COMITANDO IMAGEM A PARTIR DA CONTAINER LAYER

Já vimos como rodar o Ubuntu pelo Docker usando o comando `docker run`. No exemplo abaixo, também definimos explicitamente o nome do container para fácil identificação:

```
docker run -it --name container_com_apache ubuntu
```

No terminal, podemos agora instalar pacotes e dependências. Por exemplo, para instalar o apache2:

```
apt-get update
apt-get install apache2 -y
exit
```

Todas as alterações foram escritas na *camada do container*. O interessante é que podemos salvar essa camada como imagem para reaproveitar depois. Assim não precisamos mais instalar os pacotes, já teremos a imagem pronta.

O primeiro passo para tal é parar o container:

```
docker container stop container_com_apache
```

Agora, podemos salvar ou *comitar* o camada do container para criar uma imagem:

```
docker container commit container_com_apache ubuntu_apache2
```

Ao executar o `docker images` podemos ver a nossa imagem `ubuntu_apache2`.

E para usá-la basta executar:

```
docker run -it ubuntu_apache2
```

Apesar de útil, na maior parte dos casos não é boa prática utilizar o comando `docker commit`, pois podemos acabar tornando nossa imagem muito grande com conteúdos desnecessários para o funcionamento real de nossa aplicação. Além disso, esse é um processo manual e dispendioso na maior parte dos casos.

4.10 CRIANDO IMAGEM A PARTIR DO DOCKERFILE

Vimos como criar uma imagem usando o comando `commit`, mas repare que executamos uma série de comandos dentro e fora do container. Seguimos um processo manual, pouco transparente, imperativo e propenso ao erro.

Seguindo a ideia de *Infrastructure as code*, devemos ter um arquivo textual versionável e de fácil entendimento para construir a nossa a imagem. Esse é exatamente o papel do `Dockerfile`.

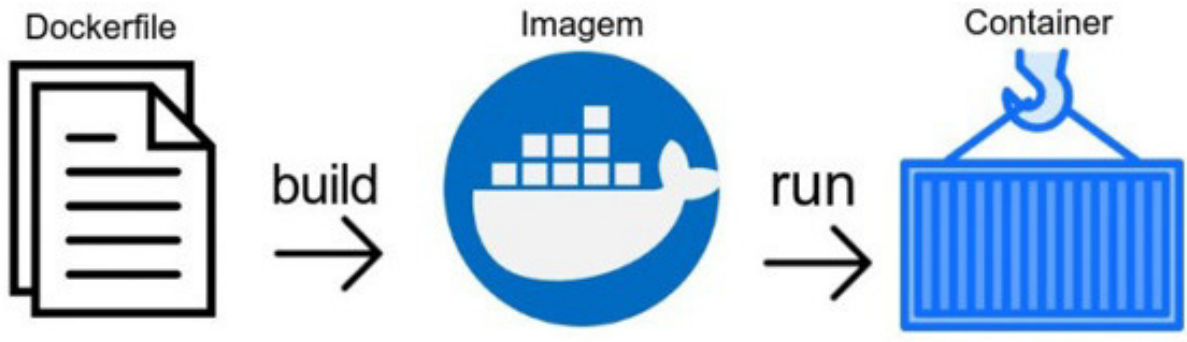


Figura 4.6: Dockerfile, Imagem e Container

Seguindo esses passos, primeiramente criaremos um `Dockerfile`, executaremos um comando de build do Docker para, seguindo as instruções dentro do arquivo, criarmos uma imagem para que por fim

possamos executar o nosso container.

Veja a definição de um `Dockerfile` que faz a mesma instalação do exemplo anterior, onde utilizamos o ubuntu e instalamos o apache2:

```
FROM ubuntu:16.04
RUN apt-get update
RUN apt-get install -y apache2
ENTRYPOINT bash
```

Falamos que uma imagem Docker muitas vezes se baseia em outras. Isso quer dizer que podemos utilizar a instrução `FROM` para definirmos que nossa imagem utilizará como base a imagem do ubuntu na versão 16.04.

Depois, executamos os comandos de instalação que, até então, eram executados dentro do container: `RUN apt-get update` e `RUN apt-get install -y apache2`. É importante ressaltar que a instrução `RUN` não executa quaisquer comandos dentro do container, o que ela faz é criar novas camadas enquanto executa os comandos na etapa de **criação da imagem**.

Nesse caso, a instrução `ENTRYPOINT` define o prompt padrão ao executar o container. O uso do `ENTRYPOINT` definir o comando que queremos executar quando o container subiu. Há uma instrução chamada `CMD`, com ela, podemos definir parâmetros que passados para a instrução `ENTRYPOINT`.

Agora só falta criar a imagem. Para tal, basta um único comando:

```
docker build -t ubuntu_apache2 .
```

Com uma possível saída:

```
Sending build context to Docker daemon  2.048kB
Step 1/4 : FROM ubuntu:16.04
16.04: Pulling from library/ubuntu
3386e6af03b0: Pull complete
49ac0bbe6c8e: Pull complete
d1983a67e104: Pull complete
1a0f3a523f04: Pull complete
Digest: sha256:181800dada37....
Status: Downloaded newer image for ubuntu:16.04
---> c6a43cd4801e
Step 2/4 : RUN apt-get update
---> Running in dfc2cb17a20cRemoving intermediate container dfc2cb17a20c
.....
---> 28b57c5df7bf
Step 3/4 : RUN apt-get install -y apache2
---> Running in 9afb25bea259
.....
Step 4/4 : ENTRYPOINT bash
---> Running in 83c35243f48d
Removing intermediate container 83c35243f48d
---> 7998b864aa04
Successfully built 7998b864aa04
Successfully tagged ubuntu_apache2:latest
```

Ao executar, o Docker vai procurar um arquivo `Dockerfile` no pasta atual. O parâmetro `-t` (*tag*)

define o nome da imagem, que veremos ainda com mais detalhes. O "ponto final" no comando define o contexto que pode ser necessário quando o Docker precisar copiar arquivos para dentro da imagem, nesse caso o contexto será referenciado como nosso diretório atual (por isso o ponto final). Repare também que cada instrução no `Dockerfile` torna-se um *step* na execução.

É possível definir o local do `Dockerfile` explicitamente pelo comando:

```
docker build -t ubuntu_apache2 -f /caminho/para/Dockerfile.
```

Mais informações da documentação do comando `build`:

https://docs.docker.com/engine/reference/builder/

4.11 MAIS INSTRUÇÕES DO DOCKERFILE

Já vimos algumas instruções no `Dockerfile`, como `FROM`, `RUN` e `ENTRYPOINT`. Caso a imagem deva conter uma aplicação, também é necessário copiar arquivos para dentro da imagem. Para tal, usamos o comando `COPY`:

```
FROM ubuntu:18.04
COPY . /app
RUN pip install -r /app/requirements.txt
ENTRYPOINT python /app/app.py
```

Pelo comando `COPY` todo o conteúdo da pasta atual será copiado para o diretório `/app` na imagem. Podemos também copiar arquivos individualmente, sem necessidade de copiar diretórios inteiros de uma só vez. Uma variante da instrução `COPY` é a `ADD`, esta permite adicionar arquivos a partir de repositórios remotos, além disso, também é capaz de extrair arquivos compactados automaticamente.

Repare também que usamos várias vezes o caminho `/app` para rodar os comandos dentro dessa pasta. Para não repetir esse caminho o tempo todo, basta definir o diretório de trabalho padrão, o `WORKDIR`:

```
FROM ubuntu:18.04
COPY . /app
WORKDIR /app
RUN pip install -r requirements.txt
ENTRYPOINT python app.py
```

Com isso, quaisquer comandos e/ou instruções serão executadas por padrão em nosso `WORKDIR` definido.

Resumindo os comandos vistos até aqui:

- `FROM` - cria uma camada da imagem a partir de uma imagem base.
- `COPY` - copia arquivos locais para um diretório dentro do container.
- `RUN` - executa um comando na etapa de construção da imagem.

- `ENTRYPOINT` - configura um container que será executado como um executável.
- `WORKDIR` - define o diretório de trabalho para qualquer execução como `CMD`, `ENTRYPOINT`, `COPY` e `ADD`.

Veja as melhores práticas para criação de um Dockerfile no link: https://docs.docker.com/develop/develop-images/dockerfile_best-practices/

4.12 EXERCÍCIO: CRIANDO NOSSO PRIMEIRO DOCKERFILE

Objetivo

É de suma importância podermos criar nossas próprias imagens. Nos passos abaixo criaremos nossa primeira imagem customizada. O container gerado para essa imagem executará uma aplicação escrita em Java.

Usaremos o `Dockerfile` junto às instruções `RUN`, `WORKDIR`, `COPY`, `FROM` e `ENTRYPOINT`, além de construir a imagem com o comando `docker build`.

Passo a passo

1. Crie em sua área de trabalho uma pasta chamada `primeira-imagem` e, dentro dela, crie um arquivo chamado *Dockerfile*.

2. O arquivo *Dockerfile* será responsável por conter as instruções que vão gerar a nossa imagem. Dentro dele adicione a seguinte linha:

   ```
   FROM ubuntu
   ```

   A instrução acima indica que nossa imagem terá como base a imagem do *ubuntu*. O uso da instrução **FROM** é **obrigatório**.

3. Copie o arquivo `springapp.jar` da pasta de arquivos do curso (`~/arquivos do curso/do26`) para a pasta `primeira-imagem`. Como já vimos, o nosso sistema de arquivos é isolado do sistema de arquivos do container. Logo, precisamos copiar o arquivo `springapp.jar` para dentro da nossa imagem de algum modo. Para isso, temos a instrução **COPY**. Seu Dockerfile deve estar algo parecido com isso:

   ```
   FROM ubuntu
   COPY springapp.jar .
   ```

   No comando acima, adicionamos o arquivo `springapp.jar` para o diretório **.**, que significa que estamos colocando o arquivo em nosso diretório atual dentro do container. Mas qual é esse diretório? Podemos definir qual será o nosso diretório de trabalho padrão dentro do container. Para isso, podemos utilizar a instrução **WORKDIR**. Altere o seu Dockerfile para ficar do seguinte modo:

```
FROM ubuntu
WORKDIR /app
COPY springapp.jar .
```

Agora, sempre que realizarmos alguma operação ou fizermos referência ao diretório atual, estaremos nos referindo ao diretório */app*.

4. Precisamos ainda instalar o Java em nossa imagem, para que, quando o container estiver em execução, tenha capacidade de executar o jar. Para executarmos comandos no momento da construção da imagem, devemos utilizar a instrução **RUN**. Como queremos instalar o Java, faremos o seguinte.

```
FROM ubuntu
WORKDIR /app
COPY springapp.jar .
RUN apt-get update && apt-get install -y openjdk-8-jdk
```

Com a instrução acima, estamos indicando que no momento da construção da imagem queremos que os comandos *apt-get update* e *apt-get install openjdk-8-jdk* sejam executados.

5. Por fim, quando o nosso container for executado, queremos que ele execute a aplicação automaticamente, sem necessidade de intervenção manual. Podemos definir qual será o primeiro comando a ser executado assim que o container for iniciado através da instrução **ENTRYPOINT**. Teremos o seguinte resultado:

```
FROM ubuntu
WORKDIR /app
COPY springapp.jar .
RUN apt-get update && apt-get install -y openjdk-8-jdk
ENTRYPOINT java -jar springapp.jar
```

6. Precisamos agora construir nossa imagem a partir do *Dockerfile*. Abra o terminal dentro da pasta `primeira-imagem` e execute o comando:

```
docker image build -t appjava .
```

O ponto no comando anterior indica que estamos construindo nossa imagem chamada `appjava` procurando o nosso *Dockerfile* a partir do diretório atual em nossa máquina.

Para ver a imagem, podemos usar o comando:

```
docker image ls
```

7. Acesse o link https://docs.docker.com/engine/reference/builder/ e veja as outras instruções possíveis para usar no *Dockerfile*.

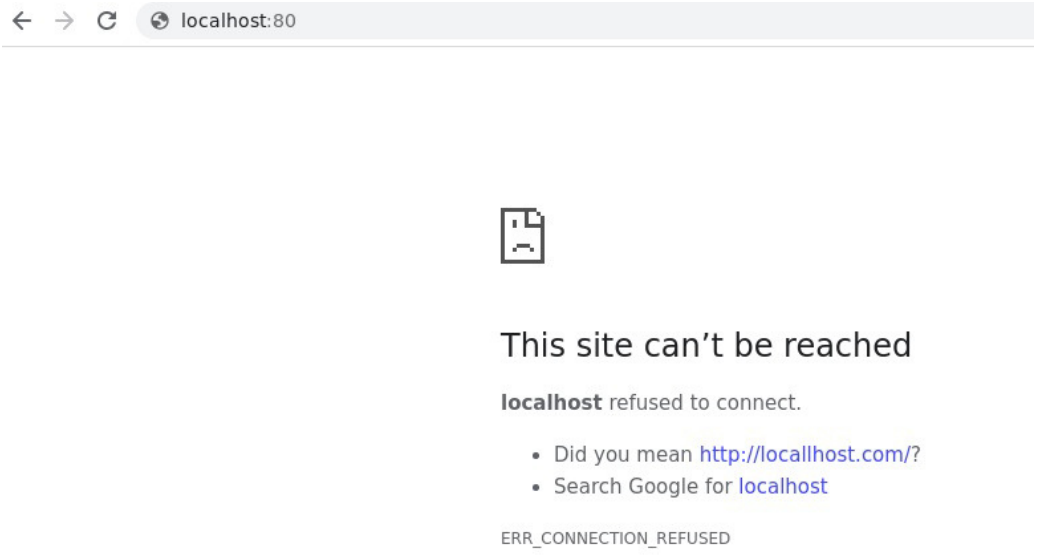
4.13 ACESSANDO PORTAS DO CONTAINER A PARTIR DA MÁQUINA HOST

No decorrer do curso, vimos que por conta dos **namespaces**, temos isolamento entre nossa máquina host e container em diversos aspectos, um deles é a interface de rede. Por conta disso, caso executássemos alguma aplicação interna ao container através do comando `docker run`, por exemplo o **nginx**, como faríamos para acessá-lo a partir de nossa máquina host?

Executando o comando `docker run nginx`, obtemos o seguinte resultado:

```
caelum@caelum-sl2-mq10:~$ docker run nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
8ec398bc0356: Pull complete
a53c868fbde7: Pull complete
79daf9dd140d: Pull complete
Digest: sha256:70821e443be75ea38bdf52a974fd2271babd5875b2b1964f05025981c75a6717
Status: Downloaded newer image for nginx:latest
```

Conhecendo o nginx, sabemos que por padrão ele é executado na porta 80. Ao tentarmos acessar a porta 80 em nossa máquina local, é retornado o seguinte:

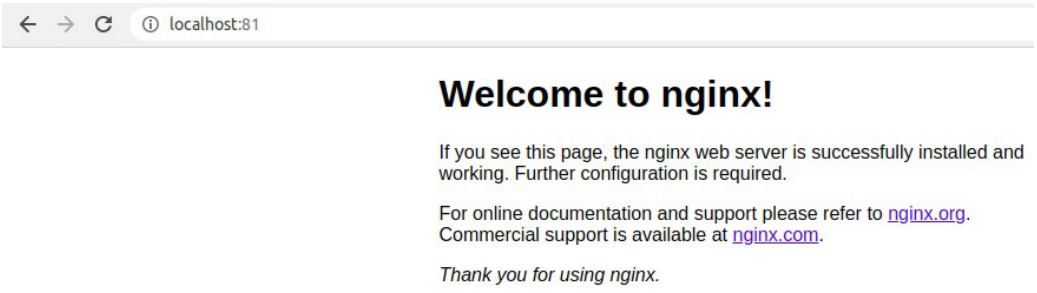


O resultado acima era esperado. Por quê? Simplesmente por conta da interface de rede do Docker estar isolada da interface de rede de nossa máquina host! Da maneira atual, jamais conseguiremos acessar a porta 80 do nosso container fazendo referência ao endereço de nossa máquina host (localhost).

Certamente o Docker provê para nós um meio de mapearmos nossas portas entre os containers e o host. Para isso, podemos utilizar a flag `-p` ou `--port`. Executando agora `docker run -p 81:80 nginx`, estamos indicando ao Docker que a porta 80 do container refletirá na porta 81 de nossa máquina

host. **Mágica**!

Acessando agora o endereço localhost:81, obtemos o seguinte resultado:



4.14 EXERCÍCIO: ACESSANDO APLICAÇÕES DENTRO DO CONTAINER

Objetivo

No último exercício, criamos a nossa imagem com uma aplicação dentro. Porém, como podemos fazer para, a partir da nossa máquina, acessar a aplicação que está dentro do container?

Passo a passo

1. Primeiramente, devemos executar o nosso container. Para isso, execute o comando:

   ```
   docker run appjava
   ```

2. A aplicação dentro do container está executando uma aplicação na porta 8080. Tente acessar a partir do navegador de sua máquina a partir de `localhost:8080`. O que aconteceu?

3. O problema é que o Docker isola as interfaces de rede de nossa máquina, sendo assim, precisamos descobrir qual endereço IP foi atribuído ao nosso container. Para isso, execute o comando:

   ```
   docker container inspect
   ```

   Inspecione o container que está executando a imagem `appjava`. Na saída do comando, procure o campo `IPAddress`, por exemplo 172.17.0.2.

4. Agora, em seu navegador, acesse a aplicação na porta 8080 utilizando o endereço IP do container, por exemplo: 172.17.0.2:8080.

   Repare que agora funcionou. Porém, tivemos que nos empenhar em procurar o endereço IP do container, o que acaba por ser um pouco dispendioso. Temos uma outra maneira mais simples de conseguir acessar aplicações de dentro do container a partir de nossa máquina.

5. Antes de seguir, vamos parar todos os containers que estão em execução, para que não haja nenhum

tipo de conflito e todos possam estar alinhados. Para parar um container, vimos a possibilidade de utilizar o comando `docker container stop` e basta informar o id do container em seguida. Porém, existe um *truque* para que possamos parar todos os containers em execução. Esse comando é o `docker container stop $(docker container ls -qa)`.

O comando `docker container stop $(docker container ls -qa)` usa como entrada para o comando `docker container stop` todos os containers retornados pelo comando `docker container ls -q`. Para que serve exatamente a flag `-q`? Pesquise através do link https://docs.docker.com/engine/reference/commandline/container_ls/.

6. Agora, para executar o nosso container de maneira que possamos acessar a aplicação diretamente a partir de nossa máquina, vamos utilizar novamente o comando `docker run appjava`. A diferença agora será que passaremos uma flag extra. Execute o comando:

   ```
   docker run -p 9000:8080 appjava
   ```

   Com o comando acima, estamos executando o container da mesma maneira de antes. A diferença, é que agora estamos falando para o Docker que a porta 9000 da nossa máquina, refletirá na porta 8080 do nosso container.

7. Tente acessar novamente a aplicação através do endereço `localhost:9000` e você verá que a aplicação está funcionando normalmente.

4.15 VERSIONANDO IMAGEM COM TAGS

Uma imagem evolui diante do ciclo da vida de um projeto, ela pode e as vezes deve ser recriada pois a aplicação ou alguma dependência mudou. Podemos sempre sobrescrever a imagem mantendo uma única versão, porém o mais interessante é ser explícito com a versão da imagem e manter um histórico de diferentes versões. O Docker permite o versionamento das imagens através de tags. Para tal existe um comando específico, o `docker tag`.

Ao listar as imagens locais já aparece a tag:

```
docker image ls
REPOSITORY            TAG           IMAGE ID          ...
primeira-image        latest        7998b864aa04      ...
```

Ou seja, como não definimos a tag no momento da criação da imagem, o Docker automaticamente definiu a tag como `latest`. Porém, podemos mudar isso:

```
docker tag primeira-imagem primeira-imagem:1.0
```

E ao listar aparece a imagem com TAG:

```
docker image ls
REPOSITORY            TAG         IMAGE ID          ...
primeira-image        1.0         7998b864aa04      ...
```

Também podemos definir a tag na hora de buildar a imagem:

```
docker build -t primeira-imagem:1.0 .
```

A boa prática é sempre definir o repositório do criador da imagem. Assim, o comando fica:

```
docker build -t caelum/primeira-imagem:1.0 .
```

e ao listar a imagem com o comando `docker image ls`:

```
REPOSITORY                TAG         IMAGE ID          ...
alura/primeira-image      1.0         7998b864aa04      ...
```

Mais informações na documentação do comando `docker tag`:

https://docs.docker.com/engine/reference/commandline/tag/

4.16 EXERCÍCIO: VERSIONAMENTO DA IMAGEM

Objetivo

Muitas vezes, é necessário alterar a nossa imagem de algum modo, seja através de alguma instrução modificada ou arquivos copiados. Em outras palavras, vamos ter várias versões da imagem que precisamos distinguir de alguma forma.

Nos passos seguintes, definiremos outra versão para a a mesma imagem criada no exercício anterior.

Passo a passo

1. Como vimos nas explicações anteriores, existem diversas imagens para diferentes fins. Uma dessas imagens é o `openjdk`, ou seja, não precisamos utilizar uma imagem do Ubuntu e instalar manualmente o `openjdk` por cima dessa imagem. Podemos usar diretamente a imagem do `openjdk`. Crie uma pasta na área de trabalho chamada `segunda-imagem`. Nela, crie o seguinte *Dockerfile*:

   ```
   FROM openjdk:8
   WORKDIR /app
   COPY springapp.jar .
   ENTRYPOINT java -jar springapp.jar
   ```

   Repare que algumas mudaram, como por exemplo: agora utilizamos diretamente a imagem do `openjdk` na versão 8, e não temos mais a necessidade de utilizar a instrução `RUN` para instalar nada em nossa imagem em tempo de build.

2. Agora, precisamos novamente construir a imagem. Copie o arquivo springapp.jar para dentro da pasta `segunda-imagem` e execute o comando `docker build -t appjava:2.0 .`.

3. Execute o comando `docker image ls`. Repare que agora temos nossa imagem `appjava` na

versão **2.0** (a mais recente) e na versão **latest** (a mais antiga). Isso é um problema, pois **latest** indica que é a versão mais recente da imagem, ou seja, nossas tags estão incoerentes. Precisamos mudar a tag para algo mais plausível, e, para isso, podemos utilizar o comando `docker image tag` ou simplesmente `docker tag`. Execute o comando `docker tag appjava:latest appjava:1.0`.

4. Mais uma vez, execute o comando `docker image ls` e repare que agora temos três versões da imagem `appjava`. Porém, repare que a **latest** continua lá e podemos removê-la através do comando `docker image rm appjava:latest`.

5. Agora utilize o comando `docker image ls` novamente. Repare que a imagem na versão **1.0**, utilizando o ubuntu e depois instalando manualmente o openjdk é mais leve do que a imagem usando diretamente o openjdk. Como isso é possível? Debata com o instrutor e colegas.

4.17 MULTI-STAGE BUILDS

Uma das coisas a se pensar no momento da criação de uma imagem, é manter o seu tamanho o menor possível, afinal, quanto mais pesada uma imagem, maior será o custo em disco para amazená-la, o que acarretará em um maior custo financeiro. Instruções como `RUN`, `ADD` e `COPY`, adicionam novas camadas à imagem. Você pode verificar as camadas de uma imagem executando o comando `docker history NOME_DA_IMAGEM`, por exemplo:

```
docker history appjava:1.0
```

Nossa imagem final pode conter diversas etapas, camadas de produção, que não são necessárias para o funcionamento de nosso container. Mas como reduzimos essas camadas que não precisamos? Podemos construir a imagem em **diferentes etapas**, ou seja, um ***multi-stage build***.

Temos o seguinte `Dockerfile`, que faz a cópia de uma classe Java para dentro da pasta `projeto`, cria o seu JAR e o executa:

```
FROM openjdk:8
WORKDIR /projeto
COPY MinhaClasse.class /projeto
RUN jar -cvf meu-projeto.jar MinhaClasse.class
ENTRYPOINT java -jar meu-projeto.jar
```

Após a criação do JAR, há a necessidade de continuarmos tendo o arquivo **.class** e todo o ambiente de desenvolvimento? A resposta é não, pois, depois de compilado, só nos interessa executar o JAR. Para construir essa imagem em diferentes passos, nós podemos dividir em duas etapas, a etapa de **compilação e criação do JAR**, na qual iremos compilar a classe Java e criar o JAR, e a etapa de **execução do JAR**, na qual iremos copiar o JAR da primeira etapa e executá-lo:

```
FROM openjdk:8 as compilacao
WORKDIR /projeto
COPY MinhaClasse.class /projeto
RUN jar -cvf meu-projeto.jar MinhaClasse.class

FROM openjdk:8-jre as execucao
WORKDIR /jar
COPY --from=compilacao /projeto/meu-projeto.jar /jar/
ENTRYPOINT java -jar meu-projeto.jar
```

Com isso, no momento do *build* do `Dockerfile`, a compilação e criação do JAR será feita em uma imagem, e, na segunda imagem, a final, o arquivo JAR será copiado da primeira imagem, pronto para ser executado apenas com o ambiente de execução. Assim, não teremos mais conteúdo desnecessário na nossa imagem final, como por exemplo o arquivo `MinhaClasse.class` e o JDK completo, apenas o JRE.

É claro que podemos expandir para diversas outras linguagens e plataformas. A utilização do Java nesse caso foi meramente ilustrativa.

Mais informações e exemplos podem ser adquiridos na documentação oficial: https://docs.docker.com/develop/develop-images/multistage-build/.

Capítulo 5

PUBLICANDO IMAGENS NO DOCKER HUB

5.1 O QUE É O DOCKER HUB?

Já usamos imagens como `hello-world` e `ubuntu`. Porém, como essas imagens foram parar em nossa máquina? Deve existir algum lugar central de imagens para armazenamento, este local é o **Docker Hub**.

O Docker Hub é um serviço fornecido pela própria empresa Docker para guardar, pesquisar e compartilhar imagens. Por meio do Docker Hub, desenvolvedores podem acessar repositórios de imagens públicos, além de criar seus próprios repositórios particulares, definir builds automatizados, usar webhooks e muito mais.

O link do site é: https://hub.docker.com/

5.2 DOCKER HUB VS DTR

Empresas e organizações do governo nem sempre podem ou querem publicar as imagens no Docker Hub, já que não controlam o local e a segurança das imagens. Nesse caso, faz sentido ter um repositório exclusivo e privado, gerenciado pela empresa. Para tal existe o DTR, o **Docker Trusted Registry** que faz parte da edição empresarial do Docker: o **Docker EE**.

Em poucas palavras, o *Docker Trusted Registry (DTR)* é a solução de armazenamento de imagens no nível empresarial da Docker. A empresa que adquire os serviços do **Docker EE** fica responsável por instalar o DTR, que nada mais é do que um container. Assim, é possível que uma empresa publique, gerencie e distribua imagens baseado em suas próprias regras e restrições.

Evite use o Docker Hub em produção! Esse conselho é dado tanto pela equipe Docker quanto pela comunidade no geral. Confiar no Docker Hub para manter seus projetos em produção é algo arriscado, pois o mesmo está sujeito a sair do ar, passar por manutenções, entre outros problemas, e caso isso aconteça poderá impactar no processo de desenvolvimento e entrega do seu software.

Mais informações sobre o DTR e Docker EE na documentação: https://docs.docker.com/ee/dtr/

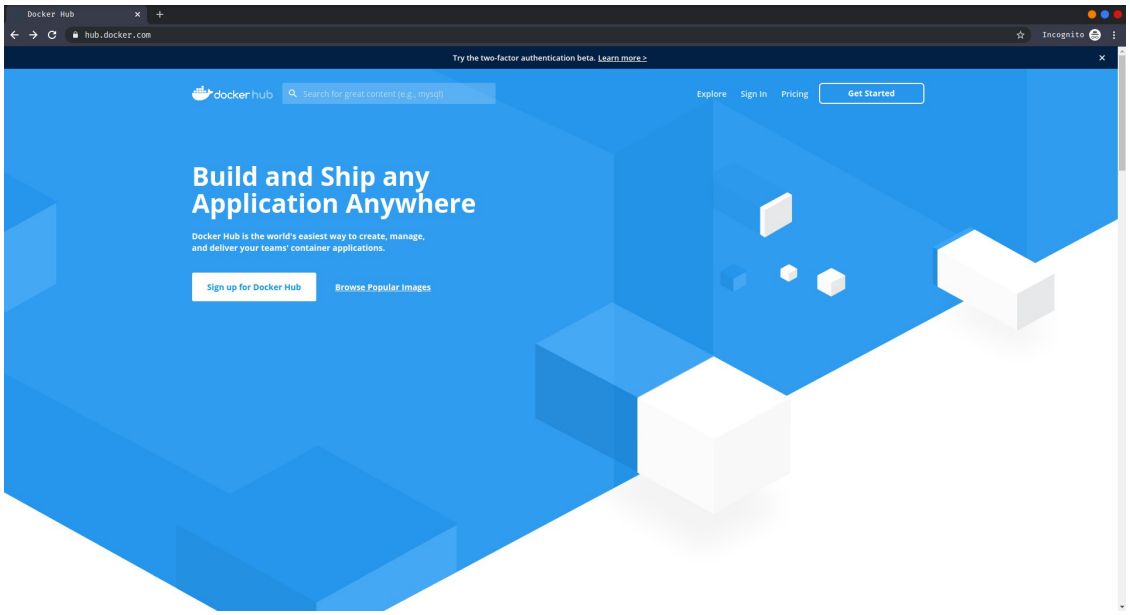
5.3 EXERCÍCIO: CRIANDO SUA CONTA NO DOCKER HUB

Objetivo

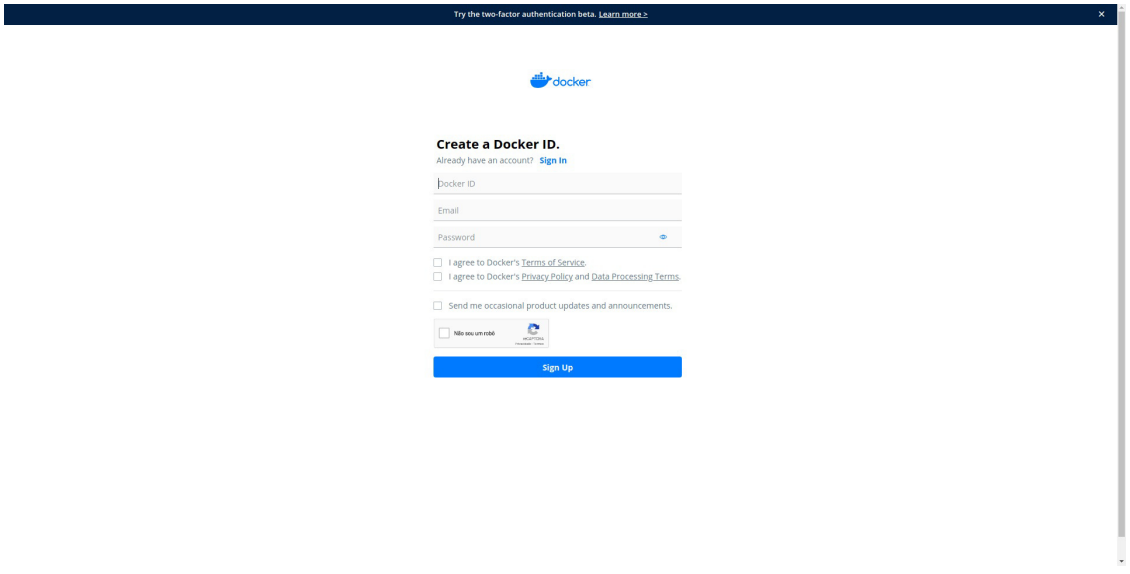
Para podermos criar nossos próprios repositórios, fazer upload de imagens e afins, precisamos criar nossa conta no Docker Hub. Nos passos a seguir será mostrado como fazê-lo.

Passo a passo

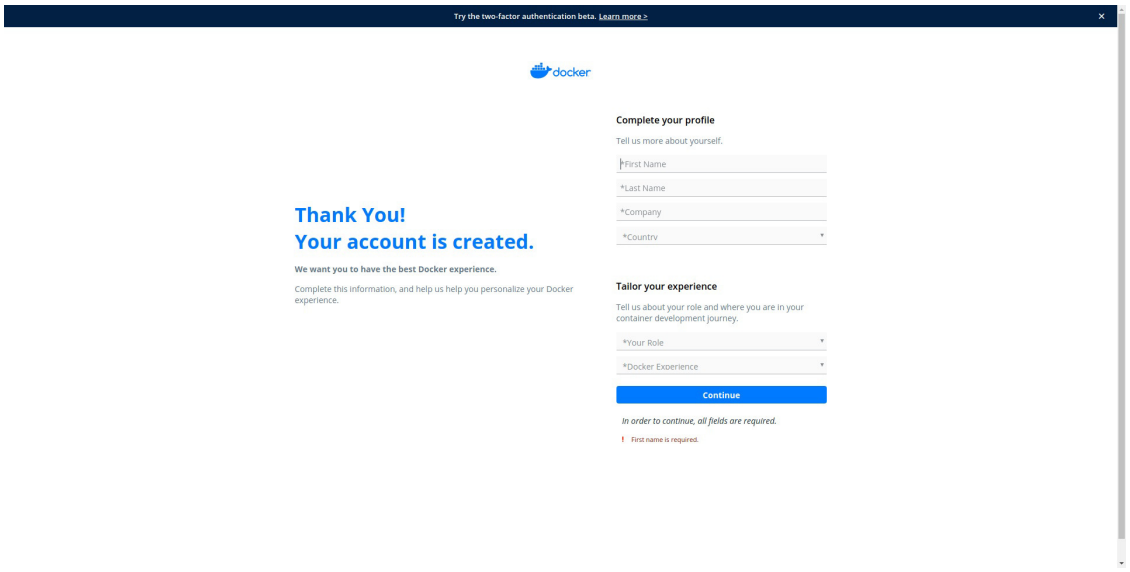
1. Acesse o link https://hub.docker.com/ e, no canto superior direito, clique em *Get Started*.



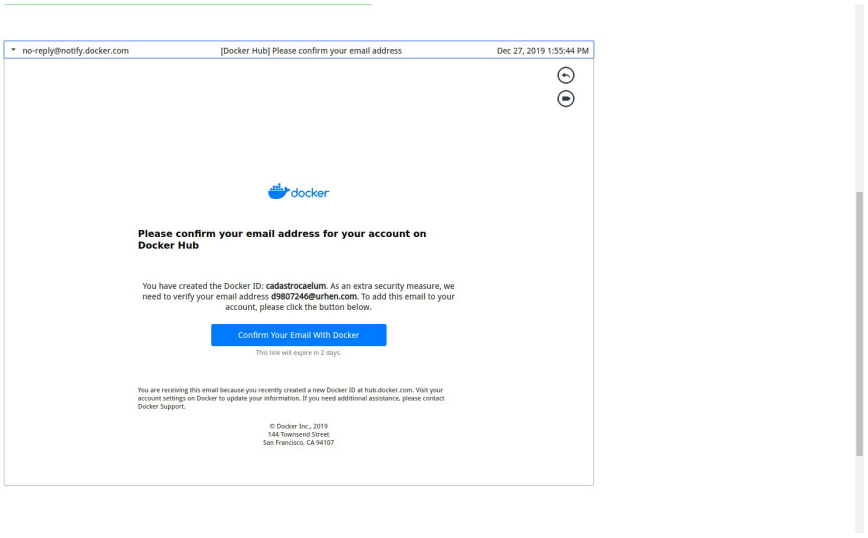
2. Preencha seus dados. O Docker ID será seu nome de usuário.



3. Caso deseje, preencha suas informações pessoais na página carregada. Não é obrigatório o preenchimento.



4. Acesse o e-mail cadastrado e confirme sua conta clicando no botão dentro da mensagem. Aguarde, você será redirecionado automaticamente para a tela de login.

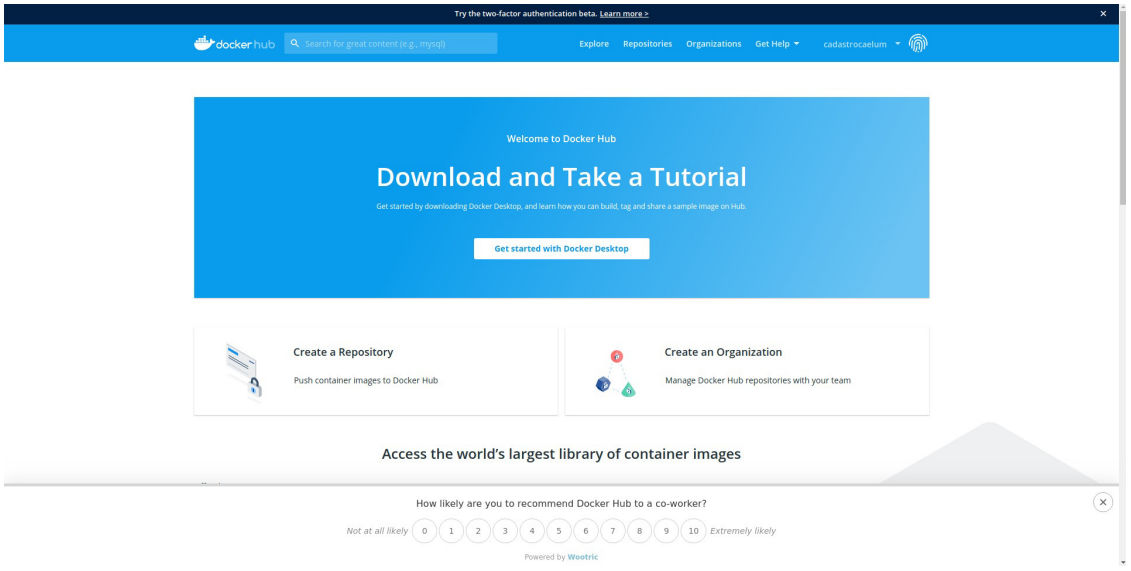


5. Faça login normalmente com os dados cadastrados previamente.



---

6. Você será redirecionado para a página principal do Docker Hub. Faça algumas procuras e buscas por imagens. Quaisquer dúvidas, pergunte ao instrutor.



5.4 PARA SABER MAIS: REGISTRY

Caso o desenvolvedor queira ter controle total sobre o processo de armazenamento e distribuição de suas imagens, muito maior do que o obtido no Docker Hub, é possível executar um registro de imagens privado. No próprio Docker Hub, é oferecido a imagem **registry**. Com ela, podemos definir um container que funcionará como um registro local, podendo ser executado apenas em nossa máquina ou em algum servidor.

O link para o repositório da imagem e como executá-la é https://hub.docker.com/_/registry.

O uso do registry em produção divide opiniões, por ser complexo e difícil de manter. A alternativa Enterprise oferecida pela Docker é o Docker Trusted Registry (DTR), uma plataforma completa que permite o gerenciamento completo de imagens, controle de acesso, segurança entre outras funcionalidades.

5.5 PUBLICANDO NOSSAS IMAGENS EM UM REGISTRO

Já aprendemos como baixar uma imagem usando o comando:

```
docker pull <nome-imagem:versao>
```

Para quem já trabalhou com `git`, fica fácil de reconhecer o comando `pull`. E, assim como o `git`, o docker também tem um comando `push` para enviar os dados. Ou seja, para enviar uma imagem, usamos o comando `docker push`:

```
docker push appjava:2.0
```

No entanto, ao executar recebemos um erro pois o Docker Hub exige autenticação:

```
The push refers to repository [docker.io/library/appjava]
f0dedbb0ba3b: Preparing
ccb5835dcd98: Preparing
77008e118980: Preparing
6cb741cb00b7: Preparing
f36b28e4310d: Preparing
91d23cf5425a: Waiting
unauthorized: incorrect username or password
```

Em outras palavras, temos que criar uma conta no Docker Hub e usar as credenciais na linha de comando antes de enviar a imagem.

O processo de login é simples! Uma vez criada a conta no Docker Hub, precisamos efetuar o login através da linha de comando:

```
docker login --username=seu-docker-id
Password: <digite sua senha>
Login Succeeded
```

Após o sucesso do login, podemos testar o comando `push` novamente. Mas ainda ocorrerá um problema de acesso:

```
docker push appjava:2.0
The push refers to repository [docker.io/library/appjava]
....
denied: requested access to the resource is denied
```

O problema é que a nossa imagem deve usar uma tag correta que contemple o *docker-id* do usuário como repositório. Só assim o Docker saberá onde a image deve ser colocada.

Vamos então usar o comando `docker tag` para corrigir o nome da imagem e usar a sintaxe `repositorio/nome-imagem`. Veja o comando:

```
docker tag appjava:2.0 <seu-docker-id>/appjava:2.0
```

E, finalmente podemos enviar (*push*) a imagem para o Docker Hub:

```
docker push <seu-docker-id>/appjava:2.0
The push refers to repository [docker.io/<seu-docker-id>/appjava]
f0dedbb0ba3b: Pushed
ccb5835dcd98: Pushed
77008e118980: Pushed
6cb741cb00b7: Pushed
f36b28e4310d: Pushed
91d23cf5425a: Pushed
2.0: digest: sha256:8cf45... size: 1574
```

Mais informações na documentação do Docker:

- https://docs.docker.com/docker-hub/publish/publish/
- https://docs.docker.com/engine/reference/commandline/push/

5.6 BUSCANDO IMAGENS DE UM REGISTRO

Podemos buscar e pesquisar imagens confortavelmente pela interface gráfica do Docker Hub. Porém, nem sempre temos acesso a interface, por exemplo, usando uma máquina virtual na nuvem como o EC2.

Nesse caso, pode ser necessário executar uma pesquisa na linha de comando através do comando `docker search` passando o nome da imagem como parâmetro:

```
docker search ubuntu
```

Como resultado, recebemos uma lista de imagens com uma descrição que pode ser enorme e confusa de ler na linha de comando. Para melhorar a pesquisa, podemos aplicar vários filtros. Por exemplo, o comando abaixo busca todas as imagens `redis` que possuem no mínimo 3 estrelas de avaliação:

```
docker search --filter stars=3 redis
```

Através da flag `filter` é possível definir outros critérios de busca como o de *imagens oficiais*:

```
docker search --filter "is-official=true" redis
```

Também é útil a possibilidade de limitar a quantidade de resultados através da flag `limit`:

```
docker search --limit=5 redis
```

5.7 EXERCÍCIO: MANIPULANDO IMAGENS EM UM REGISTRO

Objetivo

Vimos que podemos fazer diversas operações com nossas imagens. Além disso, temos um grande registro chamado `Docker Hub` onde podemos guardar diversas imagens. Nos passos abaixo veremos algumas coisas que podem ser feitas através da linha de comando.

Usaremos os comandos:

- `docker login`
- `docker push`
- `docker search`
- `docker tag`

Passo a passo

1. De início, iremos fazer o upload de nossas imagens `appjava:1.0` e `appjava:2.0` para o Docker Hub. Mas antes, precisamos pensar no seguinte: como o Docker sabe para qual repositório ele deve enviar as imagens? Isso significa que, de algum modo, precisamos nos autenticar antes. Para isso, execute o comando `docker login` e, ao ser requisitado, preencha o seu login e senha.

2. Agora sim! Para subir as nossas imagens, podemos executar o comando `docker push` . No caso, para as imagens `appjava:1.0` e `appjava:2.0` , executamos os comandos `docker push appjava:1.0` e `docker push appjava:2.0` .

3. O processo começou, porém foi interrompido por conta de um erro de acesso negado. O que pode ter causado esse erro? Tente deduzir antes de avançar ao próximo passo.

4. Lembra que imagens não oficiais sempre são criadas no seguinte formato: `nomedousuario/nomedaimagem:tag` ? Por conta disso, precisamos etiquetar a nossa imagem nesse formato. O comando necessário, é o já utilizado `docker tag` . Queremos definir a imagem `appjava:1.0` como `<seu-usuario-aqui>/appjava:1.0` . Como podemos fazer isso? Em caso de dúvidas, consulte o exercício **4.13**. Faça o processo para a imagem `appjava:1.0` e `appjava:2.0` .

5. Suba as imagens para o Docker Hub com o comando `docker push <seu-usuario-aqui>/appjava:1.0` e `docker push <seu-usuario-aqui>/appjava:2.0` . Acesse sua página no Docker Hub e observe os resultados.

6. Podemos também procurar por imagens. Por exemplo, se quisermos encontrar imagens do `redis` , podemos executar o comando `docker search redis` . Analise o resultado.

7. Podemos criterizar a busca, como por exemplo retornar imagens que só tenham a classificação de 100 ou mais estrelas. Por exemplo: `docker search dotnet --filter stars=100` .

8. Outra possibilidade, é formatar a nossa saída. Se quisermos exibir apenas os nomes das imagens encontradas, podemos executar o comando `docker search ubuntu --format="{{.Name}}"`

CAPÍTULO 6

CONTAINERS E PERSISTÊNCIA DE DADOS

6.1 CAMADAS E IMAGENS

Os containers Docker representam a aplicação ou uma parte da aplicação. Cada container é baseado em uma imagem, cada container usará a mesma imagem como base, tendo como diferença apenas a camada de escrita.

A imagem, por sua vez, é baseada em várias pequenas camadas. Essas camadas, que também são chamadas de imagens intermediárias, são geradas pela **Docker Engine** quando os comandos no `Dockerfile` são executados durante a criação da imagem Docker.

Por exemplo, vamos rever o nosso arquivo `Dockerfile` para a construção de uma aplicação Java. Ele mostra os comandos que são executados para criar com base no Ubuntu:

```
#imagem base
FROM ubuntu:18.04

#criando o diretorio de trabalho
WORKDIR /app

#copiando a aplicacao
COPY springapp.jar .

#instalando o ambiente de execução Java
RUN apt-get update && apt-get install -y openjdk-8-jre

#definindo o comando inicial do container
ENTRYPOINT java -jar springapp.jar
```

Executando o *build* da imagem, podemos ver na saída que cada comando gera uma camada intermediária, até chegarmos à imagem final, com o ID `151bf536474e`:

```
docker build -t caelum/appjava:1.0 .
Sending build context to Docker daemon  72.74MB
Step 1/5 : FROM ubuntu:18.04
18.04: Pulling from library/ubuntu
Digest: sha256:250cc6f3f3ffc5cdaa9d8f4946ac79821aafb4d3afc93928f0de9336eba21aa4
Status: Downloaded newer image for ubuntu:18.04
 ---> 549b9b86cb8d
Step 2/5 : WORKDIR /app
 ---> Using cache
 ---> 127889d23c34
Step 3/5 : COPY target/springapp.jar .
```

```
 ---> 531f56ad58a9
Step 4/5 : RUN apt-get update && apt-get install -y openjdk-8-jre
 ---> Running in b4dd1d9f4231
 .....
Removing intermediate container b4dd1d9f4231
 ---> 8b6ecf13fce4
Step 5/5 : ENTRYPOINT java -jar springapp.jar
 ---> Running in ab6fb40ef292
Removing intermediate container ab6fb40ef292
 ---> 151bf536474e
Successfully built 151bf536474e
Successfully tagged caelum/springapp:1.0
```

Podemos ver as camadas mais explícitas, usando o comando `docker history`:

```
docker image history 151bf536474e
151bf536474e  3 minutes ago  /bin/sh -c #(nop)  ENTRYPOINT ["/bin/sh" "-c…   0B
8b6ecf13fce4  3 minutes ago  /bin/sh -c apt-get update && apt-get install…   356MB
531f56ad58a9  3 minutes ago  /bin/sh -c #(nop) COPY file:7eeccd6bcf3d2957…   19.1MB
127889d23c34  3 minutes ago  /bin/sh -c #(nop) WORKDIR /app                  0B
549b9b86cb8d  2 weeks ago    /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B
<missing>     2 weeks ago    /bin/sh -c mkdir -p /run/systemd && echo 'do…   7B
<missing>     2 weeks ago    /bin/sh -c set -xe   && echo '#!/bin/sh' > /…   745B
<missing>     2 weeks ago    /bin/sh -c [ -z "$(apt-get indextargets)" ]     987kB
<missing>     2 weeks ago    /bin/sh -c #(nop) ADD file:53f100793e6c0adfc…   63.2MB
```

Repare que aparecem mais camadas do que os *steps* do comando `build .` O `docker image history` mostra todas as *layers*, também as da imagem base. Por isso, têm camadas que foram criadas "*3 minutes ago*" e outras "*2 weeks ago*". Além disso, podemos ver qual comando causou a criação da camada. Camadas que constam como `<missing>` são camadas intermediárias que foram necessárias para a criação da imagem, porém a referência não está mais em nossa máquina.

6.2 O PROBLEMA DA CAMADA DE ESCRITA

Como já falamos, quando criamos um container, uma camada de escrita é criada automaticamente. Essa camada de escrita é exclusiva do container e será removida quando o container for removido. Por conta disso, cada container em execução, se utilizar a mesma imagem que os outros, conseguirá reutilizar as mesmas camadas, fazendo com que o peso do container seja mínimo em relação ao tamanho da imagem. Caso não conseguíssemos reutilizar as mesmas imagens para execução de containers, teríamos um problema grave de uso de disco para nossas aplicações.

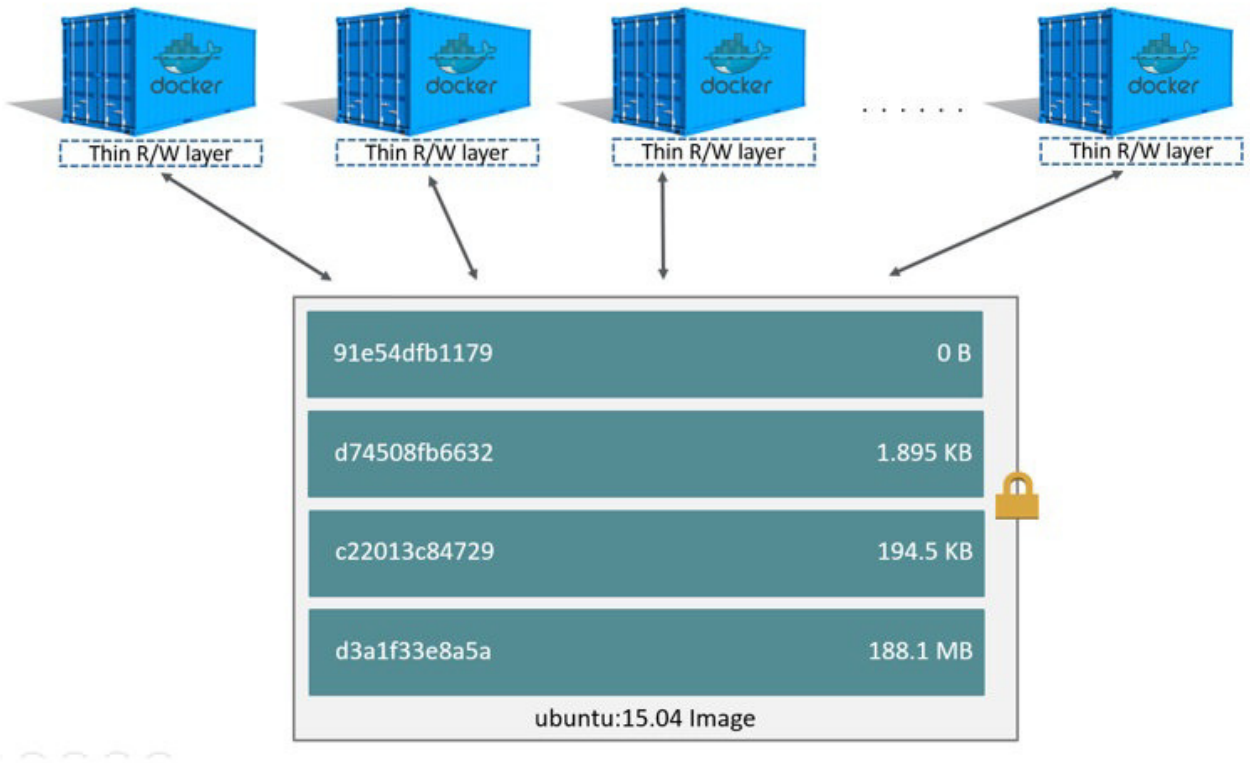


Figura 6.1: Container - Camada de escrita

Nesse momento conseguimos responder uma pergunta dos capítulos anteriores. Por que o comando `docker container ls -s` exibe dois valores na coluna **SIZE**? A resposta é simples, o valor exibido fora dos parêntesis representa o tamanho em disco apenas para a camada de escrita do container, enquanto o tamanho dentro representa o tamanho total da camada de escrita somado ao tamanho da imagem.

Aí vem uma dúvida: onde vamos guardar, por exemplo, os arquivos de logs de uma aplicação? Esses arquivos não devem ser removidos quando o container morre! Ou então, pense em um container que roda um banco de dados, um container com MySQL, por exemplo. Onde vamos guardar os registros e tabelas, já que a camada de escrita será removida junto ao container?

Não devemos nos apegar aos containers, eles devem estar sempre prontos para morrerem e então criamos outros para suprir as nossas necessidades. Quando precisamos persistir dados, não podemos confiar na camada de escrita e sim usar uma outra forma. Essa forma chama-se **volume**, o foco desse capítulo.

6.3 EXERCÍCIO OPCIONAL: CONHECENDO A CAMADA DE ESCRITA

Objetivo

Neste exercício, faremos algumas experiências com o nosso container. Criaremos alguns arquivos dentro, e, ao subirmos novamente esse e outros containers, analisaremos o resultado.

Passo a passo

1. Crie um container baseado na imagem do **Ubuntu**. em modo interativo. Quaisquer dúvidas, volte ao primeiro exercício. Podemos dar um nome customizado ao container no momento de sua criação. Para isso, utilize a *flag* `--name` e dê o nome de **ubuntu-container**. Se não conseguir, consulte a documentação em https://docs.docker.com/engine/reference/commandline/container_run/.

2. Abra um segundo terminal e execute o comando `docker container ls -s`. Repare que, diferente do comando tradicional, este exibiu uma coluna extra, informando o tamanho do container. Por que temos dois valores?

3. De volta ao primeiro terminal, execute os comandos `apt-get update && apt-get install net-tools`. Ao terminar, execute o comando `ifconfig`. Repare que algumas informações sobre a rede do container foram exibidas.

4. Novamente no segundo terminal, execute o comando `docker container ls -s`. Repare que os valores mudaram. O que isso quer dizer?

5. No primeiro terminal, aperte as teclas Ctrl + D para encerrar o container. Para confirmar que o container realmente foi encerrado, execute mais uma vez o comando `docker container ls -s` e veja que o container criado previamente não existe mais.

6. Ainda no primeiro terminal, crie outro container, assim como no passo 1 deste exercício.

7. Tente executar o comando `ifconfig`. O que aconteceu?

8. No segundo terminal, execute o comando para listar os containers, exibindo seus tamanhos. Qual tamanho está sendo exibido?

9. Debata com os colegas e instrutor como esse problema pode ser contornado.

10. Caso criássemos outro container, quanto ele pesaria?

6.4 CRIANDO O PRIMEIRO VOLUME

Para criar um volume, a documentação do Docker recomenda hoje em dia a utilizar a *flag* `--mount`, ao invés da antiga *flag* `-v` (ou `--volume`). Vamos mostrar o `--mount`, que é mais claro e explícito, e

depois, no resumo final, mostraremos também o `-v`.

Para criar um volume, devemos usar o comando `docker volume`. Ou seja, criamos um volume totalmente independente do container e daremos um nome para ele. Este desacoplamento é importante, pois permite associar o mesmo volume com vários containers.

Segue o comando inicial para a criação do volume:

```
docker volume create app-logs
app-logs
```

Criamos um volume chamado **app-logs**, mas qual é a pasta concreta na máquina *host*? Para descobrir, podemos listar e inspecionar o volume:

```
docker volume ls
DRIVER              VOLUME NAME
local               app-logs
```

Não recebemos muita informação, apenas o fato que usamos um *driver local*. Vamos tentar o `inspect`:

```
docker volume inspect app-logs
[
    {
        "CreatedAt": "2020-01-02T18:34:36Z",
        "Driver": "local",
        "Labels": {},
        "Mountpoint": "/var/lib/docker/volumes/app-logs/_data",
        "Name": "app-logs",
        "Options": {},
        "Scope": "local"
    }
]
```

Agora sim, repare no `Mountpoint`, que define o local onde o volume armazena seus dados em nossa máquina *host*. Certo, definimos o volume, que possui um nome, mas agora falta usar esse volume no container.

Para tal, usamos a já mencionada *flag* `--mount` junto ao comando `docker run`! Simples assim:

```
docker run --mount source=app-logs,target=/logs caelum/appjava:1.0
```

O `source` é o volume nomeado, o `target` é a pasta dentro do container. Ou seja, se gravarmos algo dentro do container, na pasta `/logs`, será acessível na máquina *host*, na pasta `/var/lib/docker/volumes/app-logs/_data` e vice-versa.

Podemos analisar o `Mountpoint` concreto também pelo comando `docker container inspect`. Em um novo terminal, vamos descobrir o ID do container em execução:

```
docker container ls
CONTAINER ID        IMAGE               COMMAND                  CREATED
c936314f0e6d        caelum/appjava:1.0  "/bin/sh -c 'java -j…"   30 seconds ago
```

E usando comando `inspect`, já com a *flag* `--format`, mostra o `Mountpoint`:

```
docker container inspect --format '{{ .Mounts }}' c936314f0e6d

... /var/lib/docker/volumes/app-logs/_data ...
```

Volume através da flag -v

Ao usar comando `docker run`, também podemos usar a *flag* `-v` (ou `--volume`), com o mesmo efeito:

```
docker run -v app-logs:/logs caelum/appjava:1.0
```

Repare no separador `:` entre o nome do volume e o diretório do container. A sintaxe fica um pouco mais resumida, porém, perdemos em expressividade.

Uma das vantagens dessa abordagem é que conseguimos lidar com os volumes da mesma maneira que vínhamos fazendo antes, através de comandos do próprio Docker. Como o Docker será 100% responsável por gerenciar e manipular os volumes, não teremos preocupações quanto a isso.

6.5 EXERCÍCIO: USANDO VOLUMES

Objetivo

Neste exercício, veremos como o uso de volumes gerenciados pelo Docker pode facilitar a nossa vida, além de tornar menos "perigosa" a persistência de dados.

Criamos o volume explicitamente e usamos a *flag* `--mount` no comando `run`, que é mais explícito e verboso do que o `-v`, além de criar um volume explicitamente pelo comando `docker volume create`.

Passo a passo

1. Antes de mais nada, remova todos os containers criados para evitar possíveis conflitos:

   ```
   docker container rm $(docker container ls -qa) --force
   ```

2. Dessa vez, para fazermos uso de volumes gerenciados diretamente pelo Docker, executaremos o comando `docker run` usando a *flag* `--mount`. O comando será:

   ```
   docker run -p 9000:8080  \
     --mount target=/output-container \
     --name container-persistido-v2 \
     appjava:2.0
   ```

3. Em sua máquina, abra o navegador e acesse *localhost:9000/geraArquivos*. Com isso, geramos alguns arquivos dentro da pasta `output-container`.

4. O questionamento levantado agora, é: como não definimos o local onde os dados serão persistidos em nossa máquina *host* dentro do comando `docker run`, onde os dados estão? Qual comando pode ser útil para descobrirmos isso? Tente deduzir antes de avançar para o próximo passo.

5. O comando que resolverá o nosso problema é o comando `docker inspect`. Com ele, conseguimos diversas informações sobre o container, e uma delas é a informação sobre pontos de montagem:

   ```
   docker container inspect --format='{{json .Mounts}}' container-persistido-v2
   ```

   Repare que, usando os artifícios de formatação, podemos retornar um trecho específico de interesse da nossa inspeção.

6. Veja que temos informações como tipo da persistência, nome, origem, destino e afins. O valor de `Source` indica exatamente a localização de onde os dados estão sendo persistidos em nossa máquina. Já o valor de `Name` indica o nome do volume gerado para armazenar os dados.

7. Abra um novo terminal e acesse o caminho especificado em `Source`. Por exemplo `cd /var/lib/docker/volumes/6870cc6d8d1c0991dee00928f9fa6c21c58ffb3b62e24c598826d27dd330c915/_data`. Dica: você pode copiar o caminho através do comando Ctrl + Shift + C ou usar as sugestões de preenchimento do terminal, através da tecla TAB.

8. Execute o comando `ls` e veja que os arquivos gerados dentro da pasta `output-container` estão aparecendo na pasta atual.

9. Afinal, sempre precisaremos passar por esse processo de inspeção para utilizar um volume? A resposta é... não. Podemos criar e manipular volumes diretamente através do Docker.

10. Para evitar quaisquer tipos de conflitos, vamos parar e remover todos os containers em execução/criados até o momento. Para isso, podemos utilizar o comando:

    ```
    docker container rm $(docker container ls -qa) --force
    ```

11. Precisamos agora criar o nosso volume, devidamente nomeado. Podemos fazer isso através do comando `docker volume create`. Execute o comando:

    ```
    docker volume create meu-volume
    ```

    Em seguida, acesse o caminho (através do comando `cd`) `/var/lib/docker/volumes/`. Execute o comando `ls` dentro dessa pasta e veja que foi criado um diretório chamado `meu-volume`.

12. Em seguida, execute o comando:

    ```
    docker run -p 9000:8080 \
      --mount source=meu-volume,target=/output-container \
      --name container-persistido-v2 \
      appjava:2.0
    ```

    Acesse mais uma vez o link *localhost:9000/geraArquivos*.

3. Acesse a pasta `/var/lib/docker/volumes/meu-volume/_data` e veja que os arquivos gerados estão dentro desse diretório.

4. Acesse o link https://docs.docker.com/storage/volumes/ e veja outros possíveis comandos para a utilização de volumes.

6.6 CONHECENDO O BIND MOUNT

Vimos a criação e uso de um volume nomeado. Como alternativa ao volume, existe um outro tipo de ponto de montagem, o ***bind mount***. Esse tipo de ponto de montagem permite montar uma pasta qualquer do *host* para o container. A montagem usa o caminho absoluto, que normalmente é considerado má pratica. Veja o exemplo:

```
docker run -it --mount type=bind,source="$(pwd)/logs",target=/logs ubuntu:18.04
```

Repare que especificamos o tipo explicitamente, aqui `type=bind` , e no *source* usamos um caminho absoluto (no lugar de um volume nomeado). Criamos uma associação direta entre uma pasta do *host* e uma pasta do container, sem ter a abstração do volume. Ao usar o comando `docker container inspect` , podemos ver o tipo definido:

```
docker container ls
CONTAINER ID        IMAGE               ...
7a5c52b370be        ubuntu:18.04        ...

docker container inspect 7a5c
...
"Mounts": [
            {
                "Type": "bind",
                "Source": "/home/caelum/apps/logs",
                "Destination": "/logs",
                "Mode": "",
                "RW": true,
                "Propagation": "rprivate"
            }
        ],
...
```

Resumindo, ambos, volumes e *bind mounts*, permitem compartilhar arquivos entre a máquina *host* e o container, para manter os dados mesmo após a parada do container. O recomendado atualmente pelo Docker é que se utilize volumes no lugar de bind mounts, pois assim temos a total capacidade do Docker gerenciar os volumes para nós e não temos o risco de modificar ou perder o caminho absoluto da pasta utilizada para o bind mount.

6.7 EXERCÍCIO: PERSISTINDO DADOS ATRAVÉS DE BIND MOUNTS

Objetivo

Neste exercício, através do uso de volumes, contornaremos a limitação da camada de escrita, que vimos no exercício anterior. Inicializaremos um container com um ponto de montagem do tipo *bind*.

Passo a passo

1. Antes de mais nada, remova todos os containers criados para evitar possíveis conflitos:

   ```
   docker container rm $(docker container ls -qa) --force
   ```

2. Crie uma pasta chamada `dados-bind-mounts` no seu `Desktop`. Deixe-a vazia.

3. A pasta criada no passo anterior será responsável por salvar os dados de dentro do container em nossa máquina host. Agora, precisamos criar um container baseado em sua imagem `appjava:2.0` e mapeando a porta 8080 do container na porta 9000 da sua máquina *host*. Porém, como faremos o mapeamento de dados de dentro do container para a nossa máquina *host*?

4. Para fazermos o *bind* de dados, por assim dizer, precisamos passar mais uma informação no momento da criação do nosso container. Essa informação será passada através da *flag* `-v`. O comando será:

   ```
   docker run -p 9000:8080 -v ~/Desktop/dados-bind-mounts:/output-container --name container-persist
   ido appjava:2.0
   ```

   Isso significa que, o que for feito dentro da pasta `output-container`, dentro do container, refletirá na pasta `dados-bind-mounts`, em nossa máquina *host*.

5. Em sua máquina, abra o navegador e acesse *localhost:9000/geraArquivos*. Com isso, geramos alguns arquivos dentro da pasta `output-container`.

6. Como podemos verificar se os dados foram gerados dentro do container? Afinal, o terminal está travado com a execução da nossa aplicação. Podemos, através do Docker, executar comandos dentro de containers que já estão em execução. Para isso, podemos utilizar o comando `docker exec`.

7. Queremos acessar o terminal do nosso container chamado `container-persistido`, para que possamos verificar se o conteúdo está lá dentro. Então, utilizando o comando `docker exec`, podemos executar um comando que acessa o terminal. Para isso, vamos executar o comando `docker exec -it container-persistido bash`. Com isso, estamos executando, em modo interativo, o comando `bash` (nada mais é do que o terminal) dentro do nosso container chamado `container-persistido`.

8. Veja que agora estamos acessando o terminal do mesmo container. Para acessarmos a pasta `output-`

`container`, podemos executar o comando `cd output-container`. E, para listar o conteúdo dentro dela, podemos executar o `ls`. Veja que os arquivos gerados pela aplicação estão dentro da pasta.

9. Em sua máquina *host*, abra a pasta `dados-bind-mounts` em sua área de trabalho e veja que os mesmos arquivos estão contidos ali.

10. Experimente remover um ou mais arquivos dessa pasta `dados-bind-mounts`. Execute o comando `ls` novamente dentro do terminal do container. O que aconteceu?

11. Quais possíveis problemas do comando do passo 3? Veja mais informações em https://docs.docker.com/storage/bind-mounts/.

6.8 TRABALHANDO COM TMPFS

Existe uma terceira opção, o **tmpfs**, mas disponível apenas no Linux como máquina *host*.

Ao contrário dos volumes e *bind mounts*, um *tmpfs* é temporário, ou seja volátil, persistido apenas na memória principal da máquina *host*. Quando o container para, a montagem *tmpfs* é removida e os arquivos gravados não serão mantidos. Isso pode ser útil para aplicações que precisam de desempenho no ponto de montagem (por exemplo, para caches).

Ou seja, temos um local externo para gravar dados, no entanto apenas em memória. Para usar o *tmpfs*, basta definir o tipo, como apresentado no comando abaixo:

```
docker run -d -it --mount type=tmpfs,destination=/app nginx:latest
```

Dessa forma, temos os 3 tipos, *volumes* e *bind mounts* e *tmpfs* no Linux:

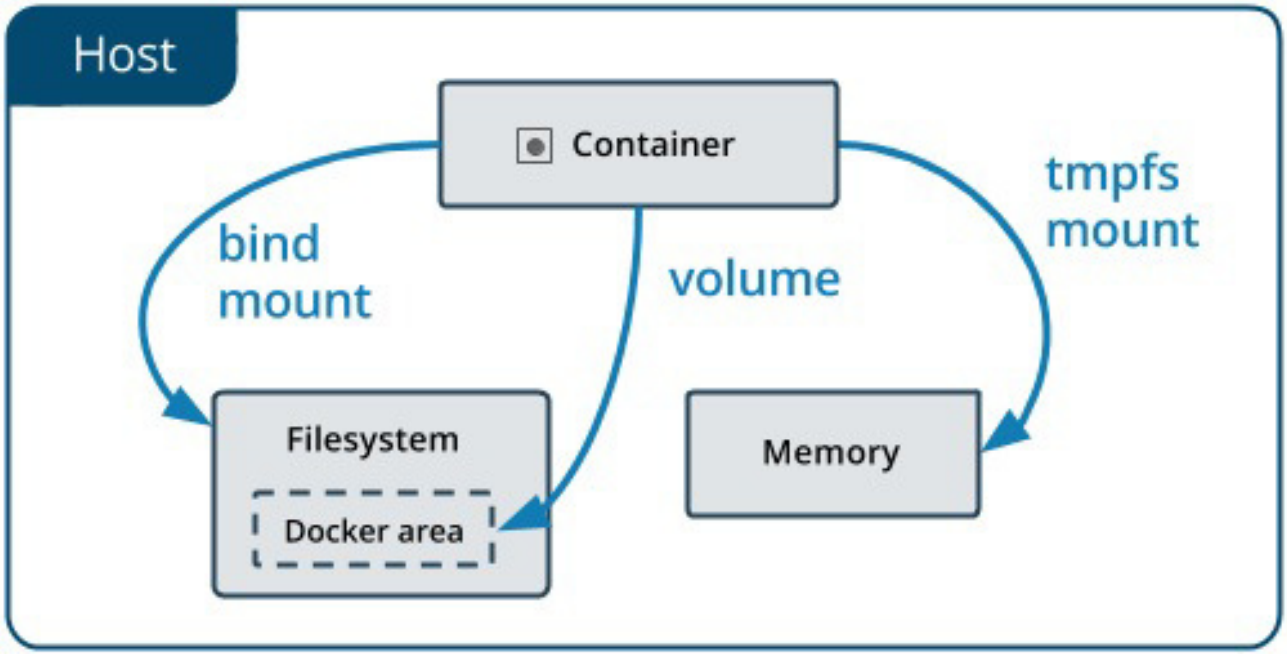


Figura 6.2: Tipos de volumes

6.9 VARIÁVEIS DE AMBIENTE

Quando criamos uma imagem, ela deve ser auto-contida. Isso quer dizer que ela possui todos os arquivos e dependências que são precisos para rodar o container. No entanto, nem sempre podemos e desejamos configurar tudo na hora do *build* da imagem.

Por exemplo, pode ser necessário definir o ambiente que o container está rodando. Nesse contexto, o ambiente tem o significado de *desenvolvimento*, *teste*, *homologação* e *produção*. Pode ser que a mesma imagem seja utilizada em cada ambiente (ou *stage*), mas assuma dinamicamente novas configurações para, por exemplo, redefinir o nível de log ou habilitar/desabilitar otimizações na aplicação.

Outro exemplo clássico são as credenciais do banco de dados. Na hora do *build* da imagem, não queremos e não podemos definir a senha do banco em produção fixo pela aplicação. As credenciais devem ser definidos no ambiente de produção, normalmente pelo administrador desse ambiente.

Como solução, o Docker permite declarar **variáveis de ambiente**. Essas variáveis são definidas na hora de rodar o container, através da *flag* `-e` (ou `--env`). Veja o exemplo da documentação da imagem do banco PostgreSQL:

```
docker run --name some-postgres -e POSTGRES_PASSWORD=mysecretpassword -d postgres
```

Repare na *flag* `-e`, seguida pelo parâmetro `chave=valor` (a *flag* `-d` é para rodar de maneira *desacoplada*). O parâmetro `POSTGRES_PASSWORD` é passado para o container e pode ser usado pela aplicação. Para prova de conceito, veja o exemplo abaixo:

```
docker run -it -e senha=1234 ubuntu
root@894b7858810c:/# echo $senha
1234
```

6.10 VARIÁVEIS DE CONSTRUÇÃO: ARG

Já aprendemos que variáveis do tipo *ENV* são úteis para personalizar o container, ou seja, no hora de usar o comando `run`. Existe um segundo tipo de variável chamado *ARG*, bem parecido com *ENV*, no entanto, servem para personalizar a criação da imagem.

No exemplo abaixo, estamos definindo um *ARG*, com o nome `JAR_FILE_NAME` e o valor padrão `springapp.jar`:

```
FROM ubuntu:18.04
ARG JAR_FILE_NAME=springapp.jar
WORKDIR /app
COPY target/$JAR_FILE_NAME .
RUN apt-get update && apt-get install -y openjdk-8-jre
ENTRYPOINT java -jar springapp.jar
```

Repare que usamos o *ARG* na instrução `COPY` e rodando o comando `docker build .` , usará o valor `springapp.jar` nessa instrução. Podemos sobrescrever o valor padrão através da *flag* `--build-arg` , por exemplo:

```
docker build --build-arg JAR_FILE_NAME=javaapp.jar .
```

Ainda temos um problema no `Dockerfile` , pois *não* usamos o *ARG* na última instrução:

```
ENTRYPOINT java -jar springapp.jar
```

O mais intuitivo seria usar o *ARG* aqui também, no entanto o `ENTRYPOINT` será executado pelo container e as variáveis de construção estão disponíveis apenas na construção (*build*) da imagem! Então, como resolver?

Já aprendemos que variáveis do tipo *ENV* servem para personalizar o container, ou seja, vamos usar ambos, *ENV* e *ARG*. Vejo o exemplo:

```
FROM ubuntu:18.04
WORKDIR /app
ARG JAR_FILE_NAME=springapp.jar
COPY target/$JAR_FILE_NAME .
RUN apt-get update && apt-get install -y openjdk-8-jre

#nova variável do tipo ENV JAR_NAME
#baseado no valor da variavel do tipo ARG
ENV JAR_NAME=$JAR_FILE_NAME

#usando a variável do tipo ENV
ENTRYPOINT java -jar $JAR_NAME
```

Agora sim temos um `Dockerfile` que personaliza tanto o *build* quanto o *run*!

Mais informações na documentação do Docker:

- https://docs.docker.com/engine/reference/commandline/run/
- https://docs.docker.com/engine/reference/builder/

6.11 EXERCÍCIO: TRABALHANDO VARIÁVEIS DE AMBIENTE.

Objetivo

Agora, faremos um uso mais robusto de volumes, juntamente com banco de dados e variáveis de ambiente. Utilizaremos a imagem do **MySQL** para esse exercício.

Passo a passo

1. Primeiramente, crie um volume chamado `volume-db` :

   ```
   docker volume create volume-db
   ```

2. Como utilizaremos a imagem do `mysql:5.7`, o comando para executar o container agora será o seguinte:

```
docker run \
     --publish 3000:3306 \
     --mount source=volume-db,target=/var/lib/mysql \
     --name container-db \
     mysql:5.7
```

   Como o **MySQL** roda por padrão na porta **3306**, estamos indicando que a porta **3306** do container será refletida na porta **3000** da nossa máquina. Além disso, estamos guardando em nosso `volume-db` todo o conteúdo que venha a ser gravado no banco.

   Repare que o comando anterior funcionou parcialmente, pois é necessário indicarmos se o nosso banco terá senha ou não:

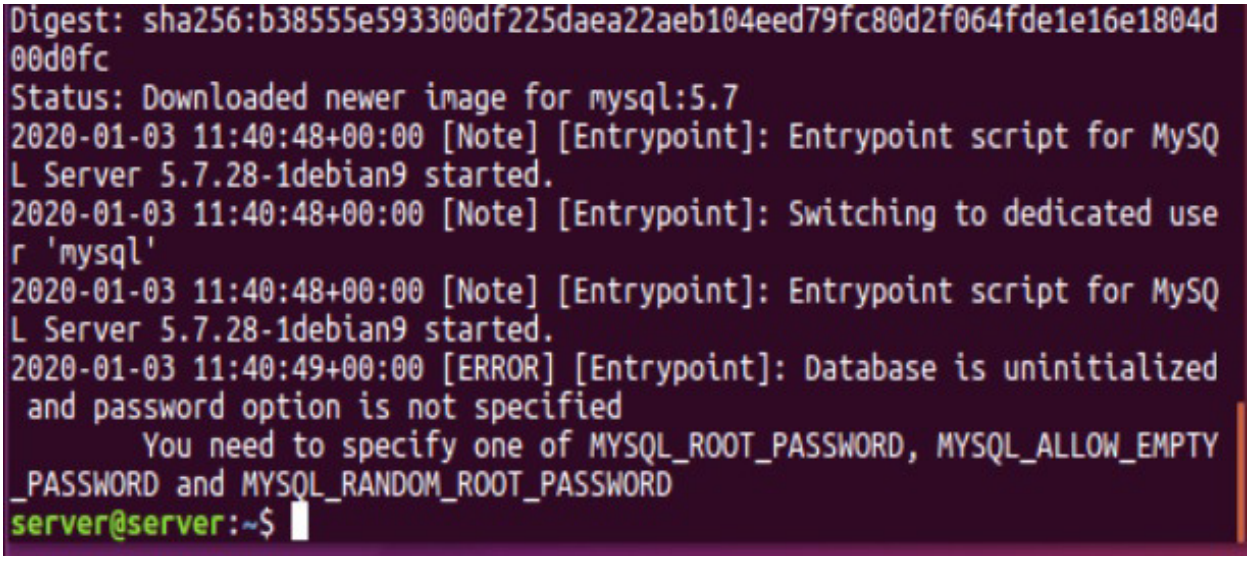


Figura 6.3: Erro MySQL

3. Para resolvermos esse problema, precisamos passar, como variável de ambiente, que não utilizaremos senha no nosso banco. Antes de executar esse comando, remova o container criado anteriormente, com o comando:

```
docker rm container-db --force
```

   Agora, vamos passar uma variável de ambiente. Para tal, use a *flag* `-e` (ou `--env`). Logo, o comando será:

```
docker run \
  --publish 3000:3306 \
  --mount source=volume-db,target=/var/lib/mysql \
  --env MYSQL_ALLOW_EMPTY_PASSWORD=yes \
  --name container-db mysql:5.7
```

4. Podemos acessar o banco de dados criado dentro do container. Abra um *segundo terminal* e execute o comando:

```
mysql -h localhost -P 3000 --protocol=tcp -u root
```

5. No segundo terminal, dentro do **MySQL**, execute o comando:

```
CREATE DATABASE primeirodb;
```

6. Para confirmar se tudo deu certo, abra um terceiro terminal e execute o comando:

```
docker exec -ti container-db bash
```

   Ainda nesse terceiro terminal, execute o comando:

```
mysql -u root
```

   Para visualizar os bancos, execute:

```
SHOW DATABASES;
```

   O comando exibe todos os bancos criados e deve mostrar o `primeirodb`.

7. Caso criássemos um novo container, como poderíamos reutilizar os dados do container anterior?

Capítulo 7

COMUNICANDO CONTAINERS

7.1 CONHECENDO A REDE BRIDGE

A arquitetura baseado em *Microservices* está sendo cada vez mais utilizada, e o Docker por sua vez, é a escolha favorita para empacotar e rodar os serviços. Nesse contexto, o Docker isola a execução desse processo que faz parte de uma aplicação maior. Ou seja, para a aplicação como um todo funcionar, os containers precisam se comunicar via rede para passar e obter informações.

Até agora, não usamos nenhuma rede explicitamente pois, por padrão, o Docker já possui uma principal. A rede padrão do Docker é chamada *bridge*. Para ser correto, o *bridge* é um **driver de rede** e existem vários outros, como *host* e *overlay*. O driver *bridge* cria uma ponte entre os containers que usam o mesmo driver em um mesmo host. Isso faz que os containers dessa rede possam se comunicar:

Por exemplo, rodando um container `alpine`:

```
docker run -it alpine:3
```

E inspecionando em outro terminal, podemos ver a rede e endereço IP:

```
docker container inspect <id-container>
...

"Networks": {
  "bridge": {
    "IPAMConfig": null,
    "Links": null,
    "Aliases": null,
    "NetworkID": "e74d79d28e0...",
    "EndpointID": "eced0cf5aa...",
    "Gateway": "172.17.0.1",
    "IPAddress": "172.17.0.2",
    "IPPrefixLen": 16,
...
  }
}
...
```

Subindo um segundo container podemos usar esse endereço de IP para se conectar. É importante entender que a rede `bridge` se trata de uma rede interna entre containers apenas, e totalmente isolada da rede do host. Logo, o endereço IP da máquina *host* não pode ser usado para se referenciar ao container. Para acessar o container a partir do *host*, devemos usar o mapeamento de portas ou o driver **host**, que veremos a seguir.

Como existem comandos específicos para gerenciar containers (`docker container`), para trabalhar com imagens (`docker image`) e volumes (`docker volume`) e outros recursos, também existe um comando para gerenciar as redes: `docker network`

Por exemplo, para listar as redes que já existem, usamos:

```
docker network ls
NETWORK ID          NAME                DRIVER          SCOPE
7fca4eb8c647        bridge              bridge          local
cf03ee007fb4        host                host            local
```

Aqui temos um ID e nome (como sempre), além do driver e escopo. Você já conhece o driver *bridge*, logo veremos o driver *host*.

Mais informações sobre a rede *bridge*:

- https://docs.docker.com/network/
- https://docs.docker.com/network/bridge/

7.2 CONHECENDO A REDE HOST

Vimos que o driver *bridge* isola os containers totalmente da rede do *host*. O driver *host*, ao contrário, compartilha a rede do host com o container. Em outras palavras, o container faz parte da mesma rede do *host*.

Como o driver `host` não é o padrão, devemos configurá-lo explicitamente na hora de rodar o container. Isso é feito pela flag `--network` .

Veja o exemplo que usa a imagem `nginx` . Repare a flag `--network` com o parâmetro `host` :

```
docker run --rm -d --network host --name cont-nginx nginx
```

> **@NOTE**
>
> O network host é melhor testar no linux pois no windows e mac tem o docker desktop que sobe uma vm para rodar docker. ou seja, precisamos conhecer a IP da VM para testar a rede host.

E inspecionando novamente o container podemos ver a rede com o nome "host" e nenhum IP:

```
"Networks": {
    "host": {
        "IPAMConfig": null,
        "Links": null,
        "Aliases": null,
        "NetworkID": "5ac54f7fca...",
        "EndpointID": "6d9caead8...",
        "Gateway": "",
```

```
        "IPAddress": "",
...
    }
}
```

Qual IP devemos então usar para acessar o container? Já que o container faz parte da rede do host, a mesma IP do host! Acessando `curl http://localhost:80` deve imprimir a pagina padrão do servidor Nginx.

Resumindo, se você usa a rede *host* para um container, a rede dele *não* será isolada do host do Docker e o container *não* recebe seu próprio endereço IP alocado. Devemos usar o **ip** do **host** para acessar a aplicação.

No driver *bridge*, há justamente esse isolamento entre container e host. O container possui seu próprio IP.

Mais informações sobre a rede *host*:

- https://docs.docker.com/network/host/
- https://docs.docker.com/network/network-tutorial-standalone/
- https://docs.docker.com/network/network-tutorial-host/

7.3 PARA SABER MAIS: REDES MACVLAN

O Docker também provê um outro tipo de rede, a **macvlan**. Em algumas aplicações, principalmente as legadas, há a necessidade de haver uma conexão direta com a rede física. Com a rede **macvlan**, conseguimos atribuir um endereço MAC para cada interface de rede de cada container, fazendo então com que a interface se comunique diretamente com a rede física.

Para uma rede macvlan funcionar da maneira adequada, precisamos designar a sub-rede e gateway necessários para tal. Podemos definir isso no momento da criação da rede:

```
docker network create -d macvlan \
  --subnet=172.16.86.0/24 \
  --gateway=172.16.86.1 \
  mynet
```

Essas e outras informações você pode acessar em https://docs.docker.com/network/macvlan/

7.4 CRIANDO NOSSA PRÓPRIA REDE

Já sabemos listar as redes existentes pelo comando `docker network ls`. Porém, como podemos fazer para criar a nossa própria rede? Para tal, usaremos o comando `docker network create` e a flag `-d` (ou `--driver`) para especificar o driver que utilizaremos, seja ele *bridge* ou *macvlan*:

```
docker network create -d bridge rede-bridge
```

ou

```
docker network create -d macvlan --subnet=172.16.86.0/24 --gateway=172.16.86.1 rede-mcvlan
```

E ao listar as redes aparece o nome de nossas novas redes:

```
docker network ls
NETWORK ID          NAME         DRIVER     SCOPE
e74d79d28e08        bridge       bridge     local
5ac54f7fca4b        host         host       local
f533088455a9        rede-bridge  bridge     local
9e6e12d2442e        rede-mcvlan  macvlan    local

...
```

No caso da rede *bridge* que criamos, esse tipo de rede é chamado de **user-defined bridge**. Para associar a rede nova ao container usamos o nome junto à flag `--network`:

```
docker run -it --network rede-bridge --name alpine1 alpine:3
```

Ao inspecionar, aparece a nossa rede e o endereço IP da rede interna:

```
"Networks": {
    "rede-bridge": {
...
        "NetworkID": "4c0ea52e37...",
        "EndpointID": "228450d70...",
        "Gateway": "172.24.0.1",
        "IPAddress": "172.24.0.2",
...
    }
}
```

O interessante é que podemos trocar a rede de um container em execução basta usar o comando `docker network connect`. Por exemplo:

```
docker network connect <nome-da-rede> <id|nome-do-container>
```

Assim podemos adicionar um container em novas redes.

Rede bridge: default vs user-defined

Por que criamos a nossa própria rede usando o driver *bridge* se já existe uma rede padrão com exatamente o mesmo driver?

A resposta não é tão simples. Pois, além do driver, existem outras regras que são aplicadas, porém em termos gerais podemos dizer que a rede padrão é considerada uma rede legada. Ela sim simplifica o uso do Docker no inicio pois é usada automaticamente e aplica algumas regras de acesso, no entanto não

deve ser usado em produção. Em produção sempre devemos aplicar a nossa própria rede pois:

- as portas são publicadas automaticamente na rede interna
- a rede pode ser associada ao container em execução
- é uma rede configurável
- faz a resolução DNS automaticamente
- não compartilha variáveis de ambiente entre containers

Em geral, sempre dê preferencia ao uso de uma rede *user-defined bridge* que traz mais flexibilidade e é superior à rede padrão.

Mais informações sobre a rede *host*:

- https://docs.docker.com/network/bridge/#use-the-default-bridge-network

7.5 EXERCÍCIO: CONCEITOS DE REDES DO DOCKER

Objetivo

Nesse exercício, fixaremos o conteúdo visto na explicação anterior, além de colocar em prática alguns conceitos importantes sobre redes com Docker. Além disso, utilizaremos a imagem `alpine` e veremos como ela pode ser útil. Vamos usar os comandos novos:

- `docker network create`
- `docker network ls`
- `docker network connect`

Passo a passo

1. Primeiramente, crie dois containers em modo interativo, um em cada terminal, chamados **alpine1** e **alpine2** utilizando a imagem `alpine:3` :

   ```
   docker run -it --name alpine1 alpine:3
   ```

   e

   ```
   docker run -it --name alpine2 alpine:3
   ```

2. Agora, abra um terceiro terminal e descubra através do comando `docker inspect` , o endereço IP dos dois containers recém-criados.

   ```
   docker inspect alpine1 --format="{{.NetworkSettings.IPAddress}}"
   ```

   e

   ```
   docker inspect alpine2 --format="{{.NetworkSettings.IPAddress}}"
   ```

3. Volte ao terminal do container `alpine1` e tente se comunicar com o container `alpine2` a partir de seu endereço IP. Para isso, execute o seguinte comando:

   ```
   ping <ENDEREÇO_IP_DO_CONTAINER_ALPINE2>
   ```

   Veja que a comunicação está sendo feita sem problemas. Você também pode tentar fazer o processo inverso. No terminal do container `alpine2` , tente:

   ```
   ping <ENDEREÇO_IP_DO_CONTAINER_ALPINE1>
   ```

4. Porém, qual é o problema dessa abordagem? Conforme mais containers aparecerem, mais endereços IP's teremos que procurar. E mais, teremos que saber exatamente a qual container cada endereço IP se refere. A solução para esse problema, é que podemos também comunicar containers via nome. Mas, para isso, precisamos criar nossa própria rede. No terceiro terminal:

   ```
   docker network create -d bridge primeira-rede
   ```

   Você pode listar as redes existente com o comando:

   ```
   docker network ls
   ```

5. Precisamos agora conectar os dois containers em nossa rede. Para isso, podemos executar o comando:

   ```
   docker network connect primeira-rede alpine1
   ```

   e

   ```
   docker network connect primeira-rede alpine2
   ```

6. Agora os dois containers estão conectados em uma mesma rede criada por nós. Isso significa que podemos comunicá-los via nome. Agora teste nos terminais dos containers `alpine1` e `alpine2` , respectivamente:

   ```
   ping alpine2
   ```

   e

   ```
   ping alpine1
   ```

7. Veja outros comandos e informações em https://docs.docker.com/engine/reference/commandline/network/.

7.6 ACESSANDO SERVIÇOS EM OUTROS CONTAINERS

Já aprendemos como subir um banco de dados e também aprendemos como subir a nossa própria aplicação. No exemplo, usamos uma API Java. Nada impede então subir uma *stack* de aplicações, correto?

Correto, desde que os dois containers façam parte da mesma rede, eles podem se comunicar sem

problemas.

Para subir o mysql usamos:

```
  docker run --name db --publish 3000:3306 --network network-db --env MYSQL_ALLOW_EMPTY_PASSWORD=yes
mysql:5.7
```

E para subir a nossa webapp:

```
  docker run -p 9000:8080 --network=network-db appjava:2.0
```

Assumindo que a rede `network-db` foi criada, os dois containers podem se conectar sem dificuldade. Além disso, o segundo container pode usar o nome do primeiro como nome DNS. Ou seja, o comando `ping db` funcionará, o que facilita muito o desenvolvimento.

O que ainda atrapalha são os parâmetros e flags, visto que são muito imperativos e propensos ao erro humano. Repare que não tem *infraestrutura como código* na hora de rodar os containers. Na construção da imagem temos um `Dockerfile` , mas na hora de rodar continuamos usando comandos e várias flags. Isso não pode ficar assim e vamos resolver essa questão no próximo capítulo, mas agora é a hora do exercício.

7.7 EXERCÍCIO: COMUNICANDO DIFERENTES CONTAINERS

Objetivo

Nesse exercício, utilizaremos os conceitos do exercício anterior para comunicar a aplicação já utilizada anteriormente com um banco de dados.

Passo a passo

1. Antes de mais nada, remova todos os containers criados para evitar possíveis conflitos:

   ```
   docker container rm $(docker container ls -qa) --force
   ```

2. Crie uma rede chamada `network-db` utilizando o driver `bridge` :

   ```
   docker network create --driver bridge network-db
   ```

   Você pode listar as redes existentes com o comando `docker network ls` .

3. Crie e execute um container que use o banco MySQL com as seguintes características:

   - use a imagem `mysql` na versão 5.7
   - mysql deve subir *sem senha*
   - defina o nome do container como `db`
   - mapeie a porta `3306` do container na porta `3000` de sua máquina
   - defina a rede do container com `--network=network-db`

- não use volumes

Caso tenha dúvidas, pergunte ao instrutor.

4. Em outro terminal, execute um container a partir da imagem `appjava:2.0`. Também adicione-o na rede `network-db`:

```
docker run -p 9000:8080 --network=network-db appjava:2.0
```

5. Acesse em seu navegador o link `localhost:9000/geraBancoV2`.

6. Abra um novo terminal e execute o comando:

```
mysql -h localhost -P 3000 --protocol=tcp -u root
```

7. Dentro do banco execute o comando `SHOW DATABASES;` e veja que o banco *javadb* foi gerado.

8. Debata com o instrutor e colegas sobre o resultado apresentado.

7.8 CRIANDO SUBREDES COM O DOCKER

Em diversos casos, precisamos respeitar determinado intervalo de IPs para nossas aplicações. Um exemplo disso seria: para evitar conflitos de IP em nossa infraestrutura, os containers só podem utilizar IPs entre 192.10.0.1 e 192.10.0.255. Como podemos resolver esse problema? Utilizando subredes!

Ao criar uma rede com o Docker, podemos definir uma subrede que será utilizada com a flag `--subtnet`.

```
docker network create -d bridge --subnet 192.10.0.0/24 subnet-bridge
```

Desse modo, definimos que utilizaremos uma máscara de 24 bits e reservamos 8 bits para o intervalo dos IPs. Logo, no exemplo anterior nossos containers que utilizarem essa rede, terão um IP entre 192.10.0.1 e 192.10.0.255.


```
"Networks": {
    "subnet-bridge": {
        "IPAMConfig": null,
        "Links": null,
        "Aliases": [
            "0a4b1576d4ba"
        ],
        "NetworkID": "bb356c5aa19868b1e1004146174513b30c4ba50dd2b1c53337b53c9bdfab7dda",
        "EndpointID": "c282fe94c756cad8e7145917548d90648743d85065046d28328ee14ff6d0d91f"
        "Gateway": "192.10.0.1",
        "IPAddress": "192.10.0.2",
        "IPPrefixLen": 24,
        "IPv6Gateway": "",
        "GlobalIPv6Address": "",
        "GlobalIPv6PrefixLen": 0,
        "MacAddress": "02:42:c0:0a:00:02",
        "DriverOpts": null
    }
}
```


Veja que, desse modo, foi atribuído automaticamente um IP dentro do intervalo para o container.

Porém, também podemos definir manualmente um IP dentro do intervalo utilizando a flag `--ip`.

```
docker run --ip 192.10.0.20 alpine
```

Assim definiremos o IP manualmente para o container, sempre respeitando nossa subrede.



7.9 EXERCÍCIO OPCIONAL: CRIANDO UMA SUBREDE

Objetivo

Nesse exercício, utilizaremos os recursos do Docker para criarmos nossa própria rede com uma subrede definida.

Passo a passo

1. Primeiramente, utilizaremos o comando `docker network create` para criação da rede. A diferença é que agora usaremos uma flag adicional chamada `--subnet`.

   ```
   docker network create -d bridge --subnet 192.168.0.0/24 subnet-bridge
   ```

2. Agora, crie um container utilizando a rede recém-criada.

   ```
   docker run --network subnet-bridge -d --name subrede-container alpine sleep 1d
   ```

3. Inspecione o container recém-criado com o comando `docker container inspect`.

   ```
   docker container inspect subrede-container
   ```

   Veja que o container possui um IP que respeita a faixa da subrede criada.

4. Também podemos definir o IP no momento da criação do container. Para isso, basta utilizar a flag `--ip`.

   ```
   docker run --network subnet-bridge --ip 192.168.0.100 -d --name subrede-container-ip alpine sleep 
   d
   ```

5. Inspecione o container recém-criado com o comando `docker container inspect`.

   ```
   docker container inspect subrede-container-ip
   ```

   Veja que o IP definido no comando está sendo respeitado na saída obtida.

6. Por fim, remova todos os containers com o comando `docker container rm $(docker container ls -qa) --force` e a rede recém-criada com o comando `docker network rm subnet-bridge`.

CAPÍTULO 8

UTILIZANDO LOGS E DEBUG

8.1 INTRODUÇÃO E PROBLEMAS

Nesta aula, veremos como o Docker nos ajuda com a questão de logs gerados pelos nossos containers. Além disso, para fins de depuração, veremos como podemos executar o Docker em modo debug.

Consequentemente, conheceremos mais detalhes internos do Docker, como seus drivers de logging, como alterá-los e como o Docker lida com isso.

Podemos exemplificar uma situação com a nossa imagem *appjava:2.0*. Quando executamos algum container com essa imagem, criamos um processo que executará nossa API. Porém, como podemos acompanhar o status atual da execução de nossa aplicação? Como podemos acompanhar as últimas requisições recebidas, possíveis erros que tenham acontecido, e assim por diante?

8.2 EXIBINDO E SALVANDO LOGS

Assim como tudo que vimos até agora, o Docker também possui comandos para manipulação de logs, sendo o comando base o `docker logs`. Com este comando, podemos obter tudo que está sendo escrito como saída padrão de nosso container, ou seja, o **stdout**.

```
danielartine@daniel:~$ docker logs sad_brown

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.2.2.RELEASE)

2020-02-12 19:52:24.999  INFO 7 --- [           main] br.com.caelum.docker.DockerApplication   : Starting DockerApplication v0.0.1-SNAPSHOT on 401e579
775f4 with PID 7 (/app/springapp.jar started by root in /app)
2020-02-12 19:52:25.015  INFO 7 --- [           main] br.com.caelum.docker.DockerApplication   : No active profile set, falling back to default profil
es: default
2020-02-12 19:52:27.353  INFO 7 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8080 (http)
2020-02-12 19:52:27.378  INFO 7 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2020-02-12 19:52:27.379  INFO 7 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.29]
2020-02-12 19:52:27.493  INFO 7 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2020-02-12 19:52:27.493  INFO 7 --- [           main] o.s.web.context.ContextLoader            : Root WebApplicationContext: initialization completed
in 2271 ms
2020-02-12 19:52:27.873  INFO 7 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicationTaskExecutor
'
2020-02-12 19:52:28.056  INFO 7 --- [           main] o.s.b.a.w.s.WelcomePageHandlerMapping    : Adding welcome page template: index
2020-02-12 19:52:28.373  INFO 7 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http) with context p
ath ''
2020-02-12 19:52:28.376  INFO 7 --- [           main] br.com.caelum.docker.DockerApplication   : Started DockerApplication in 4.534 seconds (JVM runni
ng for 5.419)
```

No exemplo anterior, exibimos o log de um container criado a partir da imagem `appjava:2.0`.

Também podemos ser mais específicos em relação ao log, aplicando outras flags, como por exemplo a `--tail e -f` . Com elas, podemos definir um limite para a "cauda" de nosso log e acompanhar o log em tempo real, respectivamente.

```
danielartine@daniel:~$ docker logs --tail 5 sad_brown
2020-02-12 19:52:27.493  INFO 7 --- [           main] o.s.web.context.ContextLoader            : Root WebApplicationContext: initialization completed
in 2271 ms
2020-02-12 19:52:27.873  INFO 7 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicationTaskExecutor
'
2020-02-12 19:52:28.056  INFO 7 --- [           main] o.s.b.a.w.s.WelcomePageHandlerMapping    : Adding welcome page template: index
2020-02-12 19:52:28.373  INFO 7 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http) with context p
ath ''
2020-02-12 19:52:28.376  INFO 7 --- [           main] br.com.caelum.docker.DockerApplication   : Started DockerApplication in 4.534 seconds (JVM runni
ng for 5.419)
```

Figura 8.2: Exibição das últimas 5 linhas com o tail definido

```
danielartine@daniel:~$ docker logs -f sad_brown

  .   ____          _            __ _ _
 /\\ / ___'_ __ _ _(_)_ __  __ _ \ \ \ \
( ( )\___ | '_ | '_| | '_ \/ _` | \ \ \ \
 \\/  ___)| |_)| | | | | || (_| |  ) ) ) )
  '  |____| .__|_| |_|_| |_\__, | / / / /
 =========|_|==============|___/=/_/_/_/
 :: Spring Boot ::        (v2.2.2.RELEASE)

2020-02-12 19:52:24.999  INFO 7 --- [           main] br.com.caelum.docker.DockerApplication   : Starting DockerApplication v0.0.1-SNAPSHOT on 401e579
775f4 with PID 7 (/app/springapp.jar started by root in /app)
2020-02-12 19:52:25.015  INFO 7 --- [           main] br.com.caelum.docker.DockerApplication   : No active profile set, falling back to default profil
es: default
2020-02-12 19:52:27.353  INFO 7 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat initialized with port(s): 8080 (http)
2020-02-12 19:52:27.378  INFO 7 --- [           main] o.apache.catalina.core.StandardService   : Starting service [Tomcat]
2020-02-12 19:52:27.379  INFO 7 --- [           main] org.apache.catalina.core.StandardEngine  : Starting Servlet engine: [Apache Tomcat/9.0.29]
2020-02-12 19:52:27.493  INFO 7 --- [           main] o.a.c.c.C.[Tomcat].[localhost].[/]       : Initializing Spring embedded WebApplicationContext
2020-02-12 19:52:27.493  INFO 7 --- [           main] o.s.web.context.ContextLoader            : Root WebApplicationContext: initialization completed
in 2271 ms
2020-02-12 19:52:27.873  INFO 7 --- [           main] o.s.s.concurrent.ThreadPoolTaskExecutor  : Initializing ExecutorService 'applicationTaskExecutor
'
2020-02-12 19:52:28.056  INFO 7 --- [           main] o.s.b.a.w.s.WelcomePageHandlerMapping    : Adding welcome page template: index
2020-02-12 19:52:28.373  INFO 7 --- [           main] o.s.b.w.embedded.tomcat.TomcatWebServer  : Tomcat started on port(s): 8080 (http) with context p
ath ''
2020-02-12 19:52:28.376  INFO 7 --- [           main] br.com.caelum.docker.DockerApplication   : Started DockerApplication in 4.534 seconds (JVM runni
ng for 5.419)
```

Figura 8.3: Log sendo acompanhado em tempo real através da flag -f

8.3 CONHECENDO OS DIFERENTES DRIVERS DE LOG

Porém, como todos esses logs conseguem ser gerados e gerenciados pelo Docker? Nativamente, o Docker possui diferentes drivers que podem ser utilizados para manipulação de logs, sendo seu padrão o driver **json-file**. Dentre outros drivers, possuímos a seguinte lista:

- none - Desse modo, não utilizaremos quaisquer drivers para log.
- local - Com esse driver, os logs serão armazenados em um formato customizado para redução de overhead.
- json-file - O driver padrão. Os logs gerados por ele são formatados como JSON.
- syslog - Com esse driver, podemos usar o padrão syslog. O daemon syslog deve estar em execução no sistema host.
- journald - De modo análogo ao anterior, escreverá os logs utilizando journald do sistema. O daemon journald deve estar em execução no sistema host.
- fluentd - Seguindo a mesma ideia, permite a escrita para fluentd. O daemon fluentd deve estar em execução no sistema host.

- gelf - Com esse driver, podemos escrever logs para um endpoint GELF, como por exemplo Graylog ou Logstash.
- awslogs - Escreve logs utilizando o Amazon CloudWatch Logs.
- gcplogs - Com o mesmo propósito do anterior, escreve logs para o Google Cloud Platform.
- splunk - Escreve logs para o splunk, utilizando o coletor de eventos HTTP.
- etwlogs - Esse driver está disponível apenas em Windows. Ele escreve logs como eventos ETW.
- logentries - Escreve logs para o Rapid7 Logentries.

8.4 EXERCÍCIO: UTILIZANDO COMANDOS DE LOGS

Objetivo

Nesse exercício, veremos como exibir e salvar logs gerados por containers em execução, colocando em prática o conteúdo previamente visto.

Passo a passo

1. Execute o comando `docker system info` e encontre o **Logging Driver** utilizado pelo Docker. Como você poderia formatar a saída para exibir apenas o Logging Driver?

2. Execute um container utilizando a imagem `appjava:2.0` e fazendo o mapeamento de portas.

   ```
   docker run -p 9000:8080 --name log-container -d appjava:2.0
   ```

3. Execute o comando `docker logs log-container` e veja que conseguimos exibir a saída atual do container.

4. Também podemos restringir o número de linhas de log exibidas através da flag `--tail`.

   ```
   docker logs --tail=3 log-container
   ```

   Com isso, exibiremos as 3 linhas mais recentes do log apenas.

5. Para salvarmos o log em um arquivo externo, podemos usar recursos do próprio linux.

   ```
   docker logs log-container > output.log
   ```

   Com isso, criaremos um arquivo chamado **output.log** em nossa máquina host com todo o log dentro. Veja que ainda assim podemos usar a flag `--tail`.

6. Podemos acompanhar o log também em tempo real.

   ```
   docker logs -f log-container
   ```

   Desse modo, quaisquer novos dados no log, serão exibidos em tempo real.

7. Abra um novo terminal e execute o comando `docker exec -it log-container bash` .

   Agora, encerraremos o processo do java dentro do container e veremos como isso refletirá no log em exibição. Para encerrarmos o processo do java, basta dentro do terminal do container executarmos o comando `pkill java` .

8. Repare que o log acompanhou a saída do container.

8.5 PARA SABER MAIS: ALTERANDO O DRIVER DE LOGGING

Como dito anteriormente, possuímos diversos drivers de logging. Porém, como podemos fazer para especificar o driver que queremos utilizar? Assim como os drivers de armazenamento, também podemos definir o driver de logging através do arquivo `/etc/docker/daemon.json` . Por exemplo, para configurarmos o **journald** como driver padrão, bastaria adicionar o seguinte trecho ao arquivo **daemon.json**:

```
{
  "log-driver": "journald"
}
```

Após essa alteração, será necessário reiniciar o Docker através do comando `sudo systemctl restart docker` .

Também é possível definir o driver que será utilizado para um container em específico utilizando a flag `--log-driver` :

```
docker run -it --log-driver none appjava:2.0
```

Qual das duas práticas seria a melhor? Debata com os colegas e instrutor.

8.6 UTILIZANDO O DOCKER EM MODO DEBUG

Vimos que os logs nos informam a saída dos containers em execução. Porém, será que podemos ir mais além? Na verdade, sim. O Docker também permite a execução de **todo o daemon** em modo debug, ou seja, podemos saber em tempo real as informações que o daemon recebe, processa e executa os comandos recebidos. Para alcançarmos esse resultado, precisamos colocar o daemon em modo **debug**.

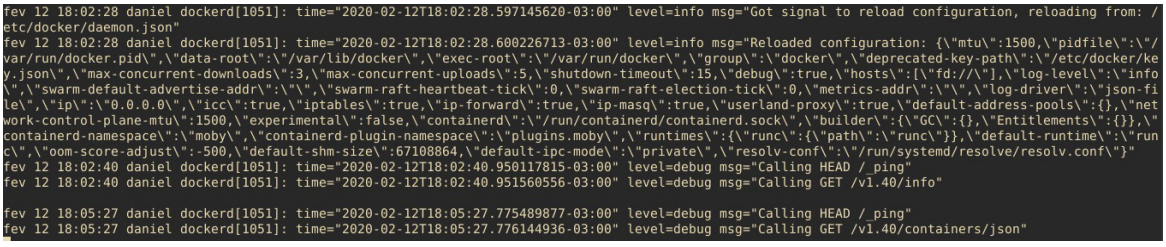
Isso é bem simples e prático. Mais uma vez, basta adicionar uma configuração dentro do nosso arquivo `/etc/docker/daemon.json` :

```
{
  "debug": true
}
```

Nesse caso, podemos fazer o Docker recarregar suas configurações sem reiniciar o serviço com o comando `sudo kill -SIGHUP $(pidof dockerd)` . Para confirmar se realmente estamos em modo debug, executamos o comando `docker system info --format="{{.Debug}}"` .

```
danielartine@daniel:~$ docker system info --format="{{.Debug}}"
true
```

Por fim, podemos acessar e acompanhar as informações exibidas pelo daemon com o comando `journalctl -u docker.service -f` . Veja que todas as informações que dizem respeito ao debug possuem um trecho com o texto **level=debug**. Ou seja, essas informações apenas estão sendo exibidas pois colocamos o Docker em modo debug.



```
fev 12 18:02:28 daniel dockerd[1051]: time="2020-02-12T18:02:28.597145620-03:00" level=info msg="Got signal to reload configuration, reloading from: /etc/docker/daemon.json"
fev 12 18:02:28 daniel dockerd[1051]: time="2020-02-12T18:02:28.600226713-03:00" level=info msg="Reloaded configuration: {\"mtu\":1500,\"pidfile\":\"/var/run/docker.pid\",\"data-root\":\"/var/lib/docker\",\"exec-root\":\"/var/run/docker\",\"group\":\"docker\",\"deprecated-key-path\":\"/etc/docker/key.json\",\"max-concurrent-downloads\":3,\"max-concurrent-uploads\":5,\"shutdown-timeout\":15,\"debug\":true,\"hosts\":[\"fd://\"],\"log-level\":\"info\",\"swarm-default-advertise-addr\":\"\",\"swarm-raft-heartbeat-tick\":0,\"swarm-raft-election-tick\":0,\"metrics-addr\":\"\",\"log-driver\":\"json-file\",\"ip\":\"0.0.0.0\",\"icc\":true,\"iptables\":true,\"ip-forward\":true,\"ip-masq\":true,\"userland-proxy\":true,\"default-address-pools\":{},\"network-control-plane-mtu\":1500,\"experimental\":false,\"containerd\":\"/run/containerd/containerd.sock\",\"builder\":{\"GC\":{},\"Entitlements\":{}},\"containerd-namespace\":\"moby\",\"containerd-plugin-namespace\":\"plugins.moby\",\"runtimes\":{\"runc\":{\"path\":\"runc\"}},\"default-runtime\":\"runc\",\"oom-score-adjust\":-500,\"default-shm-size\":67108864,\"default-ipc-mode\":\"private\",\"resolv-conf\":\"/run/systemd/resolve/resolv.conf\"}"
fev 12 18:02:40 daniel dockerd[1051]: time="2020-02-12T18:02:40.950117815-03:00" level=debug msg="Calling HEAD /_ping"
fev 12 18:02:40 daniel dockerd[1051]: time="2020-02-12T18:02:40.951560556-03:00" level=debug msg="Calling GET /v1.40/info"

fev 12 18:05:27 daniel dockerd[1051]: time="2020-02-12T18:05:27.775489877-03:00" level=debug msg="Calling HEAD /_ping"
fev 12 18:05:27 daniel dockerd[1051]: time="2020-02-12T18:05:27.776144936-03:00" level=debug msg="Calling GET /v1.40/containers/json"
```

Ao executar um container, podemos ver ver todo o processo que foi arquitetado pelo Docker por baixo dos panos, por exemplo.



```
fev 12 18:07:00 daniel dockerd[1051]: time="2020-02-12T18:07:00.448625578-03:00" level=debug msg="Calling HEAD /_ping"
fev 12 18:07:00 daniel dockerd[1051]: time="2020-02-12T18:07:00.449986425-03:00" level=debug msg="Calling POST /v1.40/containers/create"
fev 12 18:07:00 daniel dockerd[1051]: time="2020-02-12T18:07:00.450281363-03:00" level=debug msg="form data: {\"AttachStderr\":true,\"AttachStdin\":true,\"AttachStdout\":true,\"Cmd\":null,\"Domainname\":\"\",\"Entrypoint\":null,\"Env\":[],\"HostConfig\":{\"AutoRemove\":false,\"Binds\":null,\"BlkioDeviceReadBps\":null,\"BlkioDeviceReadIOps\":null,\"BlkioDeviceWriteBps\":null,\"BlkioDeviceWriteIOps\":null,\"BlkioWeight\":0,\"BlkioWeightDevice\":[],\"CapAdd\":null,\"CapDrop\":null,\"Capabilities\":null,\"Cgroup\":\"\",\"CgroupParent\":\"\",\"ConsoleSize\":[0,0],\"ContainerIDFile\":\"\",\"CpuCount\":0,\"CpuPercent\":0,\"CpuPeriod\":0,\"CpuQuota\":0,\"CpuRealtimePeriod\":0,\"CpuRealtimeRuntime\":0,\"CpuShares\":0,\"CpusetCpus\":\"\",\"CpusetMems\":\"\",\"DeviceCgroupRules\":null,\"DeviceRequests\":null,\"Devices\":[],\"Dns\":[],\"DnsOptions\":[],\"DnsSearch\":[],\"ExtraHosts\":null,\"GroupAdd\":null,\"IOMaximumBandwidth\":0,\"IOMaximumIOps\":0,\"IpcMode\":\"\",\"Isolation\":\"\",\"KernelMemory\":0,\"KernelMemoryTCP\":0,\"Links\":null,\"LogConfig\":{\"Config\":{},\"Type\":\"\"},\"MaskedPaths\":null,\"Memory\":0,\"MemoryReservation\":0,\"MemorySwap\":0,\"MemorySwappiness\":-1,\"NanoCpus\":0,\"NetworkMode\":\"default\",\"OomKillDisable\":false,\"OomScoreAdj\":0,\"PidMode\":\"\",\"PidsLimit\":0,\"PortBindings\":{},\"Privileged\":false,\"PublishAllPorts\":false,\"ReadonlyPaths\":null,\"ReadonlyRootfs\":false,\"RestartPolicy\":{\"MaximumRetryCount\":0,\"Name\":\"no\"},\"SecurityOpt\":null,\"ShmSize\":0,\"UTSMode\":\"\",\"Ulimits\":null,\"UsernsMode\":\"\",\"VolumeDriver\":\"\",\"VolumesFrom\":null},\"Hostname\":\"\",\"Image\":\"ubuntu\",\"Labels\":{},\"NetworkingConfig\":{\"EndpointsConfig\":{}},\"OnBuild\":null,\"OpenStdin\":true,\"StdinOnce\":true,\"Tty\":true,\"User\":\"\",\"Volumes\":{},\"WorkingDir\":\"\"}"
fev 12 18:07:00 daniel dockerd[1051]: time="2020-02-12T18:07:00.481314632-03:00" level=debug msg="container mounted via layerStore: &{/var/lib/docker/overlay2/5b77c18e4d7cacb17adc80f0e62f598d090f44f0814e16cf46a9232ed461c9ad/merged 0x563c2de60b40 0x563c2de60b40}"
fev 12 18:07:00 daniel dockerd[1051]: time="2020-02-12T18:07:00.500544901-03:00" level=debug msg="Calling POST /v1.40/containers/354fdb1214b69b8d4c57e10aa7ce0f6f4051370468141bfeb7832cc2d980428d/attach?stderr=1&stdin=1&stdout=1&stream=1"
fev 12 18:07:00 daniel dockerd[1051]: time="2020-02-12T18:07:00.500686617-03:00" level=debug msg="attach: stdin: begin"
fev 12 18:07:00 daniel dockerd[1051]: time="2020-02-12T18:07:00.500718526-03:00" level=debug msg="attach: stdout: begin"
fev 12 18:07:00 daniel dockerd[1051]: time="2020-02-12T18:07:00.500740552-03:00" level=debug msg="attach: stderr: begin"
fev 12 18:07:00 daniel dockerd[1051]: time="2020-02-12T18:07:00.501634697-03:00" level=debug msg="Calling POST /v1.40/containers/354fdb1214b69b8d4c57e10aa7ce0f6f4051370468141bfeb7832cc2d980428d/wait?condition=next-exit"
fev 12 18:07:00 daniel dockerd[1051]: time="2020-02-12T18:07:00.502759548-03:00" level=debug msg="Calling POST /v1.40/containers/354fdb1214b69b8d4c57e10aa7ce0f6f4051370468141bfeb7832cc2d980428d/start"
```

O uso do modo debug é extremamente útil. Principalmente quando nosso objetivo é depurar e acompanhar passo a passo a execução de um ou mais containers.

8.7 EXERCÍCIO EM CASA: ATIVANDO O MODO DEBUG

Objetivo

Nesse exercício, o foco será em colocar o Docker em modo debug e observar alguns resultados.

Passo a passo

1. Abra o arquivo `daemon.json` localizado em `/etc/docker`. Caso não exista o arquivo dentro do diretório, basta criá-lo. Nele, adicione o seguinte trecho:

   ```
   {
     "debug": true
   }
   ```

   Salve e feche o arquivo.

2. Será necessário recarregar o arquivo de configuração para que o Docker habilite o modo debug. Execute o seguinte comando:

   ```
   sudo kill -SIGHUP $(pidof dockerd)
   ```

3. Abra um novo terminal e execute o comando `journalctl -u docker.service -f`. Com ele, conseguiremos observar cada comando processado pelo daemon.

4. Observe o terminal que está executando o comando recém-executado. Observe que em um intervalo específico de tempo é enviado um "batimento cardíaco" para informar que o Docker "está vivo".

5. Enquanto observa o log de debug, execute em um outro terminal o seguinte comando:

   ```
   docker run -p 9000:8080 --name debug-container -d <seu-usuario-aqui>/appjava:2.0
   ```

   Observe cada ação que o Docker executa para criação do seu container.

6. Tente executar um container a partir de uma imagem inexistente, como por exemplo `docker run essa-imagem-jamais-existira`.

   O que aconteceu?

7. Feche todos os terminais e altera seu arquivo `daemon.json` para o seguinte:

   ```
   {
     "debug": false
   }
   ```

8. Mais uma vez, execute o seguinte comando:

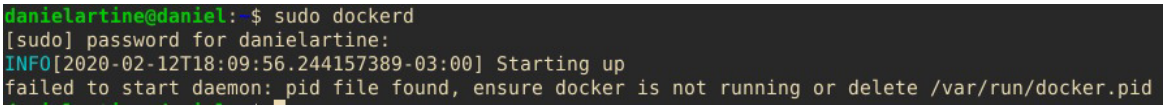
   ```
   sudo kill -SIGHUP $(pidof dockerd)
   ```

   Com isso, seu daemon voltará para o modo normal.

8.8 PARA SABER MAIS: USANDO O DAEMON EM MODO MANUAL

Uma outra maneira de utilizar o Docker, é com o daemon executado manualmente. Isso quer dizer que executaremos o daemon diretamente através do terminal com o comando `sudo dockerd` . Com isso, teremos a mesma saída que o comando `journalctl -u docker.service -f` , executado previamente, sendo a diferença que agora esse terminal não será apenas responsável pela exibição de logs, mas também pela execução do Docker.

Pode ser que o comando `sudo dockerd` exiba a seguinte saída:



Isso acontece pois já há uma instância do Docker em execução na máquina. Então devemos pará-la com o comando `sudo systemctl stop docker` e aí sim poderemos executar o comando `sudo dockerd` sem problemas.

Capítulo 9

COORDENANDO CONTAINERS COM DOCKER COMPOSE

Introdução e problema

Nesta aula, estudaremos uma tecnologia chamada **Docker Compose**, que nos auxiliará a lidar com múltiplos *containers* simultaneamente, mas qual é o problema?

Nas aula anteriores, criamos redes e subimos containers, um com o banco de dados e outro como uma aplicação que usa o banco. Para tal, é necessário:

1. Criar uma rede
2. Criar um volume
3. Rodar o container com MySQL
4. Rodar o container com a aplicação

Basicamente, executamos os comandos:

```
docker network create --driver bridge network-db
docker volume create volume-db

docker run \
    --network network-db \
    --detach \
    --rm \
    --name container-db \
    --mount source=volume-db,target=/var/lib/mysql \
    --publish 3000:3306 \
    --env MYSQL_ALLOW_EMPTY_PASSWORD=yes \
    mysql:5.7

docker run \
    --network network-db \
    --detach \
    --rm \
    --name container-app \
    --publish 9000:8080 \
    appjava:2.0
```

Esses comandos criam dois *containers* junto ao volume e a rede, porém, veja que seguimos uma abordagem muito manual, podendo ficar mais uma vez muito dispendioso. É muito comum esquecermos de passar alguma *flag*, ou subir o *container* na ordem errada, sem a devida rede. Ou seja, além de manual, é facilmente suscetível a erros, isso com somente *dois containers*.

Esse modo de subir os *containers* na mão é bom se quisermos criar um ambiente rapidamente, ou quando são poucos *containers*, mas a medida que a aplicação começa a crescer, temos que digitar muitos comandos.

No mundo real, sabemos que aplicações podem precisar de muita mais do que dois containers, além de terem que se comunicar entre si. Dessa forma, nosso trabalho se divide tanto na criação como na comunicação dos*container*.

9.1 CONHECENDO O DOCKER COMPOSE

Ao invés de subir todos esses *containers* na mão, o que vamos fazer é utilizar uma tecnologia aliada do Docker, chamada **Docker Compose**, feito para nos auxiliar a subir e configurar múltiplos *containers*. Ou seja, o Docker compose servirá como uma ferramenta de **coordenação**, não confundir com **orquestração**.

Ele funciona seguindo um arquivo de texto no formato **YAML** (extensão **.yml**), e nele nós descrevemos tudo o que queremos que aconteça para subir a nossa aplicação, todo o nosso processo de *build*, isto é, subir o banco, os *containers* das aplicações, etc.

Assim, não precisamos ficar executando muitos comandos no terminal sem necessidade. Esse será o foco desta aula, montar uma aplicação na estrutura descrita anteriormente na imagem, que é uma situação comum no nosso dia-a-dia.

Instalação do Docker Compose no Linux

O **Docker Compose** não é instalado por padrão no Linux e não vem instalado junto com o Docker, logo, você deve instalá-lo por fora. Para tal, baixe-o na sua versão mais atual, que pode ser visualizada no seu GitHub, executando o comando abaixo:

```
sudo curl -L "https://github.com/docker/compose/releases/download/1.25.1/docker-compose-$(uname -s)-$(uname -m)" -o /usr/local/bin/docker-compose
```

Após isso, dê permissão de execução para o **docker-compose**:

```
sudo chmod +x /usr/local/bin/docker-compose
```

Pronto, o **Docker Compose** já está instalado no seu Linux!

Utilizando o Docker Compose

Para utilizar o **Docker Compose**, devemos criar o seu arquivo de configuração, o **docker-compose.yml**, na raiz do projeto. Todo arquivo **Docker Compose** é uma espécie de receita de bolo para construirmos as diferentes partes da nossa aplicação, a primeira coisa que colocamos nele é a versão do Docker Compose que estamos utilizando:

```
version: '3.7'
```

Estamos utilizando a versão 3.7 pois é a versão mais recente no momento da criação da apostila. O YAML lembra um pouco o JSON, mas ao invés de utilizar as chaves para segmentar a estrutura do código, ele utiliza **espaços**.

Agora, começamos a descrever os nossos *services*, *volumes* e *networks*:

```
version: '3.7'

services:
  #definição dos containers

networks:
  #definição das redes

volumes:
  #definição dos volumes
```

Um serviço é uma parte da nossa aplicação, ou seja, o mysql como banco de dados e a aplicação no outro container. Logo, se queremos construir dois *containers*, cada um deles com um nome específico.

Então, vamos começar construindo dois **containers**, o `mysql` e o `appjava` :

```
version: '3'
services:
    mysql:
      #definicao do container com mysql
    appjava:
      #definicao do container da aplicacao
```

Em cada serviço, devemos dizer como devemos construí-lo, isto é, dizer qual é a imagem, nome da imagem, rede, volumes etc. Quando o Docker Compose for criar o *container* a partir da imagem `mysql:5.7` , seu nome será **container-mysql**:

```
version: '3.7'

services:
  mysql:
    image: mysql:5.7
    container_name: container-mysql
```

Sabemos também que o MySQL trabalha com a porta **3306**. Vamos mapear para a porta `3000` da máquina *host*, além de configurar a variável de ambiente para permitir que mysql aceita uma sehna vazia:

```
version: '3.7'

services:
  mysql:
    image: mysql:5.7
    container_name: container-mysql
    ports:
        - "3000:3306"
    environment:
      - MYSQL_ALLOW_EMPTY_PASSWORD=yes
```

No YAML, toda vez que colocamos um traço, significa que a propriedade pode receber mais de um item. Assim poderíamos configurar mais portas, se for necessário. Agora, falta ainda especificar o volume e a rede que o container vai fazer parte:

```
version: '3.7'

services:
  mysql:
    image: mysql:5.7
    container_name: container-mysql
    ports:
        - "3000:3306"
    environment:
      - MYSQL_ALLOW_EMPTY_PASSWORD=yes
    volumes:
      - volume-db:/var/lib/mysql
    networks:
        - network-db
```

Repare que definimos um volume `volume-db` e uma rede `network-db` . Qualquer volume e rede precisa estar declarada separadamente dentro da sua seção como falamos no início. Assim já temos um arquivo `docker-compose.yml` funcional:

```
version: '3.7'

services:
  mysql:
    image: mysql:5.7
    container_name: container-mysql
    ports:
        - "3000:3306"
    environment:
      - MYSQL_ALLOW_EMPTY_PASSWORD=yes
    volumes:
      - volume-db:/var/lib/mysql
    networks:
        - network-db

networks:
  network-db:
    driver: bridge

volumes:
  volume-db:
```

Repare que criamos a rede `network-db` e ela utilizará o *driver* **bridge**.

9.2 EXECUTANDO COM DOCKER-COMPOSE

Com o arquivo `docker-compose.yml` pronto, mesmo com um container apenas, podemos subir os serviços (*volume, rede container*). Para isso, dentro da pasta do nosso projeto, executamos o seguinte comando:

```
docker-compose up
Creating network "do-26_network-db" with driver "bridge"
Creating container-mysql ... done
Attaching to container-mysql
container-mysql | 2020-01-04T16:00:19.680439Z 0 [Warning] TIMESTAMP with implicit DEFAULT value is de
precated. ..
container-mysql | 2020-01-04T16:00:19.682567Z 0 [Note] mysqld (mysqld 5.7.24) starting as process 1 .
..
container-mysql | 2020-01-04T16:00:19.688222Z 0 [Note] InnoDB: PUNCH HOLE support available
container-mysql | 2020-01-04T16:00:19.688315Z 0 [Note] InnoDB: Mutexes and rw_locks use GCC atomic bu
iltins
container-mysql | 2020-01-04T16:00:19.688334Z 0 [Note] InnoDB: Uses event mutexes
...
```

O comando `docker-compose up` lê o arquivo `docker-compose.yml` e faz com que o container `container-mysql` suba com a rede e o volume configurado. Repare também que a rede foi acoplada ao container.

No entanto, toda a saída do container aparece no console. Para subir o container de maneira `desacoplada` basta adicionar a flag `-d` ao comando:

```
docker-compose up -d
Creating network "do-26_network-db" with driver "bridge"
Creating container-mysql ... done
```

Para mostrar os containers em execução temos o comando `ps`:

```
docker-compose ps
Name              Command                    State                 Ports
-------------------------------------------------------------------------------
container-mysql   docker-entrypoint.sh mysqld Up  0.0.0.0:3000->3306/tcp, 33060/tcp
```

E para parar e remover o container e a rede basta executar o comando:

```
docker-compose down
Stopping container-mysql ... done
Removing container-mysql ... done
Removing network do-26_network-db
```

9.3 SUBINDO VÁRIOS CONTAINERS

O arquivo `docker-compose.yml` já facilitou muito, pois basta chamar `up` e `down` sem se preocupar com os detalhes de rede, volumes e outras flags.

No entanto, definimos um único container, falta ainda a nossa aplicação. A declaração do segundo container não tem novidade, usando a rede já definida e declarando a imagem, nome e portas:

```
version: '3.7'

services:
  appjava:
    image: appjava:2.0
    container_name: container-app
    ports:
        - "9000:8080"
    networks:
        - network-db

  mysql:
    image: mysql:5.7
    container_name: container-mysql
    ports:
        - "3000:3306"
    environment:
      - MYSQL_ALLOW_EMPTY_PASSWORD=yes
    volumes:
      - volume-db:/var/lib/mysql
    networks:
        - network-db

networks:
  network-db:
    driver: bridge

volumes:
  volume-db:
```

Pronto. Para rodar e parar os containers usaremos o mesmos comandos sem nenhuma alteração: `docker-compose up -d` e `docker-compose down`.

9.4 BUILD DA IMAGEM COM DOCKER COMPOSE

Com o `docker-compose.yml` pronto, podemos subir os serviços, mas antes devemos garantir que temos todas as imagens envolvidas realmente foram criadas. Para isso, temos criado o `Dockerfile` que possui as instruções para construir a imagem da aplicação Java. Será que tem um jeito de associar o nosso `docker-compose.yml` com o arquivo `Dockerfile` ? Certamente. Para tal, existe um item `build:` que podemos associar com um container. Veja o exemplo na página seguinte:

```
version: '3.7'

services:

  appjava:
    build:
      dockerfile: Dockerfile
      context: .
    image: appjava:2.0
    container_name: container-app
    ports:
        - "9000:8080"
    networks:
        - network-db

#outras definições omitidas
```

O `Dockerfile` deve estar na mesma pasta do `docker-compose.yml`. Agora, só falta pedir para o compose buildar a imagem.

```
docker-compose build
mysql uses an image, skipping
Building appjava
Step 1/7 : FROM ubuntu:18.04
 ---> 549b9b86cb8d
Step 2/7 : WORKDIR /app
 ---> Using cache
 ---> 127889d23c34
Step 3/7 : ARG JAR_FILE_NAME=springapp.jar
 ---> Using cache
 ---> f202c3e1ac7e
Step 4/7 : COPY target/$JAR_FILE_NAME .
 ---> 332dee7ae7c5
Step 5/7 : RUN apt-get update && apt-get install -y openjdk-8-jre
 ---> Running in b4fce03e94ea
 ....
Step 6/7 : ENV JAR_NAME=$JAR_FILE_NAME
---> Running in 40cdeabbe5cf
Removing intermediate container 40cdeabbe5cf
---> cfc76996257a
Step 7/7 : ENTRYPOINT java -jar $JAR_NAME
---> Running in 34b461c8f4a5
Removing intermediate container 34b461c8f4a5
---> 9913d080795c
Successfully built 9913d080795c
Successfully tagged appjava:2.0
```

O comando build também pode receber a flag `--build-arg` para definir parâmetros do build, igual ao comando `docker build`.

Ao listar as imagens podemos ver que a imagem foi versionada com o nome e a tag do `docker-compose.yml`:

```
docker image ls
REPOSITORY    TAG    IMAGE ID         CREATED             SIZE
```

```
appjava       2.0   9913d080795c     17 seconds ago      439MB
```

9.5 EXERCÍCIO: PRIMEIRO ARQUIVO DE COMPOSIÇÃO

Objetivo

Nesse exercício, criaremos nosso primeiro arquivo de composição e colocaremos em prática alguns dos comandos da ferramenta `docker-compose`.

Passo a passo

1. Crie uma pasta chamada `primeiro-compose` em sua área de trabalho. Dentro dela, crie o arquivo `docker-compose.yml`.

2. Dentro do arquivo, definiremos instruções para criação dos serviços desejados. Porém, antes precisamos definir qual versão utilizaremos na composição do nosso arquivo:

```
version: '3.7'
services:
```

3. Agora, precisamos definir qual será o nome do serviço e ele será criado:

```
version: '3.7'
services:
    ubuntu_service:
        image: ubuntu:18.04
        container_name: container-ubuntu
```

4. Com o arquivo de composição definido, podemos executar a instrução de gerar o container através do arquivo. O comando deve ser executado na mesma pasta do arquivo `.yml`:

```
docker-compose up
```

   Veja que o serviço definido dentro do arquivo foi gerado sem nenhum problema.

5. Podemos ver os containers que foram gerados através do `docker-compose`. Para isso, podemos executar o comando `docker-compose ps`.

6. Para derrubarmos os serviços criados pelo arquivo de composição, podemos executar o comando `docker-compose down`.

9.6 ORDEM DOS SERVIÇOS

Conseguimos subir o banco de dados e a aplicação Java através do `docker` e `docker-compose`. Nesse exemplo a aplicação Java não vai funcionar sem o banco no ar pois ela dependende dela. Ai faz sentido definir essa dependência e o `docker-compose.yml` possiu um elemento chamado de `depends_on` que podemos adicionar na definição do container `appjava`:

```
version: '3.7'

services:
  appjava:
    image: appjava:2.0
    container_name: container-app
    ports:
        - "9000:8080"
    networks:
        - network-db
    depends_on:
    - mysql

  mysql:
    image: mysql:5.7
    container_name: container-mysql
    ports:
        - "3000:3306"
    environment:
      - MYSQL_ALLOW_EMPTY_PASSWORD=yes
    volumes:
      - volume-db:/var/lib/mysql
    networks:
        - network-db

networks:
  network-db:
    driver: bridge

volumes:
  volume-db:
```

No entanto o `depends_on` não é exatamente o que se espera dele. O `depends_on` apenas garante que o container `mysql` será inicializado antes do `appjava` mas NÃO espera até ele está "pronto". Ou seja, pode ser que o container `appjava` suba antes do `mysql` se ele for mais rápido.

O problema é que o docker não sabe o que significa "pronto", quando a aplicação realmente está inicializada ou não. Ou seja, em outras palavras não temos um controle sobre a ordem dos containers através do `docker-compose` .

9.7 EXERCÍCIO: CRIANDO MÚLTIPLOS SERVIÇOS

Objetivo

Nesse exercício, o foco será na criação de um arquivo de composição para criar múltiplos serviços, fazendo a devida comunicação entre eles.

Passo a passo

1. Crie uma pasta chamada `segundo-compose` em sua área de trabalho. Dentro dela, crie o arquivo

`docker-compose.yml`.

2. Nesse novo arquivo, criaremos dois serviços: um usará como base a nossa imagem `appjava:2.0` e o outro a imagem `mysql:5.7`.

```
version: '3.7'

services:
    appjava:

    mysql:
```

Queremos fazer a comunicação entre os dois serviços, assim como no exercício de redes.

3. Agora, precisamos definir como construíremos o serviço `appjava`.

```
version: '3.7'
services:
    appjava:
        image: appjava:2.0
        container_name: container-appjava
        ports:
            - "9000:8080"
        networks:
            - network-db

    mysql:
```

Veja que conseguimos definir configurações de mapeamento de portas e redes do container.

4. A mesma coisa deve ser feita para o serviço `mysql`:

```
version: '3.7'
services:
    appjava:
        image: appjava:2.0
        container_name: container-appjava
        ports:
            - "9000:8080"
        networks:
            - network-db

    mysql:
        image: mysql:5.7
        container_name: container-mysql
        ports:
            - "3000:3306"
        environment:
            - MYSQL_ALLOW_EMPTY_PASSWORD=yes
        volumes:
            - volume-db:/var/lib/mysql
        networks:
```

```
            - network-db
```

5. Nós definimos as redes e volumes para esses serviços. Porém, também precisamos declará-los dentro do arquivo de composição:

```
version: '3.7'
services:
    appjava:
        image: appjava:2.0
        container_name: container-appjava
        ports:
            - "9000:8080"
        networks:
            - network-db

    mysql:
        image: mysql:5.7
        container_name: container-mysql
        ports:
            - "3000:3306"
        environment:
            - MYSQL_ALLOW_EMPTY_PASSWORD=yes
        volumes:
            - volume-db:/var/lib/mysql
        networks:
            - network-db
networks:
    network-db:
        driver: bridge

volumes:
    volume-db:
```

6. Dentro do diretório `segundo-compose` , execute o comando `docker-compose up` e veja que conseguimos atingir o mesmo resultado do exercício que vinculamos os dois containers.

9.8 DESAFIO DOCKER COMPOSE

Acabamos de aprender uma das ferramentas mais importantes do ecossistema do Docker, o Docker compose. Ela será útil em diversas situações onde teremos diversos serviços para serem executados ao mesmo tempo. Para praticar os principais conceitos do Docker compose você será desafiado a escrever um arquivo `docker-compose.yml` que resolva o problema apresentado a seguir.

Considere que temos um sistema de votação através do qual o usuário poderá escolher que tipo de animal doméstico ele prefere, cachorro ou gato. A escolha de um deles gerará um voto, que será registrado por uma aplicação escrita em Python, que denominaremos como `voting-app` . Essa aplicação registrará o voto em um banco de dados chave-valor Redis.

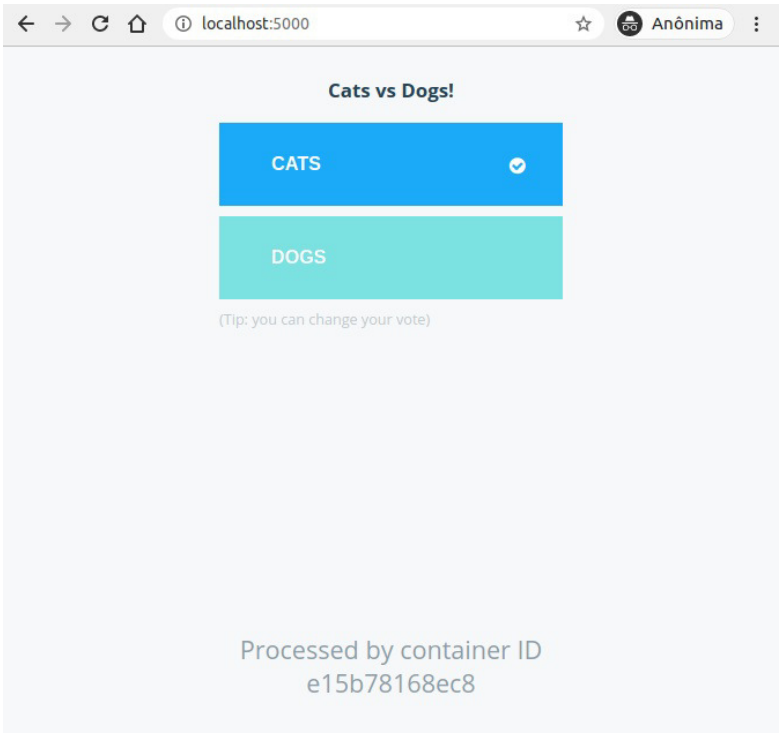


Figura 9.1: Tela de votação

Assim que um novo voto é registrado no Redis uma outra aplicação, escrita em Java e denominada como `worker`, estará monitorando esses novos registros, processará os votos e consolidará em uma base de dados relacional, utilizando o PostgreSQL.

O usuário poderá acompanhar o resultado da votação em tempo real através de uma outra aplicação, a `result-app` escrita em NodeJS e Angular, que se conectará ao banco de dados PostgreSQL para obter esses dados.

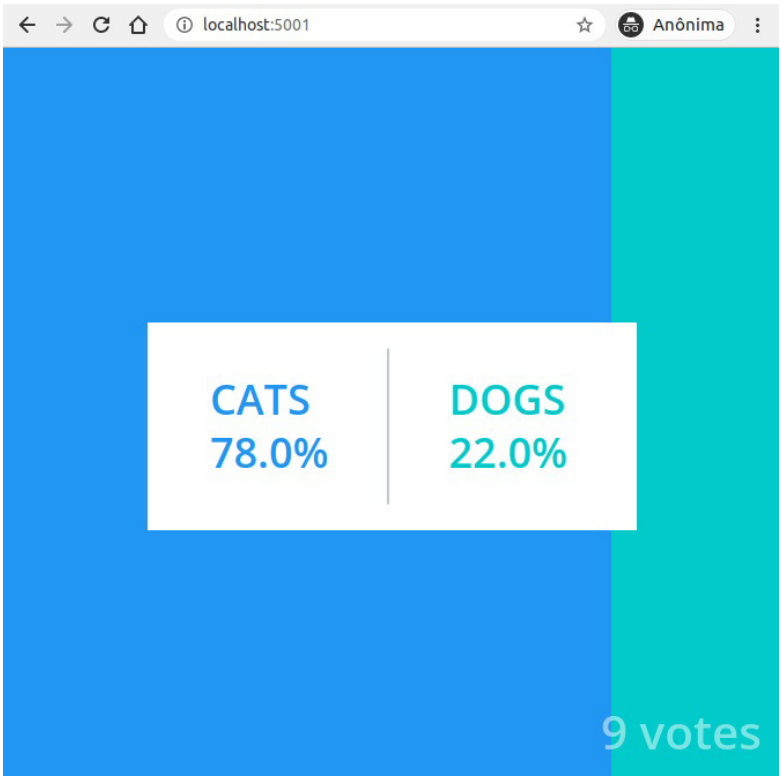


Figura 9.2: Tela de resultados

Cada uma das três aplicações citadas e os dois bancos deverão ser executados em um container próprio. A seguir temos um diagrama da comunicação entre os containers e o usuário final:

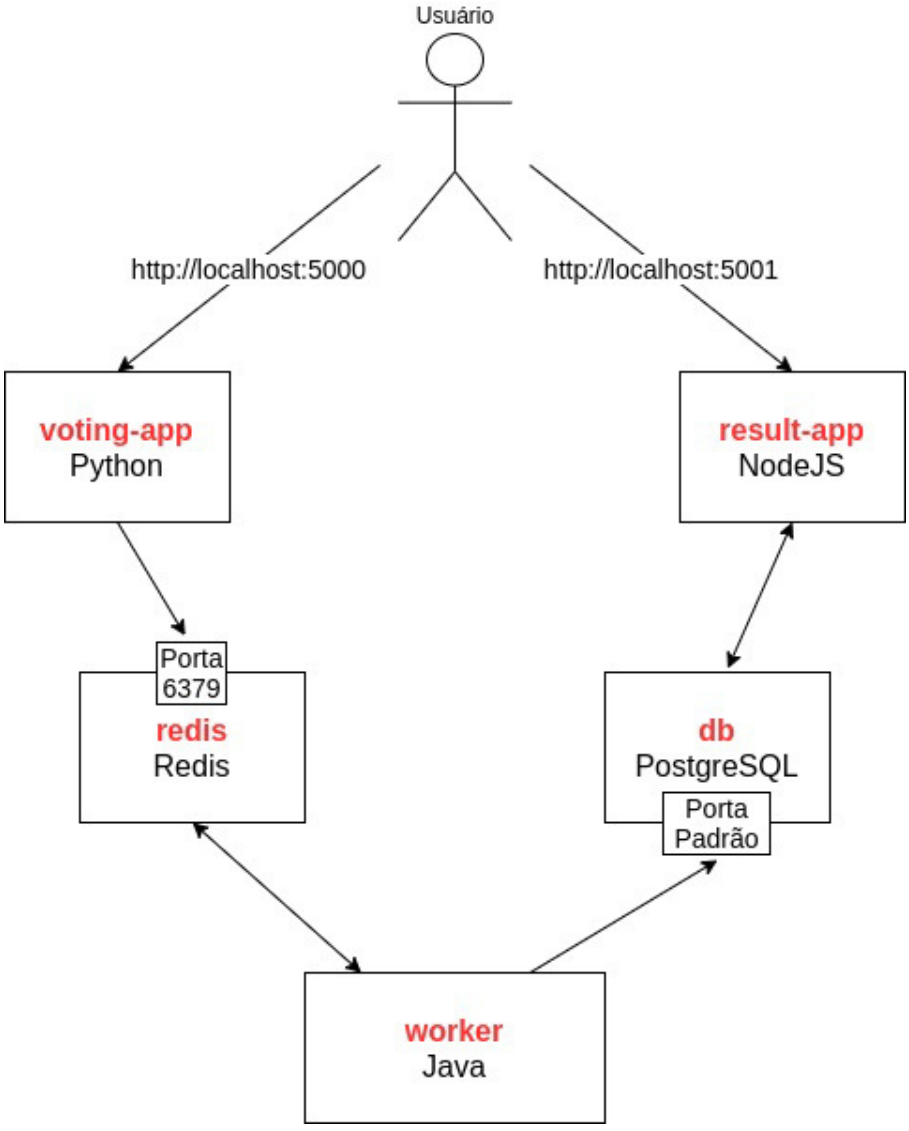


Figura 9.3: Diagrama de comunicação

O sistema de votação e de resultado deverão ser acessados pelos endereços http://localhost:5000 http://localhost:5001 respectivamente.

Queremos evitar que os dados salvos no Postgres não sejam perdidos se o container for apagado. Assim, você precisará montar um volume que mapeará um certo diretório do container para uma pasta do host. A seguir temos um diagrama desse mapeamento:

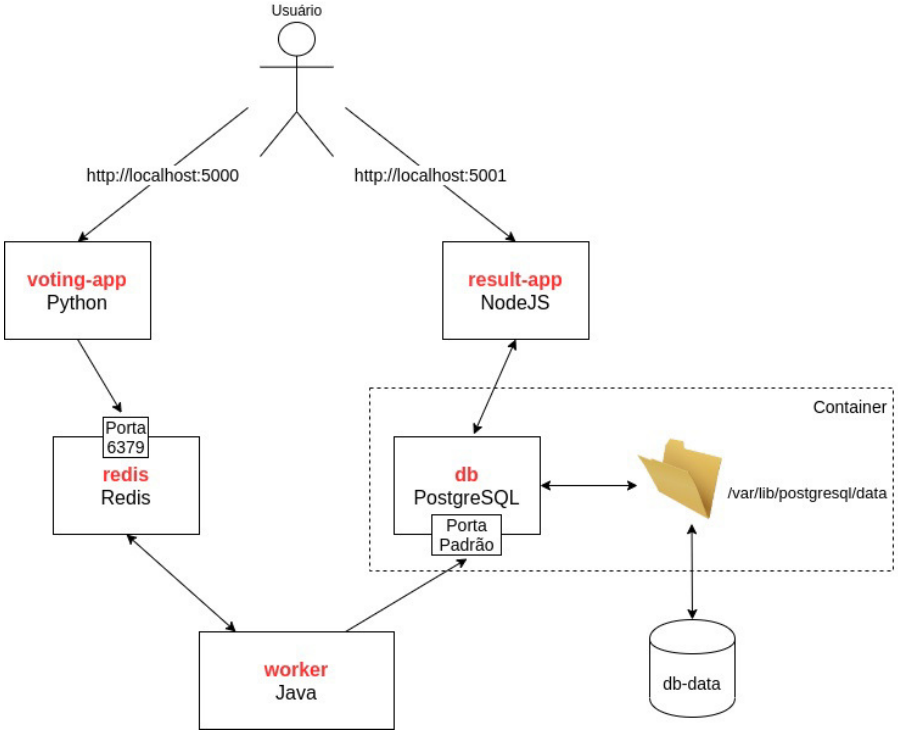


Figura 9.4: Diagrama de volumes

O tipo de montagem pode ser tanto `volume` como `bind`, fica a seu critério decidir isso.

Por fim, queremos que os container conversem entre si através da rede. Teremos duas redes diferentes, a primeira será a **front-tier**, que terá apenas as aplicações `voting-app` e a `result-app`. A segunda rede será a **back-tier**, que incluirá todas as três aplicações e os dois bancos de dados. A seguir temos um diagrama dessas duas redes:

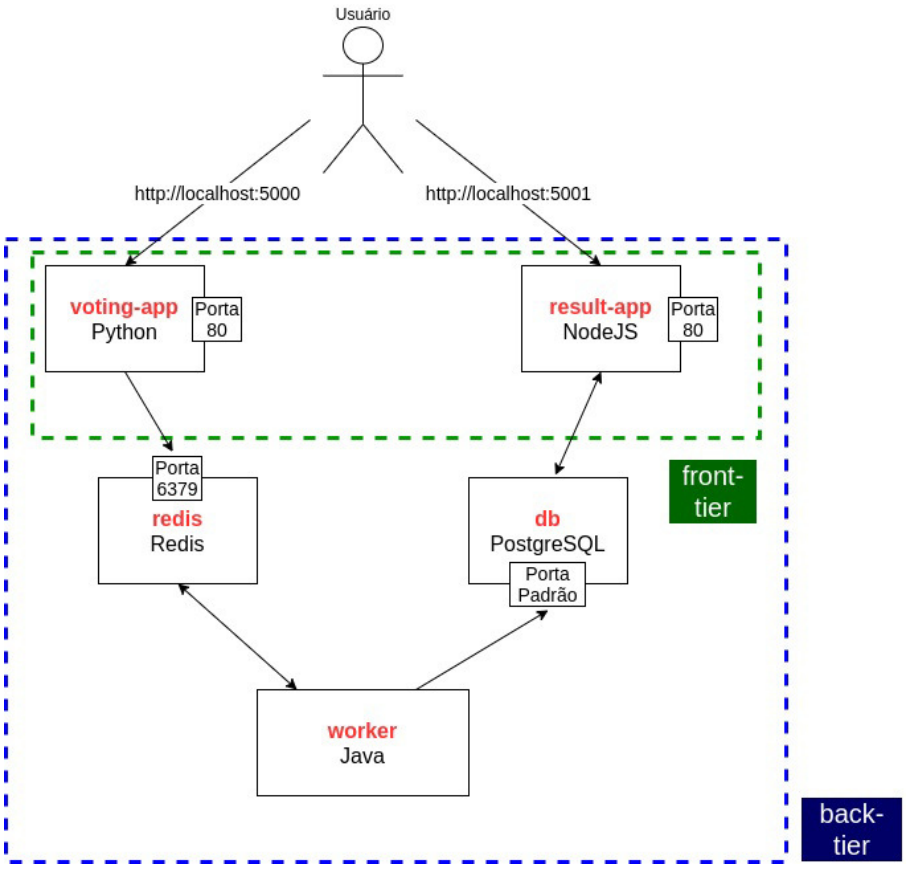


Figura 9.5: Diagrama de redes

Detalhes adicionais

Serviço voting-app

1. Sua pasta será `./vote`
2. Ele tem seu próprio Dockerfile, com o nome padrão
3. Será necessário montar a pasta `./vote` dentro do container, na pasta `/app`
4. O comando de execução da aplicação será `python app.py`
5. Deverá ser acessível através do endereço http://localhost:5000

Serviço worker

1. Sua pasta será `./worker`
2. Ele tem seu próprio Dockerfile, com o nome **Dockerfile.j**

Serviço redis

1. Será utilizado a imagem **alpine do redis**
2. Nenhuma configuração adicional precisa ser feita para que o banco funcione
3. A porta do redis não precisa ser exporta para o Host

Serviço db

1. Será utilizado a imagem **postgres versão 9.4**
2. Será necessário definir o **POSTGRES_USER** e **POSTGRES_PASSWORD**, ambos com o valor postgres

Serviço result-app

1. Sua pasta será `./result`
2. Ele tem o próprio Dockerfile, com o nome padrão
3. Será necessário montar a pasta `./result` dentro do container, na pasta `/app`
4. Deverá ser acessível através do endereço http://localhost:5001

Baixe o código fonte dos projetos através do seguinte link:

https://gitlab.com/aovs/projetos-cursos/do26-projeto

Clique no arquivo compactado `desafio-voting-app.zip` e depois no botão para baixar:

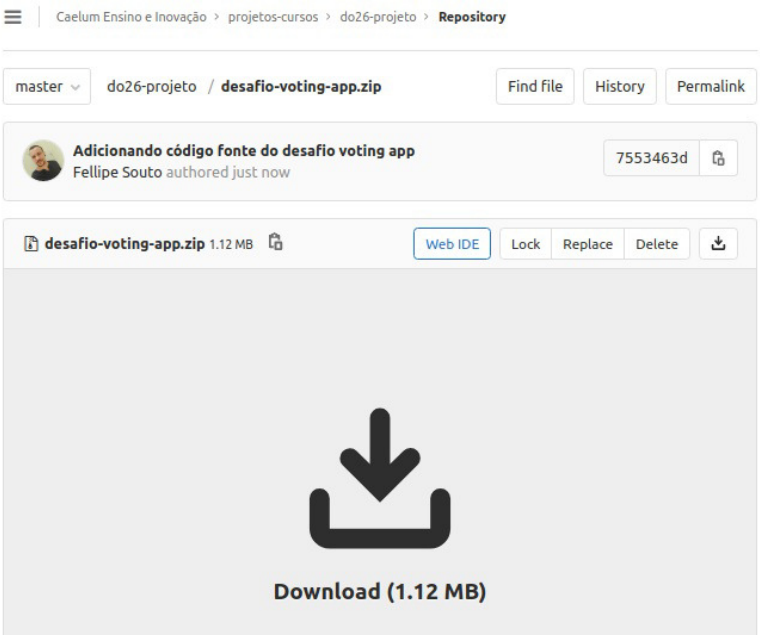


Figura 9.6: services diagram

Descompacte o arquivo, abra a pasta em algum editor de texto e comece a escrever o seu docker-compose. O código fonte das aplicações já está pronto para funcionar, não é necessário fazer nenhuma modificação.

Tente não cair na tentanção de procurar a solução pronta na internet. O objetivo aqui é validar se você conseguiu aprender os conceitos passados durante a aula. Caso tenha alguma dúvidas pergunte ao instrutor, e não deixe de consultar também a documentação oficial do Docker Compose:

https://docs.docker.com/compose/compose-file/

@NOTE

A resposta se encontra na branch desafio-docker-compose no arquivo docker-compose-javaworker.yml, acabei de fazer o teste e tudo sobe conforme esperado.

https://github.com/fllsouto/example-voting-app/blob/desafio-docker-compose/docker-compose-javaworker.yml

Capítulo 10

ISOLANDO AMBIENTES COM A DOCKER MACHINE

10.1 O QUE É DOCKER MACHINE?

Imagine que você precisa rodar o Docker em várias máquinas para, por exemplo, escalar uma aplicação ou aumentar disponibilidade dela. Veremos ainda ferramentas específicas para esses casos, mas nesse momento basta saber que precisamos algumas máquinas com a Docker Engine já instalada.

Podemos seguir as instruções de instalação apresentadas nessa apostila, mas elas são específicas para o sistema operacional, além de ser ser um trabalho manual. Claro que podemos usar provisionadores como Ansible ou Puppet para automatizar a instalação do Docker e seguir a linha da *infraestrutura como código*, mas será que é preciso aprender um outro software para instalar o Docker? Será que não existe uma ferramenta do Docker mesmo para tal? Para resolver esse problema, temos a **Docker Machine**.

A Docker Machine é uma ferramenta que cria máquinas virtuais já provisionadas para rodar o Docker. Você pode usá-la para criar hosts Docker no Linux, Mac ou Windows ou em provedores de nuvem como Azure, AWS ou Digital Ocean.

10.2 CRIANDO AMBIENTES VIRTUAIS PARA EXECUÇÃO

Para acionar o Docker Machine devemos usar o comando `docker-machine` . Com ele você pode iniciar, inspecionar, parar e reiniciar a máquina virtual e os comandos são bem parecidos com aqueles do Docker.

Para criar uma máquina virtual usamos a instrução `create` e devemos definir um `driver` . Nesse contexto o driver define qual hypervisor/nuvem queremos usar, por exemplo *VirtualBox* da Oracle, *Hyper-V* da Microsoft ou algum driver para usar uma nuvem como AWS ou Azure.

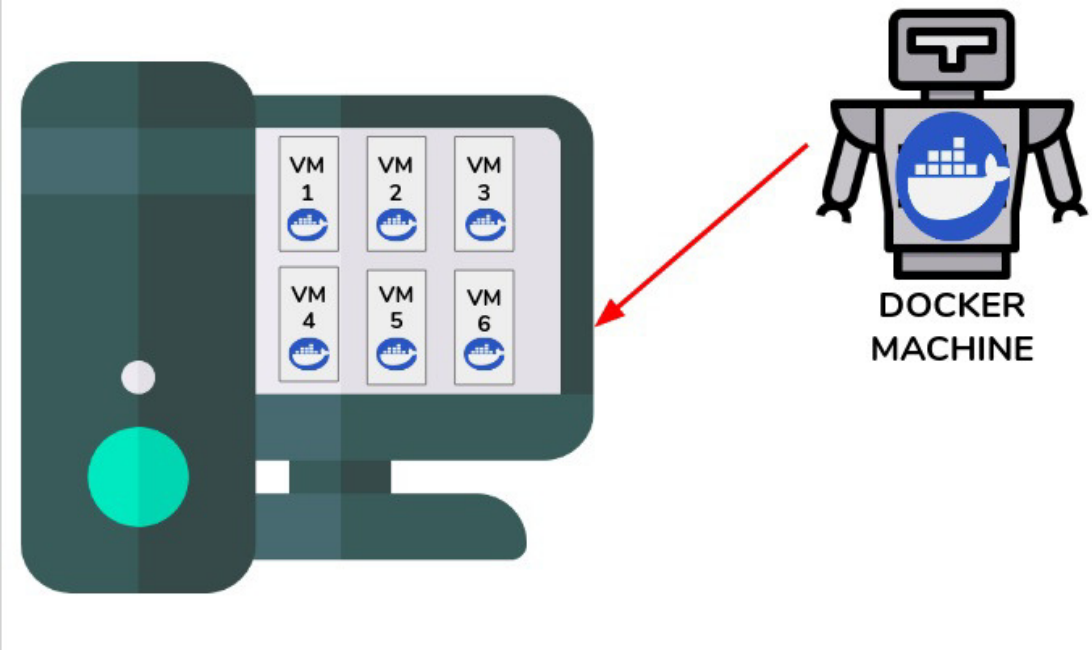
Uma lista de drivers está disponíveis na documentação:

- https://docs.docker.com/v17.12/machine/drivers/

No nosso exemplo vamos criar uma máquina virtual baseada no *VirtualBox*. Isso também significa que é preciso instalar o VirtualBox *antes* de usar a Docker Machine!

Uma vez o VirtualBox instalado, podemos criar primeira máquina através do `docker-machine create` especificando o *driver* e dando um nome, por exemplo:

```
docker-machine create --driver=virtualbox vm1
```



Na primeira execução pode demorar um pouco, pois precisa baixar a ISO (imagem) necessária. Uma vez baixado fica muito mais rápido a execução. Existem várias flags especificas para personalizar a máquina virtual baseada no VirtualBox que estão disponíveis na documentação oficial:

- https://docs.docker.com/v17.12/machine/drivers/virtualbox/#options

10.3 GERENCIANDO E ACESSANDO A MÁQUINA VIRTUAL

Para listar todas as máquinas já criadas podemos usar a instrução `docker-machine ls` :

```
$docker-machine ls
NAME  ACTIVE   DRIVER       STATE     URL                         SWARM   DOCKER
vm1   -        virtualbox   Running   tcp://192.168.99.100:2376           v19.03.5
```

Para parar e inicar existem os comando `start` e `stop` , e com `rm` podemos remover uma ambiente, sempre indicando o nome do ambiente:

```
$docker-machine stop vm1
Stopping "vm1"...
Machine "vm1" was stopped.
```

e

```
$docker-machine start vm1
Starting "vm1"...
(vm1) Check network to re-create if needed...
(vm1) Waiting for an IP...
Machine "vm1" was started.
Waiting for SSH to be available...
Detecting the provisioner...
```

Por fim, para acessar a máquina virtual, usamos a instrução `ssh`:

```
$docker-machine ssh vm1
   ( '>')
  /) TC (\   Core is distributed with ABSOLUTELY NO WARRANTY.
 (/-_--_-\)           www.tinycorelinux.net

docker@vm1:~$ docker -v
```

Repare o novo prompt `docker@vm1` que indica que estamos conectado com a máquina virtual pronta para usar o Docker!

```
docker@vm1:~$ docker -v

Docker version 19.03.5, build 633a0ea838
```

10.4 EXERCÍCIO: CRIANDO AMBIENTES COM A DOCKER MACHINE

Objetivo

Nesse exercício, criaremos um ambiente virtual para execução do Docker e veremos os principais comandos da Docker Machine.

Passo a passo

1. Abra o terminal, será através dele que trabalharemos com a Docker Machine. Para criar uma máquina, temos o comando `docker-machine create`:

   ```
   docker-machine create --driver virtualbox vm1
   ```

   Com o comando acima, criamos uma máquina virtual chamada `vm1`. Essa máquina utiliza o driver do `virtualbox`.

2. Com a máquina criada, podemos listar as máquinas gerenciadas pela Docker Machine. Para isso, temos o comando `docker-machine ls`.

3. Ainda precisamos acessar a máquina. O comando que permite essa conexão é o `docker-machine ssh`:

   ```
   docker-machine ssh vm1
   ```

Com isso, acessaremos o terminal dessa máquina virtual.

4. Veja que o Docker está instalado sem problemas. Execute o comando `docker -v`.
5. Aperte as teclas Ctrl+D para sair da máquina virtual.
6. Você pode parar a execução da máquina com o comando `docker-machine stop`. Execute o comando `docker-machine stop vm1` e veja que o status da máquina ao executar o comando `docker-machine ls` foi alterado.
7. Analogamente, inicie a máquina novamente com o comando `docker-machine start`. Não esqueça de passar o nome da máquina como parâmetro para o comando.
8. Por fim, remova a máquina com o comando `docker-machine rm vm1 --force`.

10.5 PARA SABER MAIS: DOCKER MACHINE E AWS

Vimos que o Docker Machine não se limita em criar ambientes locais, ele também possui *driver*s para criar ambientes na nuvem da AWS, Azure ou Digital Ocean. Nesse exemplo vamos focar na AWS, ou seja, vamos criar uma máquina virtual no serviço AWS EC2 usando Docker Machine.

O comando para criar a máquina é o mesmo, o que muda é o *driver*, além de algumas flags especificas da AWS EC2. Veja o exemplo:

```
docker-machine create --driver amazonec2 --amazonec2-open-port 8080 --amazonec2-region us-west-1 vm-n
o-ec2
```

Criamos uma máquina com o nome `vm-no-ec2` na região `us-west-1` abrindo a porta `8080`. Simles não? No entanto, tudo isso só funciona se você tem uma conta na AWS pois precisa das credenciais salvas na pasta `~/.aws/credentials`:

```
[default]
aws_access_key_id = SUA-ID
aws_secret_access_key = SUA-CHAVE
```

Vale informar que a AWS possui serviços gerenciados como Kubernetes ou Faregate para rodar Docker na nuvem que vão muito além de uma VM criada na EC2.

Mais informações na documentação do Docker:

- https://docs.docker.com/v17.12/machine/examples/aws/

10.6 PARA SABER MAIS: DOCKER MACHINE E WINDOWS

No início deste curso, foi mencionado que atualmente já existem os **Windows containers**, que rodam nativamente com o kernel do Windows. Porém, também conseguimos executar **Linux containers** em máquinas Windows sem quaisquer problemas. Isso é possível graças a Docker Machine! Com ela, quando executamos algum container linux em alguma máquina Windows, criamos uma pequena máquina virtual Linux, e dentro dessa máquina executamos o container.

Capítulo 11

GERENCIANDO CONTAINERS COM DOCKER SWARM

11.1 ESCALANDO A APLICAÇÃO COM CONCEITO DE MICROSERVICES

Já mencionamos a arquitetura baseada em *Microservice* que veio para resolver algumas questões da arquitetura *monolitica*. A ideia principal dessa arquitetura é dividir a aplicação em pequenos pedaços chamados de *services* que representam apenas uma parte do negócio. Cada *service* possui uma funcionalidade bem definida (*bounded context*) e roda como se fosse um "mini sistema" isolado.

A comunicação entre os services normalmente acontece a partir de uma API usando o protocolo HTTP e REST. Ou seja, em vez de termos uma única aplicação enorme, teremos várias instâncias menores que dividem e coordenam o trabalho. Essas instâncias são chamadas de Microservices.

Agora fica mais fácil monitorar cada serviço específico, descobrir gargalos, atualizá-lo ou escalá-lo pois a base de código é muito menor, e assim o deploy e o teste serão mais rápidos. Podemos agora achar soluções específicas para esse serviço sem precisar alterar os demais. Outra vantagem é que um desenvolvedor novo não precisa conhecer o sistema todo para alterar uma funcionalidade, basta ele focar na funcionalidade desse Microservice. Como uma desvantagem desse modelo podemos mencionar que agora temos a necessidade de gerenciar várias instâncias. Como o Docker poderia nos ajudar na hora de gerenciar diversos serviços?

11.2 DOCKER E MICROSERVICES

Trabalhar com uma arquitetura de Microservices gera a necessidade de publicar serviços de maneira rápida, leve, isolada e vimos que o Docker possui exatamente essas características! Com Docker e Docker Compose podemos criar um ambiente para a publicação destes serviços.

O Docker é uma ótima opção para rodar os Microservices pelo fato de empacotar a aplicação e rodar os containers. Essa utilização de containers para serviços individuais faz com que seja muito simples gerenciar e atualizar esses serviços, de maneira automatizada e rápida.

11.3 O DOCKER COMPOSE PARA GERENCIAR?

Ok, tudo bem até aqui. Agora vou ter vários serviços rodando usando o Docker. E para facilitar a criação desses containers já aprendemos usar o *Docker Compose* que sabe subir vários containers. O Docker Compose é a ótima ferramenta para simplificar a criação dos containers, no entanto ele não gerencia os containers! O que quer dizer com isso?

Imagina que nosso serviço rodando com Docker não atende mais a demanda e é preciso criar um segundo container a partir da mesma imagem para escalar um serviço horizontalmente. O *Docker Compose* consegue fazer isso? A resposta é não.

Imagina também que um container (talvez seja o único desse serviço) cai por um motivo ou fica indisponível por causa de uma falha na rede. O *Docker Compose* consegue monitorar os containers em questão e subir um novo caso aconteça uma falha? A resposta é não.

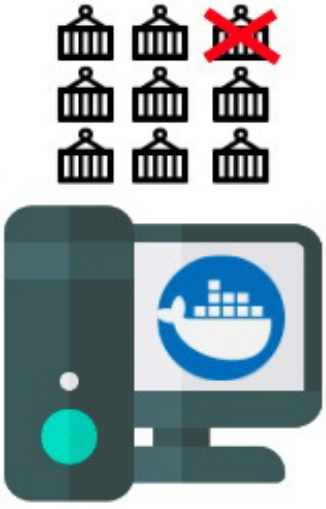


Figura 11.1: Falha na execução do Container

11.4 DOCKER SWARM - ORQUESTRANDO CONTAINERS

Para conseguir alta disponibilidade e escalabilidade, entre outros requisitos, é preciso usar uma ferramenta mais sofisticada, algo chamado de *cluster*, como podemos ver na página seguinte:

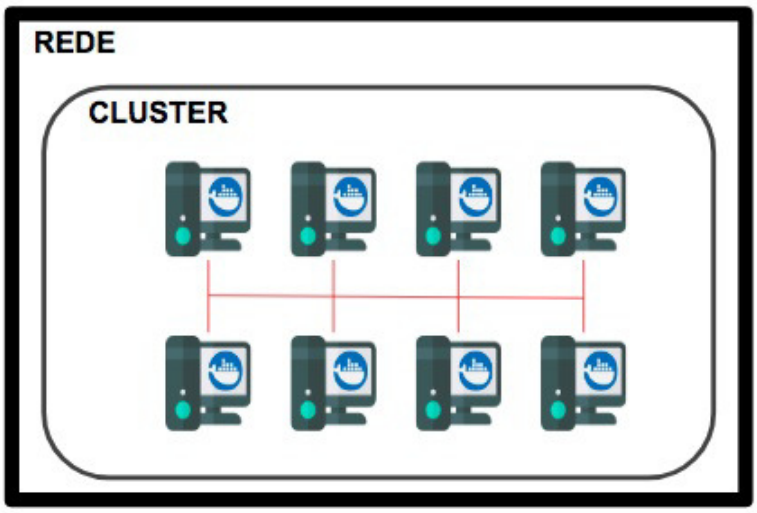


Figura 11.2: Cluster

Um *cluster* junta várias máquinas e cria uma *partição*. Um partição funciona como se fosse uma grande máquina que na verdade é baseado em várias pequenas. Juntas elas funcionam como se fosso uma coisa só. Para o usuário dessa partição não importa se tem 3 máquinas ou 50 máquinas ou *hosts* envolvidas. O cluster possui recursos para distribuir a carga, garantir a disponibilidade e monitorar as máquinas.

No mundo Docker temos uma solução da própria empresa para criar um cluster chamada de **Docker Swarm**. O nome *Swarm* remete o coletivo de animais no mundo real. No contexto de Docker teremos vários containers como coletivo.

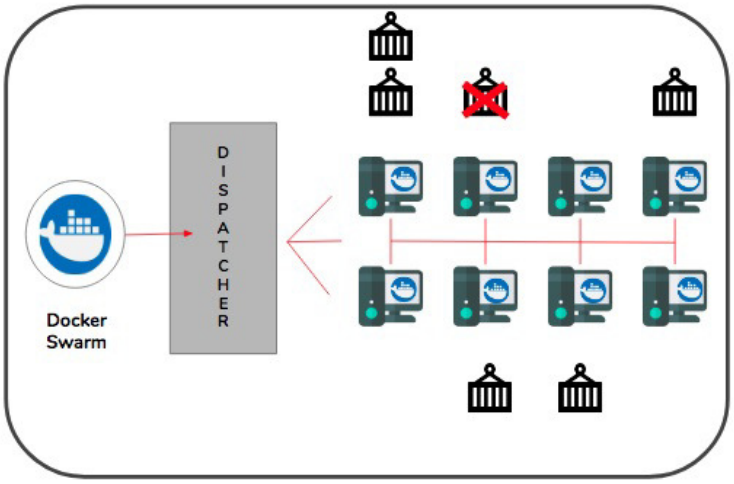


Figura 11.3: Docker Swarm

Resumindo, com Docker Swarm podemos criar um cluster de máquinas / hosts para rodar containers Docker. O Docker Swarm gerencia os containers nesse cluster. Isso significa que o Swarm vai distriuir, replicar e escalar os containers, se for necessário. Por causa dessas características o Docker Swarm é chamado de **Orquestrador de containers**.

11.5 CRIANDO NOSSO PRIMEIRO CLUSTER

Um cluster é compostos de várias máquinas ou *hosts* que podem ser máquinas virtuais. Vamos então usar o *Docker Machine* para criar essas máquinas que farão parte do cluster e vão rodar os containers Docker gerenciados pelo Swarm!

Já temos uma máquina criada na aula anterior:

```
docker-machine ls
NAME  ACTIVE   DRIVER       STATE     URL                         SWARM   DOCKER
vm1   -        virtualbox   Running   tcp://192.168.99.100:2376           v19.03.5
```

Se ela estiver parada (*Stopped*) basta rodar o comando `docker-machine start vm1` .

O próximo passo é definir que essa máquina faz parte do cluster, ou seja faz parte do *swarm*. Para tal devemos nos conectar com a máquina e rodar o comando para inicializar o cluster. Primeiro vamos nos conectar com a máquina-virtual:

```
docker-machine ssh vm1
   ( '>')
  /) TC (\   Core is distributed with ABSOLUTELY NO WARRANTY.
 (/-_--_-\)           www.tinycorelinux.net

docker@vm1:~$
```

E agora rodar o comando para inicializar o cluster com o comando `docker swarm init` . No entanto o comando precisa receber o endereço IP da máquina que fará parte do cluster. Assim fica:

```
docker@vm1:~$ docker swarm init --advertise-addr 192.168.99.100
Swarm initialized: current node (viswt8hhahrtf89l2f1ur9xsz) is now a manager.

To add a worker to this swarm, run the following command:

    docker swarm join --token SWMTKN-1-2j1jl5vw1xiu2ivntpq52bppl9rtaet5tnmhki0a5llzl6e1om-cfzwz55oi0b
y1grdr3udt14dd 192.168.99.100:2377

To add a manager to this swarm, run 'docker swarm join-token manager' and follow the instructions.
```

Repare que o comando `docker swarm init` criou um *manager*. Um *manager* é um líder das máquinas (*nós*) do cluster e possui algumas responsabilidades especiais.

Repare também o comando `docker swarm join` para juntar outras máquinas ao cluster. As outras máquinas (nós) podem ser também do tipo *manager* ou *worker*. Os *workers* realmente são os nós nós que executam os container (que rodam a aplicação / serviço). Os managers por sua vez tem um trabalho mais administrativo e ficam de "olho" nos *workers*.

Vamos então criar mais dois nós com docker machine que vão representar os *workers*. Na máquina *host* (ou seja, sai da máquina virtual com o comando `exit`) execute:

```
docker-machine create --driver=virtualbox vm2
Running pre-create checks...
Creating machine...
(vm2) Copying /Users/nico/.docker/machine/cache/boot2docker.iso to /Users/nico/.docker/machine/machin
es/vm2/boot2docker.iso...
(vm2) Creating VirtualBox VM...
(vm2) Creating SSH key...
...
```

e

```
docker-machine create --driver=virtualbox vm3
Running pre-create checks...
Creating machine...
(vm3) Copying /Users/nico/.docker/machine/cache/boot2docker.iso to /Users/nico/.docker/machine/machin
es/vm3/boot2docker.iso...
(vm3) Creating VirtualBox VM...
(vm3) Creating SSH key...
...
```

Listando todas as máquinas:

```
docker-machine ls
vm1  virtualbox   Running   tcp://192.168.99.101:2376  v19.03.5
vm2  virtualbox   Running   tcp://192.168.99.102:2376  v19.03.5
vm3  virtualbox   Running   tcp://192.168.99.103:2376  v19.03.5
```

Para `vm2` e `vm3` entrarem no cluster precisamos nos conectar a máquina e executar o comando `docker swarm join` passando o token quando definimos o manager:

```
docker-machine ssh vm2
   ( '>')
  /) TC (\   Core is distributed with ABSOLUTELY NO WARRANTY.
 (/-_--_-\)           www.tinycorelinux.net
docker@vm2:~docker swarm join --token SWMTKN-1-2j1jl5vw1xiu2ivntpq52bppl9rtaet5tnmhki0a5llzl6e1om-cfz
wz55oi0by1grdr3udt14dd 192.168.99.101:2377

This node joined a swarm as a worker.
```

Igualmente para a máquina `vm2` . Assim temos:

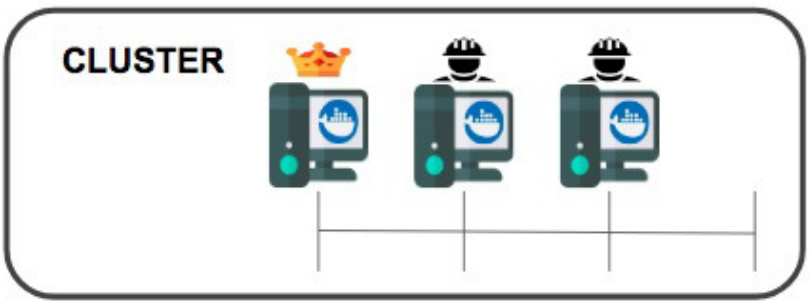


Figura 11.4: Cluster com Manager e 2 Workers

Mais informações na documentação:

- https://cursos.alura.com.br/course/docker-swarm-cluster-container/task/64504
- https://docs.docker.com/engine/swarm/
- https://docs.docker.com/engine/swarm/key-concepts/
- https://docs.docker.com/engine/swarm/swarm-mode/
- https://docs.docker.com/engine/swarm/join-nodes/

11.6 NÓS MANAGERS VS NÓS WORKERS

Uma vez conectada à VM *Manager* podemos listar todos os nós do swarm pelo comando `docker node ls`:

```
docker@vm1:~$ docker node ls
ID      HOSTNAME    STATUS     AVAILABILITY        MANAGER STATUS      ENGINE VERSION
viswt8 *   vm1      Ready      Active              Leader              19.03.5
rdy0rj     vm2      Ready      Active                                  19.03.5
tz6k5z     vm3      Ready      Active                                  19.03.5
```

Esse comando só funcionará no *Manager* indicado como *Leader* na saída. Em geral, o Manager gerencia o cluster e apenas esse tipo de nó pode ver e alterar o estado do cluster. Alterar nesse contexto é remover um nó do cluster pelo `docker node rm <id-do-no>`.

Importante é que os comandos para entrar no cluster (`docker swarm join`) e para sair (`docker swarm leave`) sempre devem ser executados no nó em questão. Ou seja, se a `vm3` deve sair do swarm, primeiro devemos nos conectar com a `vm3` e executar o comando `docker swarm leave <id-do-no>`, para depois remover o nó a partir do Manager com o comando `docker node rm <id-do-no>`.

Por fim podemos inspecionar um nó para receber mais detalhes do mesmo. Para tal usamos o comando:

```
docker node inspect vm2 --pretty
ID:            rdy0rjlbes67yeex1vwh9uuwo
Hostname:                  vm2
Joined at:                 2020-01-08 21:01:29.183797233 +0000 utc
Status:
 State:            Ready
 Availability:             Active
 Address:        192.168.99.102
Platform:
 Operating System:    linux
 Architecture:        x86_64
Resources:
 CPUs:            1
 Memory:        989.5Mi
....
```

Existem muito mais informações sobre esse nó, apresentamos apenas uma parte.

11.7 EXERCÍCIO: CRIANDO O PRIMEIRO SWARM.

Objetivo

Nesse exercício, criaremos o swarm e adicionaremos os primeiros nós. Além disso, utilizaremos a Docker Machine para criação das máquinas que executarão Docker isoladamente.

Passo a passo

1. Crie três máquinas virtuais chamadas **vm1**, **vm2** e **vm3** utilizando a Docker Machine.
2. No terminal da **vm1**. Acesse a máquina através do comando `docker-machine ssh vm1` e crie o swarm com o comando:

   ```
   docker swarm init --advertise-addr 192.168.99.100
   ```

3. Com o swarm criado, precisamos adicionar mais nós ao nosso swarm. Para isso, acessaremos o terminal de nossas **vm2** e **vm3** utilizando o comando `docker-machine ssh` e executaremos o seguinte comando:

   ```
   docker swarm join --token <TOKEN> 192.168.99.100:2377
   ```

   O comando acima será exibido logo após a criação do swarm.
4. Veja que com o comando anterior, adicionamos nós **workers** ao swarm. Execute o comando `docker node ls` no terminal da **vm1** e veja que os nós estão sendo listados. O que significa cada coluna dessa saída?
5. Tente executar o comando `docker node ls` nas **vm2** e **vm3**. O que aconteceu?
6. Caso tivéssemos perdido o token de acesso do passo 3, como poderíamos recuperá-lo? Execute o comando `docker swarm join-token worker` no terminal da **vm1** e veja que conseguimos recuperar o token.

11.8 ENTENDENDO O DRIVER DE REDE OVERLAY

Quando falamos sobre redes, já vimos dois tipos de drivers. O primeiro foi o driver `bridge` que permite a comunicação entre containers diferentes no mesmo host, e o segundo foi o driver `host` que permite comunicação entre container e host.

Agora, usando o Docker Swarm, temos vários hosts (nós) para se comunicar na rede. O que precisamos é um driver que permite uma comunicação entre containers que estão em host diferentes! Para tal existe um novo driver chamado de `overlay`.

Podemos ver esse driver ao acessar uma máquina virtual e listar as redes disponíveis:

```
docker@vm1:~$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
98980f5fd88c        bridge              bridge              local
a9ddedef9122        docker_gwbridge     bridge              local
3a03ecbc8577        host                host                local
wpcr3z62mzke        ingress             overlay             swarm
a6163f1a69bd        none                null                local
```

Repare o novo driver `overlay` que fica no escopo `swarm`. O interessante é que todos as máquinas desse swarm também terão essa rede com a mesma ID. Abaixo tem a saída do comando da segunda máquina virtual:

```
docker@vm2:~$ docker network ls
NETWORK ID          NAME                DRIVER              SCOPE
...
wpcr3z62mzke        ingress             overlay             swarm
...
```

Isso significa que a `vm1` e `vm2` estão dentro da mesma rede com o nome `ingress` e driver `overlay`. Mas o que significa isso a respeito dos nosso containers? Significa que eles podem chamar um ao outro pela IP E pelo nome, já que todos estão dentro dessa rede! Além disso, a comunicação é segura e criptografada por padrão.

Resumindo, um swarm é um cluster que possui vários hosts. Essas hosts são máquinas físicas ou virtuais que se conhecem e fazem parte da mesma rede *overlay*. Algumas máquinas funcionam como *manager* e outros como *workers*. A responsabilidade do manager é administrar e cuidar do estado do cluster. É o manager que vai atribuir o trabalho para um *worker*. O *worker* é o nó que realmente executa o container.

11.9 SUBINDO CONTAINER COMO SERVIÇO

Temos um cluster de vários nós que podem rodar containers docker mas até agora não rodamos nenhum container! A primeira tentativa seria usar o já conhecido comando `docker container run` para subir um container dentro do cluster. No entanto esse comando apenas sobe um container no escopo *local* que não se integra no cluster. Ou seja, se o container cai por causa de um problema, o Docker Swarm não garante a alta-disponibilidade nem escalabilidade.

Para subir um container no escopo *swarm* devemos usar o comando `docker service` executado no *manager*, pois é ele quem cuida o estado do cluster:

```
docker@vm1:~$ docker service create -p 8080:80 nginx
ooqtxgq8ti949pjagg02jpis2
overall progress: 1 out of 1 tasks
1/1: running
verify: Service converged
```

O container agora é considerado um *service* no cluster e por isso ele é conhecido e acessível em outros nós. Repare a mensagem `verify: Service converged` que demora um pouco para aparecer

justamente por causa da comunicação entre os nós do cluster.

Para listar, remover e inspecionar serviços não tem novidades, são os comandos `docker service ls`, `docker service rm` e `docker service inspect`, por exemplo:

```
docker@vm1:~$ docker service ls

ID            NAME            MODE        REPLICAS  IMAGE         PORTS
ooqtxgq8ti94  youthful_lewin  replicated  1/1       nginx:latest  *:8080->8080/tcp
```

Mas no final em qual nó o container foi criado? Lembrando que o nó do tipo *worker* possui esse papel. Para descobrir isso devemos usar o comando `docker service ps`:

```
docker service ps ooqt
ID                  NAME                IMAGE               NODE
3ozxhkz5kz4u        kind_tesla.1        nginx:latest        vm2

DESIRED STATE       CURRENT STATE           ERROR               PORTS
Running             Running 5 minutes ago
```

Repare que o container roda no host `vm2` mas agora no escopo *swarm*!

11.10 BALANCEANDO CARGAS ENTRE CONTAINERS

Vimos na seção anterior que o container foi alocado na `vm2`, mas como ele roda no escopo *swarm*, podemos acessar o service a partir de qualquer IP do cluster. Lembrando que temos 3 máquinas criadas com `docker-machine`.

Listando todas as máquinas:

```
docker-machine ls
vm1  virtualbox   Running   tcp://192.168.99.101:2376  v19.03.5
vm2  virtualbox   Running   tcp://192.168.99.102:2376  v19.03.5
vm3  virtualbox   Running   tcp://192.168.99.103:2376  v19.03.5
```

Pode ser usado uma dessas IPs para acessar o servidor *nginx* que mapeamos para porta `8080`, por exemplo a IP do Manager: `http://192.168.99.101:8080`

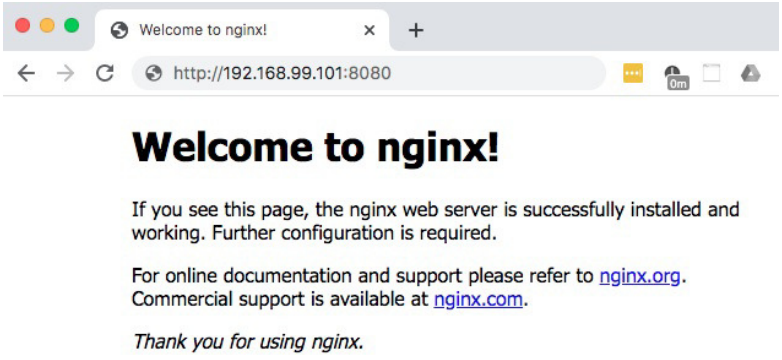


Figura 11.5: Home do Nginx

Essa porta agora é exclusiva desse serviço e não pode ser utilizada por qualquer outro serviço dentro desse cluster. Quando criamos um service, a porta é compartilhada entre nós do swarm e é acessível a partir de qualquer nó graças um conceito chamado de **routing mesh**. As chamadas de rede para o swarm são roteadas automaticamente até encontrar o nó correto que realmente roda o container!

11.11 EXERCÍCIO: AVANÇANDO COM CONCEITOS DO SWARM

Objetivo

Nesse exercício, implementaremos o caso anterior com a aplicação Java e o banco de dados. Porém, dessa vez no contexto do swarm.

Passo a passo

1. Como vimos anteriormente, possuímos diferentes *drivers* de rede no Docker. Criaremos agora uma rede específica no escopo do swarm, utilizando o comando dentro da **vm1**:

   ```
   docker network create --driver overlay my_overlay
   ```

2. Agora, criaremos o primeiro container no contexto do swarm. Para isso, utilizaremos a imagem `appjava:2.0` . Execute dentro da **vm1** os comandos:

   ```
   docker image pull <SEU_USUARIO_DOCKERHUB>/appjava:2.0
   ```

   E em seguida

   ```
   docker service create --name java --network my_overlay -p 9000:8080 <SEU_USUARIO_DOCKERHUB>/appja
   va:2.0
   ```

3. Execute o comando `docker service ls` e veja que podemos listar os serviços que estão em execução.

4. Podemos também ver onde a tarefa gerada pelo serviço foi alocada. Para isso, podemos utilizar o comando `docker service ps java` .

5. Agora precisamos rodar o serviço com a imagem do `mysql:5.7` . Analogamente ao passo 2, execute:

   ```
   docker image pull mysql:5.7
   ```

   Porém, dessa vez queremos restringir em qual nó o serviço será executado. Podemos fazer isso através da flag `--constraint` .

   ```
   docker service create --name db --network my_overlay --constraint 'node.hostname == vm3' -e MYSQL
   _ALLOW_EMPTY_PASSWORD=yes mysql:5.7
   ```

   Com isso, definimos que o serviço será criado na `vm3` .

6. Dentro do terminal da **vm1**, execute o comando `ifconfig` para descobrir o **ip** de sua máquina virtual. Descubra vendo o ip listado em **eth1**.

7. Em seu navegador, acesse `<IP_VM1>:9000/geraBancoV2` e veja que tudo continua funcionando normalmente. Porém em um ambiente distribuído agora.

8. Dentro do nó que está rodando a tarefa gerada pelo serviço `java`, execute o comando `docker container rm $(docker container ls -q) --force` para remover o container.

9. Em seu navegador, tente algumas vezes acessar o link `<IP_VM1>:9000`. O que aconteceu?

10. O que aconteceria se tentássemos acessar a porta *9000* utilizando o *ip* de outras máquinas virtuais do swarm?

11.12 SERVIÇOS REPLICADOS VS SERVIÇOS GLOBAIS

No escopo do swarm, possuímos diferentes tipos de serviços, eles podem ser globais, ou seja, executam exatamente uma tarefa em todos os nós do swarm, ou podem ser replicados, possuindo uma ou mais tarefas em determinados nós. Por padrão, serviços são criados no modo replicado com apenas uma réplica, alocando uma tarefa aleatoriamente em um nó do swarm.

Podemos definir o modo de um serviço no momento de sua criação através da flag **--mode**, veja o exemplo abaixo:

Executando o comando `docker service create --mode global nginx`, criamos um serviço em modo global. Listando o serviço, podemos ver o resultado:



Figura 11.6: Global

Mais uma vez, isso significa que o serviço terá exatamente uma réplica sendo executada em cada nó do swarm. Bons candidatos para serviços globais podem ser serviços de monitoramento ou segurança.

Serviços replicados são o padrão da execução de serviços no Docker Swarm. Caso seja executado um comando como `docker service create nginx`, criaremos o serviço utilizando a imagem do **nginx** com apenas uma réplica em um nó aleatório em nosso swarm. Podemos definir também o modo de maneira mais clara utilizando a flag **--mode**.

O comando `docker service create --mode replicated nginx` produzirá o seguinte resultado:

```
caelum@caelum-sl2-mq6:~$ docker service ls
ID                  NAME                MODE                REPLICAS            IMAGE               PORTS
7bkrap3qc87o        gracious_chaum      replicated          1/1                 nginx:latest
caelum@caelum-sl2-mq6:~$ docker service ps 7b
ID                  NAME                IMAGE               NODE                DESIRED STATE       CURRENT STATE            ERROR
   PORTS
o96srmiebzyy        gracious_chaum.1    nginx:latest        caelum-sl2-mq6      Running             Running 12 seconds ago
```

Figura 11.7: Replicated

Há a possibilidade de alterarmos o número de replicas de um serviço já em execução. Para isso, podemos utilizar os comandos `docker service update --replicas 5 gracious_chaum` ou `docker service scale gracious_chaum=5` . Os dois comandos possuem o mesmo resultado, porém o segundo é uma maneira mais prática e simples de executar.

```
caelum@caelum-sl2-mq6:~$ docker service update --replicas 5 gracious_chaum
gracious_chaum
overall progress: 5 out of 5 tasks
1/5: running   [==================================================>]
2/5: running   [==================================================>]
3/5: running   [==================================================>]
4/5: running   [==================================================>]
5/5: running   [==================================================>]
verify: Service converged
caelum@caelum-sl2-mq6:~$ docker service ps 7b
ID                  NAME                IMAGE               NODE                DESIRED STATE       CURRENT STATE            ERROR
   PORTS
o96srmiebzyy        gracious_chaum.1    nginx:latest        caelum-sl2-mq6      Running             Running 5 minutes ago

aqq9150h0nv3        gracious_chaum.2    nginx:latest        caelum-sl2-mq6      Running             Running 12 seconds ago

l5ulwmrthxol        gracious_chaum.3    nginx:latest        caelum-sl2-mq6      Running             Running 11 seconds ago

lhdgjjf32oev        gracious_chaum.4    nginx:latest        caelum-sl2-mq6      Running             Running 12 seconds ago

00rbt8bcqo0g        gracious_chaum.5    nginx:latest        caelum-sl2-mq6      Running             Running 12 seconds ago
```

Figura 11.8: Update

11.13 IMPLANTANDO MÚLTIPLOS SERVIÇOS COM STACKS

Analogamente ao Docker em sua versão *single host*, também possuímos uma maneira de utilizar arquivos de composição para criar múltipls serviços de uma só vez. A partir da versão **3**, o docker-compose.yml também pode ser utilizado para serviços de um swarm. Além disso, diversas novas instruções relacionadas diretamente ao escopo do swarm foram adicionadas.

Em poucas palavras, uma stack nada mais é do que uma pilha de serviços trabalhando em conjunto. Com o Docker, temos capacidade de criar e manipular diversas stacks de uma só vez.

Algumas novas instruções da versão 3

Como dito, a partir da versão **3**, novas instruções foram adicionadas para que pudéssemos trabalhar com o swarm de maneira mais integrada. Abaixo temos algumas delas e suas respectivas funcionalidades.

- deploy - Com essa instrução, podemos definir diversas outras instruções que tem total relacionamento com o **deploy** de determinado serviço, como política de reinício, se um serviço será replicado ou global, número de réplicas de um serviço, dentre diversas outras coisas.

- mode - Essa instrução é uma sub instrução de deploy. Com ela, podemos definir qual será o modo de execução do serviço, global ou replicado.
- replicas - Caso o serviço seja replicado, podemos definir o número de réplicas que queremos em execução.
- restart_policy - Podemos definir qual será a condição que fará o serviço reiniciar, seja uma falha ou qualquer outra coisa. Dentro dela podemos adicionar outras opções, como número de tentativas, tempo de espera, condição, etc.
- placement - Podemos definir quais preferências e restrições de alocação colocaremos em nosso serviço.

O corpo de um arquivo de composição

O funcionamento e formatação de um arquivo de composição segue exatamente a mesma regra de um arquivo `YAML` usando o docker compose. Abaixo temos um exemplo de um possível arquivo de composição válido:

```
version: "3.7"
services:
  redis:
    image: redis:alpine
    deploy:
      replicas: 6
      update_config:
        delay: 10s
      restart_policy:
        condition: on-failure
```

No exemplo acima, é criado um serviço chamado **redis** utilizando a imagem do `redis:alpine` como base. Além disso, foi definido que o serviço possuirá 6 réplicas em funcionamento, que terá um delay de 10 segundos para efetuar atualizações e que será reiniciado em caso de falhas.

Criando a stack a partir do arquivo de composição

Precisamos agora de um comando capaz de criar nossa pilha de serviços através do terminal. Para isso, podemos contar com o comando `docker stack` . Basta informar qual será o arquivo de composição utilizado e o nome da stack gerada:

```
docker stack deploy --compose-file docker-stack.yml my-stack
```

Neste caso, criaremos uma stack chamada **my-stack** a partir do arquivo chamado **docker-stack.yml** (o nome do arquivo pode ser qualquer um).

Também podemos consultar e listar quais stacks temos em execução no momento com o comando `docker stack ls` . Podemos ver na página seguinte:

```
caelum@caelum-sl2-mq6:~/stack$ docker stack deploy --compose-file docker-stack.yml my-stack
Creating network my-stack_default
Creating service my-stack_redis
caelum@caelum-sl2-mq6:~/stack$ docker stack ls
NAME                SERVICES            ORCHESTRATOR
my-stack            1                   Swarm
caelum@caelum-sl2-mq6:~/stack$
```

Figura 11.9: stack ls

Podemos também remover stacks através do comando `docker stack rm`.

11.14 PARA SABER MAIS: KUBERNETES

Existe existe um outro orquestrador de containers chamado de **Kubernetes**. O *Kubernetes* é uma alternativa ao Docker Swarm, são concorrentes de certa forma, ambos com a mesma responsabilidade de *orquestrar containers*.

A comparação entre eles é algo complexo e talvez injusta, mas em geral o Docker Swarm tende ser mais simples e leve do que o Kubernetes. Isto é pois o Docker Swarm aproveita vários comandos e configurações do Docker. Aprendendo o Docker você também já conhece uma parte do Docker Swarm.

O Kubernetes por sua vez tem uma curva de aprendizagem mais íngreme, define outras ferramentas como o *kubectl* como cliente e *kubeadm* para administração, além de vários outros arquivos de configuração. No entanto, o Kubernetes possui uma comunidade enorme e muitas ferramentas ao redor, além do amplo uso pelos provedores de nuvem como a Amazon, Microsoft Azure ou Google. O Kubernetes é baseado na experiencia do Google e se tornou um projeto opensource que ajudou muito na popularidade e evolução da ferramenta.

11.15 PARA SABER MAIS: VOLUMES NO SWARM

Por padrão, tanto o **Docker** no modo standalone quanto o **Docker Swarm**, partilham apenas de um driver **local** para uso de volumes. Isso quer dizer que o **Docker Swarm** não possui, até então, solução nativa para distribuir volumes entre os nós.

Então, no exemplo do exercício anterior, ao definirmos o volume para o serviço `db` criamos um volume local dentro de cada nó que executou a tarefa. Logo, os volumes **não** são compartilhados entre os diferentes nós do cluster.

Existem soluções que não são nativas do Docker Swarm para utilizar volumes distribuídos entre nós, que podem ser consultadas na Docker Store:

- https://store.docker.com/search?category=volume&q=&type=plugin

11.16 DESAFIO DOCKER SWARM

Nesse desafio continuaremos o trabalho desenvolvido no capítulo 9, **Desafio Docker Compose**, e expandiremos o cenário da aplicação de votação para um cluster de aplicações utilizando o Docker Swarm. É recomendando que você tenha terminado o desafio anterior antes de resolver esse aqui. Caso tenha tido problemas em resolver ele converse com o seu instrutor.

O desafio consistirá em adaptar o `docker-compose.yml` escrito no desafio anterior para que as aplicações rodem dentro de um Swarm. Iremos utilizar cinco máquinas virtuais, crie cada uma delas com o comando `docker-machine create --driver=<driver> <nome>` , nomeando elas da seguinte forma:

- manager1
- manager2
- manager3
- worker1
- worker2

A aplicação de votação é composta de pequenos serviços, o `voting-app` , o `worker` , o `result-app` , um bando de dados chave-valor Redis e um banco de dados relacional PostgreSQL. Além desses adicionaremos um novo serviço que será responsável por monitor o nosso cluster, o visualizer:

https://hub.docker.com/r/dockersamples/visualizer

Os serviços continuarão funcionando da mesma forma que foi explicado no capítulo 9, a principal diferença será no processo de orquestração dos contrainers. A seguir detalharemos como cada seviço deverá ser orquestrado, em que nós executarão, quantas réplicas eles terão, quais políticas de atualização e recuperação a falha utilizaremos e etc.

Detalhes adicionais

Serviço voting-app

1. Teremos 2 réplicas em execução, no modo replicated
2. Substitua o bind mount `/vote:/app` por um mount do tipo volume com nome `vote-data`
3. A política de atualização será:
    i. parallelism igual a 2
4. A política de restart será:
    i. on-failure
5. Deverá ser acessível através do endereço http://localhost:5000

Serviço worker

1. Teremos 2 réplica em execução, no modo replicated
2. Ela terá uma label com o conteudo `[APP=VOTING]`

3. A política de restart será:
    i. on-failure
    ii. com delay de 10s
    iii. tentativas máximas igual a 3
    iv. janela de 120s
4. Executará apenas em nós do tipo worker

Serviço redis

1. Teremos 1 réplica em execução, no modo replicated
2. A política de atualização será:
    i. parallelism igual a 2
    ii. delay de 10s
3. A política de restart será:
    i. on-failure

Serviço db

1. Teremos 1 réplica em execução, no modo replicated
2. Executará apenas em nós do tipo manager

Serviço result-app

1. Teremos 1 réplica em execução, no modo replicated
2. Substitua o bind mount `/result:/app` por um mount do tipo volume com nome `result-data`
3. A política de atualização será:
    i. parallelism igual a 2
    ii. delay de 10s
4. A política de restart será:
    i. on-failure
5. Executará apenas em nós do tipo worker
6. Deverá ser acessível através do endereço http://localhost:5001

Serviço visualizer

1. Será executado no modo global
2. Terá stop grace period de 1m30s
3. Utilizará a porta 8080 externamente, mapeada para 8080 do serviço
4. Fazer um bind mount entre `/var/run/docker.sock e var/run/docker.sock`
5. Deverá ser acessível através do endereço http://localhost:8080

Crie um novo arquivo `docker-stack.yml` a partir do arquivo `docker-compose.yml` , fazendo as

adaptações necessárias para o processo de orquestração. Note que algumas palavras chaves do Docker compose não funcionam quando estamos fazendo deploy de uma stack de aplicações, através do comando `docker stack deploy`. Uma dessas palavras chaves é a `build`, utilizada para transformar o Dockerfile em uma imagem. Será preciso que essa imagem já exista em algum lugar e seja baixada pelo arquivo compose.

Quando você terminar de escrever o seu `docker-stack.yml` será necessário copiar esse arquivo para dentro do cluster. Existem diversas formas de como fazer isso, fica a seu critério escolher a forma que você julga mais fácil. Dica: procure pelo comando `docker-machine scp`.

Tente não cair na tentanção de procurar a solução pronta na internet. O objetivo aqui é validar se você conseguiu aprender os conceitos passados durante a aula. Caso tenha alguma dúvidas pergunte ao instrutor, e não deixe de consultar também a documentação oficial do Docker Compose:

https://docs.docker.com/compose/compose-file/

@NOTE

A solução desse desafio encontra-se na branch `desafio-docker-stack`, no arquivo `docker-stack.yml`:

https://github.com/fllsouto/example-voting-app/blob/desafio-docker-stack/docker-stack.yml

Tive que fazer o build das imagens utilizando o comando `docker-compose -f <arquivo.yml> build` e subi para minha conta pessoal. Os alunos podem fazer o mesmo ou utilizar minhas imagens. Caso eles optem pela segunda opção as minhas imagens são:

- soutoxfll/example-voting-app_vote
- soutoxfll/example-voting-app_worker
- soutoxfll/example-voting-app_result

Após subir as imagens e escrever o arquivo `docker-stack.yml` será necessário criar as máquinas, adicionar elas ao swarm e copiar o arquivo para o manager para que ele faça deploy dos serviços.

Eu posso fazer o clone do projeto se ele estiver no github, ou copiar para dentro da máquina com o `docker-machine scp` e depois descompactar:

```
$ tar -vczf example.tag.gz example-voting-app
# vai compactar o arquivo

$ docker-machine scp example.tag.gz docker@manager1:/home/docker
example.tag.gz
                                      100% 1143KB  84.4MB/s   00:00

$ docker-machine ssh manager1
   ( '>')
  /) TC (\   Core is distributed with ABSOLUTELY NO WARRANTY.
 (/-_--_-\)           www.tinycorelinux.net

docker@manager1:~$ ls
example.tag.gz

docker@manager1:~$ tar -xvzf example.tar.gz

docker@manager1:~$ ls
example-voting-app  example.tar.gz
```

Depois é só executar a criação da Stack no cluster com o comando `docker stack deploy`